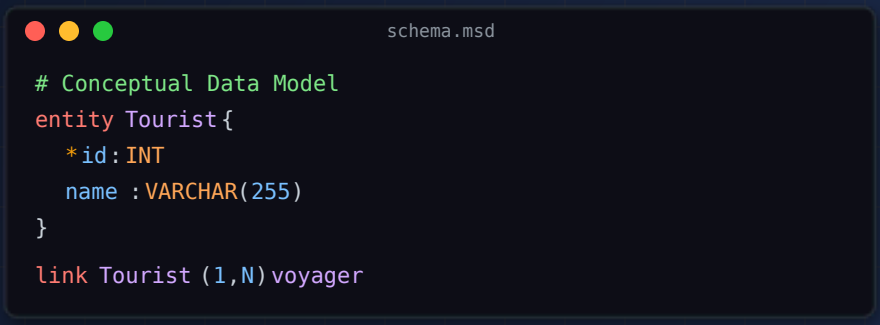

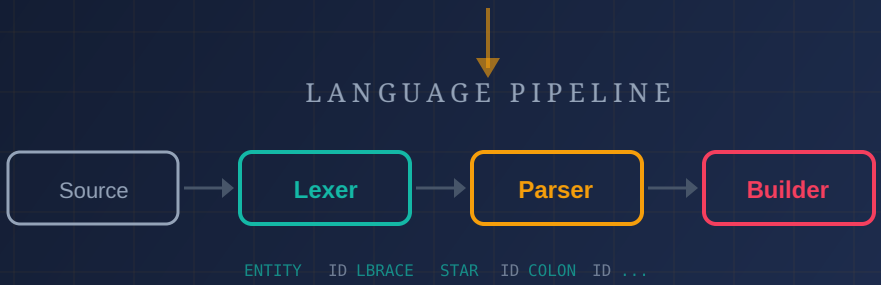

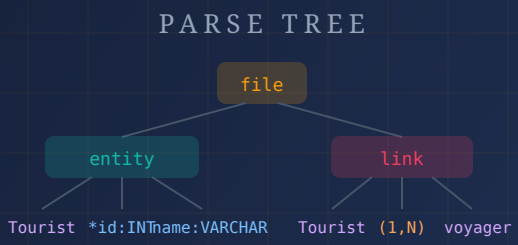

Designing Domain-Specific Languages

From Theory to Implementation

Lexing | Parsing | Semantics | Code Gen | Testing

ACHRAF SOLTANI

www.achrafsoltani.com

Designing Domain-Specific Languages

From Theory to Implementation

Achraf SOLTANI

2026

Contents

Preface **13**
Why Domain-Specific Languages? . . . 13
What You'll Learn . . . 13
The Case Study: MSD and Merisio . . . 13
Prerequisites . . . 14
How to Use This Book . . . 14
Conventions . . . 15
Acknowledgements . . . 15
Let's Begin . . . 16

Table of Contents **17**
Part I: Foundations . . . 17
Part II: Language Design . . . 17
Part III: Implementation . . . 17
Part IV: Quality and Integration . . . 18
Appendices . . . 18

Chapter 1: Introduction to Domain-Specific Languages **19**
1.1 What Is a Domain-Specific Language? . . . 19
1.2 General-Purpose vs Domain-Specific . . . 20
When to use which . . . 21
1.3 Internal vs External DSLs . . . 21
Internal DSLs . . . 21
External DSLs . . . 22
Comparing the two approaches . . . 22
1.4 When to Build a DSL (and When Not To) . . . 24
Good reasons to build a DSL . . . 24
Bad reasons to build a DSL . . . 24
A decision framework . . . 25
1.5 The Language Spectrum . . . 26
The spectrum . . . 26
Where MSD fits . . . 27
1.6 Real-World Impact . . . 27
SQL: Transforming database interaction . . . 28
CSS: Separating content from presentation . . . 28
Regular expressions: Compact pattern specification . . . 28
Make: Declarative build orchestration . . . 28
Dockerfile: Reproducible container definitions . . . 29
Common success factors . . . 29
1.7 The MSD Case Study . . . 29
What MSD does . . . 29

Why MSD makes a good case study . . . 30
What we shall build . . . 31
1.8 Exercises . . . 31

Chapter 2: The DSL Landscape **33**
2.1 A Survey of External DSLs . . . 33
SQL . . . 33
GraphQL . . . 33
Terraform HCL . . . 34
Dockerfile . . . 34
Makefile . . . 34
CSS Selectors . . . 35
2.2 Classification by Paradigm . . . 36
Declarative DSLs . . . 36
Imperative DSLs . . . 36
Hybrid DSLs . . . 36
Paradigm Comparison . . . 37
2.3 Classification by Complexity . . . 37
Micro-DSLs . . . 37
Medium DSLs . . . 38
Full DSLs . . . 38
Complexity Comparison . . . 38
2.4 Common Patterns Across DSLs . . . 39
Block Structure . . . 39
References . . . 40
Types . . . 40
Constraints . . . 40
Comments . . . 41
Metadata . . . 41
2.5 What Makes a DSL Succeed? . . . 41
Domain Fit . . . 41
Learnability . . . 42
Tool Support . . . 42
Community Adoption . . . 42
Interoperability . . . 42
2.6 Common DSL Anti-Patterns . . . 43
The “Kitchen Sink” . . . 43
The “One Weird Trick” . . . 43
The “Island Language” . . . 43
The “Error Desert” . . . 43
2.7 Exercises . . . 44
Summary . . . 45

Chapter 3: Requirements and Domain Analysis **47**
3.1 Understanding the Target Domain . . . 47
Why Domain Analysis Must Precede Language Design . . . 47
The Danger of Designing Syntax First . . . 47
Working with Domain Experts vs Working Alone . . . 48

Domain Modelling as the Foundation for Language Constructs 48
3.2 Identifying Domain Concepts 49
Extracting Nouns (Entities and Objects) 49
Extracting Verbs (Operations and Relationships) 49
Extracting Adjectives (Properties, Attributes, and Constraints) 49
Creating a Domain Vocabulary 50
3.3 Relationships and Constraints 51
How Domain Concepts Relate to Each Other 51
Cardinality of Relationships 51
Constraints That Restrict Valid Combinations 52
Which Constraints Belong in the Language vs in the Tooling? 52
3.4 User Personas and Workflow Analysis 53
Who Will Use the DSL? 53
What Tasks Will They Perform? 53
What Tools Do They Already Use? 54
Example: Merisio User Personas 54
3.5 From Domain Model to Language Constructs 55
The Mapping Principles 55
Design Table: Domain Concept to Language Construct 55
Resolving Design Tensions 56
3.6 Case Study: The MERISE Modelling Domain 56
What Is MERISE? 56
The Three Abstraction Levels 56
The MCD Level in Detail 57
Why This Domain Is Well-Suited for a DSL 57
Existing Tools 58
What Merisio Needed 58
3.7 Exercises 59

Chapter 4: Lexical Design **60**
4.1 Tokens as the Atoms of a Language 60
4.2 Keywords 61
How Many Keywords? 61
Keyword Reservation 62
Case Sensitivity for Keywords 62
Choosing Keywords That Read Naturally 63
4.3 Identifiers 63
Character Set Rules 63
Case Sensitivity for Identifiers 64
Unicode Identifiers 64
Naming Conventions 65
4.4 Literals 65
Integer Literals 65
String Literals 65
When Does a DSL Need Literals? 66
4.5 Symbols and Punctuation 66
Single-Character Tokens 67
Multi-Character Tokens 67

Choosing Symbols: Familiarity vs Domain Conventions 68
4.6 Whitespace and Indentation 69
Insignificant Whitespace 69
Significant Whitespace 69
Comparing the Two Approaches 69
The MSD Choice 70
4.7 Comments 70
Line Comments 70
Block Comments 71
Documentation Comments 72
4.8 Context-Sensitive Lexing 72
The Problem 72
The Solution 73
Keep Context Sensitivity Minimal 73
4.9 Design Trade-Offs 74
Familiarity vs Precision 74
Brevity vs Readability 74
Consistency vs Convenience 74
Summary of MSD's Lexical Choices 75
4.10 Exercises 75

Chapter 5: Syntactic Design **78**
5.1 From Tokens to Structure 78
5.2 EBNF Notation 79
Notation Reference 79
Example: An Entity Block in MSD 79
Writing EBNF Productions 80
Why Formal Grammars Matter 80
5.3 LL(1) Grammars and Why They Matter 81
What LL(1) Means 81
Why LL(1) Is Ideal for Hand-Written Parsers 81
Checking LL(1): FIRST and FOLLOW Sets 81
When a Grammar Is Not LL(1) 82
Transforming Non-LL(1) Grammars 82
5.4 Block-Structured Syntax 83
Curly-Brace Blocks 83
Indentation-Based Blocks 84
Keyword-Based Blocks 84
MSD's Choice: Curly Braces 84
5.5 Line-Oriented vs Expression-Based Syntax 85
Line-Oriented Syntax 85
Expression-Based Syntax 85
MSD's Approach 86
5.6 Nesting, Grouping, and Delimiters 86
Nesting Depth 86
Grouping with Parentheses 87
Delimiter Choices 87
5.7 Ambiguity and How to Avoid It 88

What Syntactic Ambiguity Means . . . 88
The Classic Example: Dangling Else . . . 88
How Ambiguity Arises in DSLs . . . 88
Strategies for Avoiding Ambiguity . . . 89
MSD's Ambiguity-Free Design . . . 89
5.8 Designing for Readability vs Parseability . . . 90
The Tension . . . 90
Case Study: MSD Cardinalities . . . 90
The 80/20 Rule . . . 91
Trade-offs in MSD . . . 91
5.9 Exercises . . . 91
Summary . . . 93

Chapter 6: Semantic Design **94**
6.1 Beyond Syntax: What Makes a Program Meaningful . . . 94
6.2 Type Systems for DSLs . . . 95
Why DSLs Need Types . . . 95
Simple Types . . . 95
Enumerated Types . . . 96
Parameterised Types . . . 96
When to Validate Types . . . 96
6.3 Name Binding and Scoping Rules . . . 97
Names Are the Glue . . . 97
Flat Scope . . . 97
Block Scope and Lexical Scope . . . 97
MSD's Scoping Model . . . 98
6.4 Reference Resolution and Forward References . . . 98
What Is a Reference? . . . 98
The Forward Reference Problem . . . 99
Resolution in Practice . . . 100
6.5 Uniqueness and Conflict Rules . . . 100
Duplicate Declarations . . . 100
Cross-Namespace Conflicts . . . 101
6.6 Validation Rules and Constraint Checking . . . 102
Beyond the Grammar . . . 102
MSD's Semantic Validations . . . 102
Hard Constraints vs Soft Constraints . . . 103
The Error Budget . . . 103
6.7 Error Messages as a Design Concern . . . 103
Error Messages Are User Interface . . . 103
Principles of Good Error Messages . . . 103
Anti-Patterns . . . 104
MSD's Error Message Design . . . 104
When to Stop Processing . . . 105
6.8 Exercises . . . 106

Chapter 7: Designing MSD **108**
7.1 Applying the Design Process . . . 108

7.2 Domain Analysis: MERISE Conceptual Data Modelling 109
The Domain 109
Domain Vocabulary Table 109
Domain Constraints 110
7.3 Lexical Choices 110
Keywords 111
Case-Insensitive Keywords 111
Case-Sensitive Identifiers 111
The * Prefix for Primary Keys 111
Comment Syntax 112
Context-Sensitive Colon 112
Token Summary 113
7.4 Syntactic Choices 113
Block Structure 113
Attribute Syntax 114
Link Syntax 114
No Commas, No Semicolons 114
The Project Block 115
Complete Grammar in EBNF 115
7.5 Semantic Choices 116
Data Types 116
Cardinality Constraints 117
Uniqueness Rules 117
Forward References 117
Primary Key Enforcement 118
Error Suggestions via Levenshtein Distance 118
7.6 The Complete MSD Specification 119
7.7 Comparison with Mocodo 120
Syntax Comparison 121
Feature Comparison 121
Why MSD Made Different Choices 122
7.8 Exercises 122

Chapter 8: Building a Lexer **125**
8.1 General Lexer Architecture 125
8.2 Token Representation 126
Token type design 127
8.3 Line-by-Line vs Character-by-Character Scanning 128
Character-by-character scanning 128
Line-by-line scanning 128
8.4 Keyword Recognition via Identifier-to-Keyword Mapping 130
8.5 Handling Context Sensitivity 131
The problem 131
The solution 132
8.6 Error Detection and Reporting at the Lexical Level 133
8.7 MSD Lexer: Complete Walkthrough 134
Token trace 135
Context-sensitive trace 137

8.8 Design Decisions . . . 138
Why line-by-line scanning? . . . 138
Why not regular expressions? . . . 138
Why emit NEWLINE tokens? . . . 138
Why track column numbers? . . . 139
8.9 Exercises . . . 139

Chapter 9: Building a Recursive Descent Parser 141
9.1 The Recursive Descent Technique . . . 141
One Function Per Grammar Production . . . 141
Top-Down Parsing . . . 142
Why Recursive Descent Is Popular . . . 143
9.2 Intermediate Representations . . . 144
Why Not Build the Final Output Directly? . . . 144
The Parsed Dataclasses . . . 145
ParseResult: The Container . . . 146
9.3 Token Stream Navigation . . . 147
`_peek()`: Look Without Consuming . . . 147
`_advance()`: Consume and Return . . . 147
`_check(*types)`: Test Without Consuming . . . 147
`_expect(*types)`: Consume or Fail . . . 147
`_skip_newlines()`: Consume Whitespace . . . 148
`_at_end()`: Check for EOF . . . 148
9.4 Parsing Blocks and Attributes . . . 148
Entity Blocks . . . 149
Attribute Parsing . . . 150
Type Expressions . . . 150
Association Blocks . . . 151
9.5 Parsing Statements . . . 152
Link Statements . . . 152
Cardinality Parsing . . . 153
9.6 Error Recovery in the Parser . . . 154
Panic-Mode Recovery . . . 154
Two Recovery Strategies . . . 155
How Recovery Works in Practice . . . 156
9.7 MSD Parser: Complete Walkthrough . . . 156
Input File . . . 157
Token Stream . . . 157
Step-by-Step Trace . . . 158
Final ParseResult . . . 159
9.8 Design Decisions . . . 160
Why Recursive Descent Over Parser Generators? . . . 160
Why Intermediate Representation? . . . 161
Why Multiple Error Reporting? . . . 161
Why `_ParsePanic` as an Exception? . . . 161
9.9 Exercises . . . 162
Summary . . . 163

Chapter 10: Semantic Analysis **165**
10.1 From Syntax Tree to Meaning . . . 165
The gap between syntax and semantics . . . 165
The builder's role . . . 166
Three phases of semantic analysis . . . 166
10.2 Name-to-ID Resolution and Symbol Tables . . . 166
The problem . . . 166
Symbol tables . . . 167
Build order enables forward references . . . 167
The resolution code . . . 168
10.3 Duplicate Detection and Conflict Checking . . . 169
Duplicate entity names . . . 169
Duplicate association names . . . 170
Cross-namespace conflicts . . . 170
10.4 "Did You Mean?" Suggestions via Levenshtein Distance . . . 171
The problem of near-misses . . . 171
The Levenshtein edit distance . . . 171
The dynamic programming algorithm . . . 172
The suggestion function . . . 172
Worked example . . . 173
10.5 Warning vs Error Severity Decisions . . . 174
The severity spectrum . . . 174
Where to draw the line . . . 174
The design principle . . . 175
10.6 MSD Builder: Complete Walkthrough . . . 175
Step 1: Collect existing errors . . . 176
Step 2: Apply metadata . . . 176
Step 3: Build entities . . . 177
Step 4: Build associations . . . 177
Step 5: Build links . . . 177
Step 6: Auto-layout . . . 177
The complete return . . . 178
10.7 Exercises . . . 178

Chapter 11: Output Generation **180**
11.1 Transforming the Intermediate Representation . . . 180
11.2 UUID Generation for Persistent Identity . . . 181
Why UUIDs? . . . 181
How UUIDs Appear . . . 181
The Determinism Concern . . . 182
11.3 Metadata Application and Defaults . . . 182
11.4 JSON Serialisation for Tool Interoperability . . . 183
Why JSON? . . . 183
The .merisio Structure . . . 184
Reusing Existing Serialisation . . . 184
11.5 Round-Trip Fidelity . . . 185
The Round-Trip Test . . . 185
What Is Preserved . . . 185

What Is NOT Preserved . . . 186
Testing Round-Trip Fidelity . . . 186
11.6 Output Formats for Other DSLs . . . 187
SQL DSLs . . . 187
Configuration DSLs . . . 187
Diagram DSLs . . . 188
Query DSLs . . . 188
The Common Pattern . . . 188
11.7 MSD: Building a Merisio Project from ParseResult . . . 188
Step 1: Create the Project Object . . . 188
Step 2: Apply Metadata . . . 189
Step 3: Create Entities with Attributes . . . 189
Step 4: Create Associations . . . 189
Step 5: Create Links with Resolved References . . . 190
Step 6: Auto-Layout . . . 190
Step 7: Mark as Clean and Save . . . 190
The Resulting JSON . . . 190
11.8 Exercises . . . 192

Chapter 12: Automatic Graph Layout 194
12.1 The Graph Drawing Problem . . . 194
12.2 Force-Directed Algorithms: The Physical Analogy . . . 195
12.3 The Fruchterman-Reingold Algorithm . . . 196
12.4 Implementation Walkthrough . . . 198
Initial Placement . . . 198
Main Loop . . . 198
Repulsive Forces . . . 199
Attractive Forces . . . 199
Displacement Clamping . . . 200
12.5 Post-Processing . . . 200
Overlap Removal . . . 200
Centring . . . 201
Rounding . . . 202
12.6 Determinism Through Fixed Seeds . . . 202
12.7 Complexity Analysis . . . 203
12.8 MSD: Auto-Layout for Entities and Associations . . . 204
Duck Typing via Protocol . . . 204
Edge Resolution . . . 204
12.9 Exercises . . . 205

Chapter 13: Error Handling and Recovery 207
13.1 The Error Model . . . 207
Representing Errors with Metadata . . . 207
The Compiler-Style Format . . . 208
13.2 Error vs Warning: When to Stop vs When to Continue . . . 208
Design Guideline . . . 209
13.3 Error Flow Through a Multi-Stage Pipeline . . . 209
13.4 Panic-Mode Recovery . . . 210

How Panic Mode Works . . . 210
Synchronisation Points . . . 210
Recovery Scopes . . . 211
13.5 Multi-Error Reporting . . . 212
What Gets Preserved After Errors . . . 212
13.6 Error Message Quality . . . 213
Principles . . . 213
Anti-Patterns . . . 213
13.7 The `_ParsePanic` Exception . . . 213
13.8 MSD: Complete Error Handling Walkthrough . . . 214
13.9 Testing Error Paths . . . 214
Test Categories . . . 214
Example Test . . . 215
13.10 Exercises . . . 215

Chapter 14: Testing DSL Implementations 217
14.1 Testing Philosophy . . . 217
Test Each Layer Independently . . . 217
Test Both Success and Failure Paths . . . 217
Test the Contract, Not the Implementation . . . 218
14.2 Lexer Tests . . . 219
Token Extraction Helpers . . . 219
What to Test: Lexer . . . 219
Notable Test: Context-Sensitive Project Values . . . 219
14.3 Parser Tests . . . 220
What to Test: Parser . . . 220
Notable Test: Error Recovery . . . 220
Notable Test: All Cardinalities . . . 221
14.4 Semantic Tests (Builder Tests) . . . 221
What to Test: Semantics . . . 221
Notable Test: “Did You Mean?” . . . 222
Notable Test: Round-Trip . . . 222
14.5 Integration Tests . . . 222
The Full Pipeline Test . . . 223
Other Integration Tests . . . 223
14.6 Test Fixtures and Shared Test Data . . . 223
Constant MSD Strings . . . 223
Pytest Fixtures . . . 224
14.7 MSD: Walkthrough of the 64-Test Suite . . . 225
Organisation . . . 225
Coverage Strategy . . . 225
Running the Tests . . . 225
14.8 Exercises . . . 226

Chapter 15: Tool Integration 227
15.1 CLI Integration . . . 227
Subcommands and Argument Parsing . . . 227
Exit Codes . . . 227

Error Output . . . 228
Implementation: The `cmd_parse` Function . . . 228
15.2 GUI Integration . . . 229
The Import Workflow . . . 229
Error Presentation . . . 229
Post-Import Actions . . . 230
15.3 Man Pages and Documentation . . . 231
Unix Convention . . . 231
Essential Documentation . . . 231
Documentation as a First-Class Deliverable . . . 231
15.4 Batch Processing and CI Pipelines . . . 231
Batch Conversion . . . 231
CI Pipeline . . . 232
Version Control Workflows . . . 232
15.5 The Integration Pattern: Library → CLI Wrapper → GUI Wrapper . . . 232
15.6 MSD: The `parse` Command and File > Import MSD . . . 233
15.7 Exercises . . . 234

Appendix A: MSD Formal Grammar **235**
A.1 Notation . . . 235
A.2 Grammar . . . 235
A.3 Lexical Rules . . . 236
A.4 Semantic Constraints . . . 236
A.5 Grammar Properties . . . 237
A.6 Token Summary . . . 237

Appendix B: MSD Data Type Reference **239**
B.1 Type Summary . . . 239
B.2 Numeric Types . . . 239
INT . . . 239
BIGINT . . . 240
SMALLINT . . . 240
DECIMAL . . . 240
FLOAT . . . 240
DOUBLE . . . 240
B.3 String Types . . . 241
VARCHAR . . . 241
CHAR . . . 241
TEXT . . . 241
B.4 Logical Types . . . 242
BOOLEAN . . . 242
B.5 Temporal Types . . . 242
DATE . . . 242
TIME . . . 242
TIMESTAMP . . . 242
B.6 Size Parameter Rules . . . 243
B.7 Type Selection Guide . . . 243

Appendix C: DSL Design Checklist **244**
Phase 1: Domain Analysis (Chapter 3) 244
Phase 2: Lexical Design (Chapter 4) 244
Phase 3: Syntactic Design (Chapter 5) 245
Phase 4: Semantic Design (Chapter 6) 245
Phase 5: Implementation (Chapters 8–12) 246
Lexer (Chapter 8) 246
Parser (Chapter 9) 246
Semantic Analyser / Builder (Chapter 10) 246
Output Generation (Chapter 11) 247
Layout (Chapter 12, if applicable) 247
Phase 6: Integration (Chapters 13–15) 247
Error Handling (Chapter 13) 247
Testing (Chapter 14) 247
Tool Integration (Chapter 15) 247
Summary 248

Appendix D: Glossary **249**
General DSL Terms 249
MSD-Specific Terms 251

Preface

Why Domain-Specific Languages?

Every programmer uses domain-specific languages daily. SQL queries databases. CSS styles web pages. Regular expressions match text patterns. Makefiles orchestrate builds. Dockerfiles describe containers. These are not general-purpose programming languages — they are **domain-specific languages** (DSLs), each designed to excel at one particular task.

Yet while we all *use* DSLs, very few of us *design* them. The craft of creating a new language — choosing its syntax, defining its semantics, building its toolchain — remains something of a dark art, scattered across compiler textbooks, academic papers, and tribal knowledge. This book aims to change that.

What You'll Learn

By the end of this book, you will:

- Understand the spectrum from configuration files to general-purpose languages, and where DSLs fit
- Know how to analyse a domain and extract the concepts that a language should express
- Design lexical, syntactic, and semantic rules for a new DSL
- Implement a complete language toolchain: lexer, parser, semantic analyser, and output generator
- Build automatic graph layout for visual output
- Design robust error handling with multi-error reporting and recovery
- Write comprehensive tests for every stage of the pipeline
- Integrate a DSL into both command-line and graphical tools

The Case Study: MSD and Merisio

Theory without practice is incomplete. Throughout this book, we use a real DSL as our primary case study: **MSD** (Merisio Schema Definition), a language for defining MERISE conceptual data models as plain text. MSD is part of the Merisio project — a visual database modelling tool.

MSD is small enough to understand completely in one book, yet complex enough to illustrate every important DSL design decision: context-sensitive lexing, recursive

descent parsing, name resolution with typo suggestions, force-directed graph layout, and multi-error recovery. Every code example in the implementation chapters is drawn from MSD's production codebase.

Prerequisites

This book assumes you have:

- **Programming experience**: Comfort with at least one language; implementation examples use Python, but the concepts are language-agnostic
- **Basic data structures**: Familiarity with lists, dictionaries, trees, and graphs
- **Curiosity about languages**: An interest in how programming languages work beneath the surface

No prior compiler construction or formal language theory experience is required — we build everything from first principles.

How to Use This Book

The book is organised into four parts:

1. **Foundations** (Chapters 1–3) introduces domain-specific languages, surveys the DSL landscape, and walks through domain analysis using the MERISE modelling domain as a case study.
2. **Language Design** (Chapters 4–7) covers the three layers of language design — lexical, syntactic, and semantic — then applies all three to design MSD from scratch.
3. **Implementation** (Chapters 8–12) builds a complete DSL toolchain: lexer, recursive descent parser, semantic analyser, output generator, and automatic graph layout engine.
4. **Quality and Integration** (Chapters 13–15) addresses error handling and recovery, testing strategies for DSL implementations, and integrating a DSL into CLI and GUI tools.

Each chapter follows a consistent structure:

- **Concepts**: The general theory, applicable to any DSL
- **Case Study**: How MSD applies the theory
- **Code Walkthrough**: Annotated implementation (in the implementation chapters)
- **Exercises**: 6–7 per chapter, progressing from comprehension to design challenges

The exercises are an essential part of the book. Many ask you to design syntax for hypothetical DSLs or extend MSD — these design exercises develop intuition that cannot be gained from reading alone.

Conventions

Code examples use Python syntax highlighting:

```python
class MSDLexer:
    def tokenize(self, source: str) -> list:
        ...
```

MSD examples use a dedicated syntax:

```
entity Student {
    *student_id: INT
    name: VARCHAR(100)
}
```

Callout boxes highlight important information:

Tip

Tips provide helpful shortcuts or best practices.

Warning

Warnings alert you to common pitfalls or design traps.

Note

Notes offer additional context or background information.

Info

Info boxes provide supplementary technical details.

Acknowledgements

MSD was designed as an improvement over the Mocodo DSL for MERISE modelling, addressing its limitations in explicit primary keys, data types, and error reporting. The Merisio project itself was inspired by AnalyseSI, a Java-based MERISE tool widely used in French computer science education.

The general principles in this book draw on decades of research in programming language design, from Backus-Naur Form to modern language workbenches. The specific techniques — recursive descent parsing, panic-mode error recovery,

Fruchterman-Reingold layout — are established methods refined through practical application.

Let's Begin

Designing a domain-specific language is one of the most rewarding tasks in software engineering. You are not just writing code — you are creating a new way for people to express ideas. A well-designed DSL can turn a complex, error-prone task into something elegant and intuitive.

Turn the page, and let's design a language.

Achraf SOLTANI www.achrafsoltani.com

Table of Contents

Part I: Foundations

1. **Introduction to Domain-Specific Languages** — What is a DSL? General-purpose vs domain-specific. Internal vs external DSLs. The spectrum from configuration files to full languages. When to build a DSL and when not to.
2. **The DSL Landscape** — Survey of real-world external DSLs: SQL, GraphQL, Terraform HCL, Dockerfile, Makefile, CSS. Classification by paradigm and complexity. Common patterns across DSLs.
3. **Requirements and Domain Analysis** — Understanding the target domain. Identifying concepts, relationships, and constraints. User personas and workflow analysis. Case study introduction: MERISE modelling and Merisio.

Part II: Language Design

4. **Lexical Design** — Tokens as atoms. Keywords, identifiers, literals, symbols. Comment styles. Case sensitivity. Context-sensitive lexing. Design trade-offs: familiarity vs precision.
5. **Syntactic Design** — From tokens to structure. EBNF notation. LL(1) grammars. Block-structured vs line-oriented syntax. Ambiguity and how to avoid it. Designing for readability vs parseability.
6. **Semantic Design** — Beyond syntax: meaning. Type systems for DSLs. Name binding and scoping. Reference resolution and forward references. Validation rules. Error messages as a design concern.
7. **Designing MSD** — Applying Chapters 3–6 to MERISE modelling. Domain analysis. Lexical, syntactic, and semantic choices. The complete MSD specification. Comparison with Mocodo.

Part III: Implementation

8. **Building a Lexer** — Lexer architecture. Token representation. Line-by-line scanning. Keyword recognition. Context sensitivity. Error detection. MSD lexer walkthrough.
9. **Building a Recursive Descent Parser** — The recursive descent technique. Intermediate representations. Token stream navigation. Parsing blocks, attributes, types, statements. MSD parser walkthrough.

10. **Semantic Analysis** — From syntax tree to meaning. Name-to-ID resolution. Duplicate detection. "Did you mean?" suggestions via Levenshtein distance. Warning vs error severity. MSD builder walkthrough.
11. **Output Generation** — Transforming intermediate representations. UUID generation. Metadata application. JSON serialisation. Round-trip fidelity. MSD output walkthrough.
12. **Automatic Graph Layout** — The graph drawing problem. Force-directed algorithms. Fruchterman-Reingold: repulsion, attraction, cooling. Overlap removal. Determinism. MSD layout walkthrough.

Part IV: Quality and Integration

13. **Error Handling and Recovery** — Error representation. Error vs warning. Error flow through multi-stage pipelines. Panic-mode recovery. Multi-error reporting. Error message quality. MSD error handling walkthrough.
14. **Testing DSL Implementations** — Testing philosophy. Lexer tests. Parser tests. Semantic tests. Integration tests. Test fixtures. MSD test suite walkthrough.
15. **Tool Integration** — CLI integration: subcommands, exit codes, error output. GUI integration: file dialogs, import workflows. Man pages. Batch processing. The library-CLI-GUI pattern.

Appendices

A. **MSD Formal Grammar** — Complete EBNF specification, lexical rules, semantic constraints, and token summary.

B. **MSD Data Type Reference** — All 13 supported data types with descriptions, size rules, and usage guidance.

C. **DSL Design Checklist** — Step-by-step checklist for designing a new DSL from scratch.

D. **Glossary** — General DSL terms and MSD-specific definitions.

Chapter 1: Introduction to Domain-Specific Languages

Every day, programmers reach for languages that are not Python, not Java, not Go. They write `SELECT * FROM users WHERE age > 21` to query a database. They write `margin: 0 auto` to centre a box on a web page. They write `^[a-z]+@[a-z]+\.[a-z]{2,}$` to validate an email address. These are not general-purpose programming languages. They are **domain-specific languages** — languages built to do one thing well.

This chapter introduces the concept of domain-specific languages, explains how they differ from general-purpose languages, and establishes the vocabulary we shall use throughout the rest of this book. By the end, you will understand what DSLs are, when they are worth building, and why we have chosen MSD as our running case study.

1.1 What Is a Domain-Specific Language?

A **domain-specific language** (DSL) is a programming language designed for a particular application domain. Unlike a general-purpose language (GPL) such as Python or Go, which aims to be useful across all domains, a DSL deliberately restricts its scope. It trades versatility for expressiveness: by focusing on one domain, a DSL can offer notations, abstractions, and safety guarantees that a general-purpose language cannot.

Consider SQL. It cannot draw graphics, serve web pages, or train a neural network. But for querying relational databases, no general-purpose language comes close to its clarity:

```sql
SELECT s.name, c.title
FROM students s
JOIN enrolments e ON s.id = e.student_id
JOIN courses c ON e.course_id = c.id
WHERE c.department = 'Computer Science'
ORDER BY s.name;
```

The equivalent logic in Python — constructing strings, managing connections, iterating over result sets — would be longer, more error-prone, and harder to review. SQL succeeds precisely *because* it does not try to be a general-purpose language. It encodes the relational model directly into its syntax and semantics, allowing users to think in terms of tables, joins, and filters rather than loops and pointers.

This is the fundamental insight behind every DSL: **domain knowledge becomes part of the language itself**. The language does not merely *support* the domain — it *speaks* the domain.

Here are several widely-used DSLs, each excelling in its own niche:

DSL	Domain	Example expression
SQL	Relational databases	`SELECT name FROM users`
CSS	Visual styling	`color: red; font-size: 14px`
Regular expressions	Text pattern matching	`\d{3}-\d{4}`
Make	Build automation	`app: main.o util.o`
Dockerfile	Container definitions	`FROM python:3.11-slim`
Cron	Task scheduling	`0 5 * * 1`

Each of these languages could, in principle, be replaced by a library in a general-purpose language. You could build HTML elements with Python function calls instead of writing CSS selectors. You could schedule tasks with a Python loop instead of cron expressions. But the DSL version is invariably more concise, more readable to domain experts, and more amenable to static analysis and tooling.

1.2 General-Purpose vs Domain-Specific

The distinction between general-purpose and domain-specific languages is not always sharp — it is better understood as a spectrum (which we explore in Section 1.5). Nonetheless, there are clear differences in philosophy and design:

Characteristic	General-Purpose Language	Domain-Specific Language
Scope	Any problem domain	One specific domain
Turing-complete?	Yes	Often no
Target audience	Professional programmers	Domain experts (may not be programmers)
Learning curve	Steep (broad surface area)	Shallow (narrow surface area)
Expressiveness	Wide but shallow per domain	Narrow but deep within its domain
Tooling	IDE, debugger, profiler, etc.	Often specialised: linters, validators, visualisers
Error messages	Generic (“type mismatch”)	Domain-aware (“unknown data type”)
Longevity	Decades (Python, C, Java)	Tied to the domain’s lifespan

A general-purpose language gives you the building blocks to solve *any* problem, but it does not know anything about *your* problem. You must encode domain rules yourself

— in validation logic, naming conventions, and documentation. A DSL, by contrast, makes domain rules explicit. Violating them is not a logic error caught by a unit test; it is a *syntax error* caught by the language itself.

Consider data type declarations. In Python, nothing stops you from writing:

```
column_type = "VARCAHR(255)"  # Typo: VARCAHR instead of VARCHAR
```

This is a valid Python string. The mistake will surface only at runtime, perhaps in production, when the DDL statement fails against the database. In MSD, our case study language, the same typo produces an immediate, clear error:

```
entity Student {
    *id: INT
    name: VARCAHR(255)
}
```

```
Error [line 3]: Unknown data type 'VARCAHR'. Did you mean 'VARCHAR'?
```

The language itself knows what a valid data type is. It does not merely carry the value — it *understands* it. This is the power of encoding domain knowledge into the language.

When to use which

Use a **general-purpose language** when:

- The problem spans multiple domains
- You need arbitrary computation (algorithms, data structures, I/O)
- The audience is professional programmers comfortable with full languages
- Requirements change frequently and unpredictably

Use a **domain-specific language** when:

- The problem is well-defined and recurs often
- The audience includes non-programmers or domain specialists
- Correctness matters and domain constraints can be enforced at the language level
- The notation should match how experts already think about the domain

1.3 Internal vs External DSLs

DSLs come in two fundamental flavours: **internal** and **external**.

Internal DSLs

An internal DSL (sometimes called an *embedded* DSL) is built within the syntax of an existing host language. It exploits the host language's features — operator overloading, closures, builder patterns — to create an API that reads like a specialised language, while remaining valid code in the host.

For example, Kotlin's Gradle DSL is an internal DSL for build configuration:

```
dependencies {
    implementation("org.jetbrains.kotlin:kotlin-stdlib:1.9.0")
    testImplementation("junit:junit:4.13.2")
}
```

This is valid Kotlin code. The curly braces are lambdas. `implementation` and `testImplementation` are method calls. But it *reads* like a dedicated build language. Similarly, Ruby's RSpec is an internal DSL for testing:

```
describe Calculator do
  it "adds two numbers" do
    expect(Calculator.add(2, 3)).to eq(5)
  end
end
```

Again, this is valid Ruby, but it reads like a natural description of behaviour.

In Java, the fluent API pattern creates a more modest form of internal DSL:

```
Query query = new QueryBuilder()
    .select("name", "email")
    .from("users")
    .where("age > 21")
    .orderBy("name")
    .build();
```

External DSLs

An external DSL is a standalone language with its own syntax, its own parser, and (typically) its own toolchain. It does not piggyback on any host language. SQL, CSS, regular expressions, Make, and Dockerfile are all external DSLs. So is MSD, our case study:

```
entity Student {
    *student_id: INT
    name: VARCHAR(100)
    email: VARCHAR(255)
    enrolled_on: DATE
}
```

This is not Python. It is not Java. It is MSD — a language with its own lexer, parser, semantic analyser, and output generator.

Comparing the two approaches

Aspect	Internal DSL	External DSL
Parser needed?	No (host language parser)	Yes (you build one)

Aspect	Internal DSL	External DSL
Tooling cost	Low (reuse host IDE, debugger)	High (custom editor support, linting)
Syntactic freedom	Limited by host language grammar	Complete freedom
Error messages	Host language errors (often cryptic)	Custom domain-aware errors
User audience	Must know the host language	Can be non-programmers
Distribution	Library/package in host ecosystem	Standalone tool or integration
Type safety	Host language's type system	Custom type system possible

Internal DSLs are faster to build: you get a parser, an IDE, and a debugger for free. But you are constrained by the host language's syntax. You cannot invent new keywords, new block structures, or new operator precedence rules. Error messages from the host compiler leak through, confusing users who do not know (or care about) the host language.

External DSLs require significantly more engineering effort — you must build a lexer, a parser, and often a semantic analyser — but they offer complete control over syntax, semantics, and error reporting. For users who are not programmers, this control is essential.

Note

This book focuses on **external DSLs**. While internal DSLs are a valuable technique, the design and implementation challenges of external DSLs are richer and more broadly applicable. If you understand how to build an external DSL, you can always fall back to an internal one — but not vice versa.

Programmer's Perspective

DSLs are far more common in daily programming than most developers realise. SQL, HTML, CSS, regular expressions, Dockerfiles, Terraform HCL, YAML-based CI configurations, and even Go's `text/template` package are all domain-specific languages you likely use every week. The distinction between internal and external DSLs maps directly to a practical choice: Go's fluent builder pattern (e.g., chaining methods like `query.Select("name").From("users").Where("age > 21")`) is an internal DSL constrained by Go's syntax, whilst a standalone `.tf` or `.sql` file is an external DSL with its own parser. When you reach for `go generate` to process a custom file format, you are building external DSL tooling.

1.4 When to Build a DSL (and When Not To)

Building a DSL is a significant investment. A well-designed DSL can save thousands of hours across an organisation; a poorly motivated one can waste just as many. Before committing, you should ask yourself a series of honest questions.

Good reasons to build a DSL

Repeated complex tasks in a narrow domain. If your team writes the same kind of logic over and over — configuring database schemas, defining workflow rules, specifying data transformations — a DSL can capture that pattern once and reuse it indefinitely. Each new instance becomes a few lines of DSL code rather than pages of general-purpose boilerplate.

Non-programmer users. If the people who understand the domain are not software engineers — database designers, business analysts, DevOps engineers, scientists — a DSL lets them express their knowledge directly, without learning a full programming language. SQL's success owes much to the fact that database administrators, not just programmers, can write and understand queries.

Enforcing domain constraints. A DSL can make invalid states unrepresentable. If every entity must have a primary key, the language can enforce that rule at parse time. If cardinalities must be one of `(0,1)`, `(0,N)`, `(1,1)`, or `(1,N)`, the grammar can reject anything else. Constraints encoded in the language are far more reliable than constraints documented in a wiki page that nobody reads.

Enabling code review of domain logic. When domain logic is expressed in a DSL, it becomes reviewable by domain experts who may not understand the surrounding application code. A database architect can review an MSD file without knowing Python. A security engineer can review firewall rules in a dedicated policy language without understanding the routing daemon's C++ codebase.

Generating multiple outputs from a single source. A single MSD file can generate PostgreSQL DDL, a visual diagram, a data dictionary, and an MLD (logical data model). This multi-output pattern is a hallmark of successful DSLs: one source of truth, many derived artefacts.

Bad reasons to build a DSL

Resume-driven development. "I've always wanted to build a language" is enthusiasm, not justification. Language design is a means, not an end. If a library, a configuration file, or an API would solve the problem equally well, prefer the simpler option.

When a library or API would suffice. If the users are already programmers and the "language" is essentially a set of function calls, an internal DSL or a well-designed API is almost certainly the better choice. You get IDE support, type checking, and an existing ecosystem for free.

When the domain is too broad. A DSL for "data processing" is really a general-purpose language in disguise. The domain must be narrow enough that you can enumerate its core concepts and operations. If you find yourself adding loops, conditionals, variables, and functions to your DSL, you are probably reinventing a GPL — badly.

When the domain is unstable. If the domain's requirements change dramatically every few months, the DSL will be in constant flux. Users will face breaking syntax changes, outdated documentation, and migrating scripts. A DSL works best when the domain is mature enough that its core concepts are settled.

A decision framework

Ask these five questions before building a DSL:

1. **Is the domain well-defined?** Can you enumerate the core concepts (entities, operations, constraints) in a page or two?
2. **Is the task repetitive?** Will users write many instances of similar specifications?
3. **Who are the users?** Are they programmers, domain experts, or both?
4. **What outputs are needed?** Will the DSL generate code, configuration, documentation, or visualisations?
5. **How stable is the domain?** Will the core concepts still make sense in two years?

If you answered "yes" to questions 1, 2, and 5, and "multiple" to question 4, a DSL is likely justified. If most answers are uncertain, start with a library and see whether a language naturally emerges from usage patterns.

Tip

A useful litmus test: write ten representative examples of what users would express in the proposed DSL. If the examples look natural, concise, and reviewable — and if a library version of the same logic would be significantly worse — a DSL is probably the right choice.

Programmer's Perspective

Go's standard library itself embeds this DSL-or-library decision. The `text/template` and `html/template` packages are a built-in DSL for text generation with their own lexer and parser, whilst the `net/http` package is a library API. The Go team chose a template DSL because expressing conditional text layout through function calls would be unreadable. Conversely, HTTP routing stayed as a library because the domain (matching paths and dispatching handlers) maps naturally to Go function signatures. This is exactly the decision framework described above: if the domain's notation is fundamentally different from the host language's syntax, an external DSL pays for itself.

1.5 The Language Spectrum

Domain-specific languages do not exist in isolation. They sit on a spectrum that stretches from simple configuration files at one end to full general-purpose languages at the other. Understanding this spectrum helps you choose the right level of complexity for your problem.

The spectrum

```
Configuration     Data         Micro-        Medium         Full
   files         formats       DSLs           DSLs        languages
 ──────────────────────────────────────────────────────────────────►
                                                          Expressiveness
INI, TOML        JSON, XML    regex, cron   SQL, CSS,    Python, Go,
.env, YAML                    glob, MIME    Make, HCL    Java, Rust
```

Configuration files (INI, TOML, `.env`) store key-value pairs. They have no logic, no computation, and no structure beyond simple nesting. They are not languages in any meaningful sense — but they *are* often the starting point from which DSLs evolve. Many DSLs begin life as configuration files that gradually grow conditional logic, variable substitution, and cross-referencing.

Data formats (JSON, XML, CSV) describe structured data. They can represent arbitrarily nested structures, but they carry no semantics — a JSON file does not know whether `"type": "VARCAHR"` is a valid data type. They are passive containers, not active languages.

Micro-DSLs (regular expressions, cron expressions, glob patterns) are extremely compact notations for a single, well-defined task. They have no blocks, no declarations, and usually no named abstractions. They excel at brevity but sacrifice readability: `^(?:[a-z0-9]+\.)*[a-z0-9]+@[a-z0-9]+\.[a-z]{2,}$` is powerful but opaque to the uninitiated.

Medium DSLs (SQL, CSS, Make, HCL, Dockerfile) have declarations, blocks, and sometimes limited computation (SQL's `CASE WHEN`, Make's pattern rules, HCL's `for_each`). They are rich enough to express complex domain logic but restricted

enough to remain analysable and safe. This is the **sweet spot** for most external DSLs — and it is where MSD sits.

Full languages (Python, Go, Java, Rust) are Turing-complete, with control flow, data structures, abstraction mechanisms, and module systems. They can do anything but know nothing about any particular domain.

Where MSD fits

MSD is a medium DSL. It has four block types (`project`, `entity`, `association`, `link`), typed attributes with explicit data types, cardinality constraints, and comment syntax — but no variables, no conditionals, no loops, and no functions. This is a deliberate choice. A database schema is a *declaration*, not a *computation*. MSD's design reflects this: you declare what exists, not how to compute it.

```
# A complete MSD file: declarations, not computation
project {
    name: Library Management System
    author: Dr. H
}

entity Book {
    *isbn: VARCHAR(13)
    title: VARCHAR(255)
    published_on: DATE
}

entity Author {
    *author_id: INT
    name: VARCHAR(100)
}

association written_by {
}

link Book (1,N) written_by
link Author (0,N) written_by
```

There are no `if` statements, no loops, no variables. The file is a complete, self-contained description of a conceptual data model. This simplicity is a feature, not a limitation.

1.6 Real-World Impact

The most successful DSLs have transformed entire industries. Studying what makes them effective reveals patterns that apply to any DSL design effort.

SQL: Transforming database interaction

Before SQL, querying a database required navigating internal data structures with procedural code — following pointers, managing cursors, understanding storage layouts. SQL abstracted all of this away. A user describes *what* data they want, not *how* to retrieve it. This declarative approach enabled query optimisers, which can choose execution strategies far more sophisticated than a human would write by hand.

Why it works: SQL's syntax mirrors the relational algebra it models. `SELECT`, `FROM`, `WHERE`, `JOIN` correspond directly to projection, relation, selection, and natural join. The language does not merely *call* database operations — it *is* the relational model, expressed as text.

CSS: Separating content from presentation

Before CSS, visual styling was tangled into HTML markup: `<font size="3" color="red">`. CSS introduced a clean separation of concerns. Content lives in HTML; presentation lives in CSS. This separation enabled responsive design, theming, accessibility features, and a global ecosystem of stylesheets.

Why it works: CSS's selector-property model maps directly to the domain. You select elements (`h1`, `.warning`, `#header`) and declare properties (`color`, `margin`, `font-size`). The cascade and specificity rules, while sometimes frustrating, encode a real domain concern: how should conflicting style instructions be resolved?

Regular expressions: Compact pattern specification

Regular expressions compress complex string-matching logic into terse, powerful patterns. What might take twenty lines of procedural string parsing becomes a single expression.

Why it works: Regular expressions are directly grounded in formal language theory (regular languages and finite automata). This mathematical foundation guarantees that any pattern can be matched in linear time — a performance guarantee that procedural alternatives cannot offer.

Make: Declarative build orchestration

Makefiles describe *what* depends on *what*, not *how* to traverse the dependency graph. This declarative approach enables parallel builds, incremental compilation, and change detection — features that would be complex to implement in a procedural build script.

Why it works: Make's `target: dependencies` syntax is a natural expression of the dependency graph that underlies every build system. The tab-indented recipes are a minor syntactic quirk, but the declarative dependency model is genuinely powerful.

Dockerfile: Reproducible container definitions

Dockerfiles describe how to build a container image as a sequence of layers. Each instruction (`FROM`, `RUN`, `COPY`, `EXPOSE`) corresponds to a distinct operation in the container build process.

Why it works: Dockerfile's layer model maps directly to the underlying technology (union filesystems). The declarative syntax makes container definitions readable, reviewable, and cacheable — three properties that matter enormously in production infrastructure.

Common success factors

Across all five examples, several patterns emerge:

1. **Syntax mirrors domain concepts.** SQL's `SELECT/FROM/WHERE` mirrors relational algebra. CSS selectors mirror the DOM tree. Dockerfile commands mirror image layers.
2. **Declarative over imperative.** Users describe *what* they want, not *how* to achieve it. This enables optimisation and analysis by tools.
3. **Constrained expressiveness.** None of these languages tries to do everything. Each stays firmly within its domain, relying on general-purpose languages for anything outside it.
4. **Readable by domain experts.** A database administrator can read SQL. A designer can read CSS. A DevOps engineer can read a Dockerfile. The language speaks the domain's vocabulary.

1.7 The MSD Case Study

Throughout this book, we use **MSD** (Merisio Schema Definition) as our primary case study. MSD is the text-based input language for Merisio, a visual database modelling tool based on the MERISE methodology.

What MSD does

MSD allows users to define MERISE **conceptual data models** (MCDs) as plain text. An MCD consists of **entities** (things that exist in the domain), **associations** (relationships between entities), and **links** (connections specifying how entities participate in associations, including cardinality constraints).

Here is a small but complete MSD file:

```
project {
    name: University Enrolment
    author: Dr. H
    description: Student course enrolment system
}
```

```
entity Student {
    *student_id: INT
    name: VARCHAR(100)
    email: VARCHAR(255)
}

entity Course {
    *course_id: INT
    title: VARCHAR(200)
    credits: SMALLINT
}

association enrolment {
    enrolled_on: DATE
    grade: DECIMAL(4)
}

link Student (0,N) enrolment
link Course (0,N) enrolment
```

This file defines two entities (`Student` and `Course`), one association (`enrolment` with its own attributes), and two links specifying that a student can be enrolled in zero or more courses, and a course can have zero or more students.

From this single file, Merisio can generate:

- A **visual diagram** showing entities as rectangles and associations as ovals, connected by labelled lines
- An **MLD** (logical data model) showing the resulting relational tables, with foreign keys derived from cardinality rules
- **PostgreSQL DDL** (SQL `CREATE TABLE` statements) ready to execute against a database
- A **data dictionary** cataloguing every entity, attribute, and relationship

One source file, four outputs. This multi-output capability is one of the hallmarks of a well-designed DSL.

Why MSD makes a good case study

MSD occupies a pedagogical sweet spot:

- **Small enough to understand completely.** MSD has four keywords (`project`, `entity`, `association`, `link`), thirteen data types, four cardinality pairs, and two comment styles. You can hold the entire language in your head.
- **Complex enough to illustrate real design decisions.** Despite its small size, MSD involves context-sensitive lexing (the `project` block uses different tokenisation rules), recursive descent parsing with error recovery, name resolution with typo suggestion (Levenshtein distance), semantic validation

(every entity needs a primary key, every association needs at least two links), and force-directed graph layout.

- **Genuinely useful.** MSD is not a toy language invented for a textbook. It is the input format for a real application used by students and educators in database courses.

What we shall build

Over the course of this book, you will see every stage of MSD's design and implementation:

- **Chapter 3** analyses the MERISE domain and identifies the concepts MSD must express
- **Chapters 4–6** cover lexical, syntactic, and semantic design as general principles
- **Chapter 7** applies those principles to design MSD from scratch, producing a complete language specification
- **Chapters 8–12** implement the full MSD toolchain: lexer, parser, semantic analyser, output generator, and automatic graph layout engine
- **Chapters 13–15** address error handling, testing, and integration into CLI and GUI tools

By the end, you will have not only a deep understanding of one DSL, but a transferable set of skills for designing and building any external DSL.

Info

All MSD implementation code is written in Python and drawn from the production Merisio codebase. You do not need to be a Python expert to follow along — the code uses straightforward constructs (classes, lists, dictionaries) that translate readily to any language.

1.8 Exercises

Exercise 1.1 — Identifying DSLs. Identify three domain-specific languages you use regularly (they need not be programming languages — configuration formats and markup languages count). For each one, describe its domain in one sentence, classify it as internal or external, and explain one design decision that makes it effective for its domain.

Exercise 1.2 — GPL vs DSL trade-offs. Consider a system that generates PDF invoices from structured data. A team is debating whether to build a small DSL for invoice templates or to use a Python library with an API. List three arguments in favour of each approach. Under what circumstances would you recommend the DSL?

Exercise 1.3 — The configuration-to-language spectrum. YAML is sometimes described as "accidentally Turing-complete" because of its alias and merge features. Take a real-world YAML configuration file you have encountered (e.g., a CI/CD pipeline, a Kubernetes manifest) and identify features that push it beyond simple key-value configuration towards language-like behaviour. Do you think those features improve or harm the format? Why?

Exercise 1.4 — Internal DSL design. Design a small internal DSL in the language of your choice for defining weekly meal plans. A user should be able to specify meals for each day, including course (starter, main, dessert) and dietary tags (vegetarian, gluten-free, etc.). Write example usage code showing what a user would write. What limitations does the host language impose on your syntax?

Exercise 1.5 — DSL success factors. Choose one of the DSLs from Section 1.6 (SQL, CSS, regular expressions, Make, or Dockerfile) and identify a design flaw or limitation that causes real problems for users. Propose a change that would fix the issue without compromising the language's core strengths. Explain why the original designers might have made the choice they did.

Exercise 1.6 — DSL feasibility assessment. A university wants a language for defining exam papers: questions, mark allocations, difficulty levels, learning outcomes, and randomisation rules. Apply the five-question decision framework from Section 1.4 to assess whether a DSL is justified. Write a one-paragraph recommendation with your reasoning.

Chapter 2: The DSL Landscape

In Chapter 1, we defined what domain-specific languages are and drew the line between DSLs and general-purpose languages. Now we step back and survey the landscape. What external DSLs exist in the wild? How do they differ? What do they have in common? And what separates the wildly successful ones from the forgotten experiments?

This chapter examines six real-world DSLs in detail, then develops two classification schemes — by paradigm and by complexity — that will guide our design decisions throughout the rest of the book. We close by identifying the patterns that recur across nearly every successful DSL, and the anti-patterns that lead to failure.

2.1 A Survey of External DSLs

The best way to understand what makes a good DSL is to study the ones that have stood the test of time. Each of the following languages was designed to solve a specific problem, and each has become the dominant tool in its domain.

SQL

SQL (Structured Query Language) is arguably the most successful domain-specific language ever created. Introduced in the 1970s and standardised in 1986, it allows users to express *what* data they want without specifying *how* to retrieve it.

```
SELECT name, email
FROM students
WHERE enrolled_year >= 2024
ORDER BY name;
```

SQL's genius lies in its declarative nature. The user describes the desired result — a filtered, sorted list of students — and the database engine decides the most efficient way to compute it. This separation of intent from implementation is a hallmark of great DSL design.

GraphQL

GraphQL, developed at Facebook and open-sourced in 2015, tackles a different data problem: letting clients specify the exact shape of the data they need from an API. Rather than the server dictating the response structure, the client's query *is* the response shape.

```
query {
  student(id: "42") {
    name
    courses {
      title
      credits
    }
  }
}
```

GraphQL is declarative for reads but introduces imperative elements through mutations — a hybrid approach we will revisit in Section 2.2.

Terraform HCL

HashiCorp Configuration Language (HCL) is the DSL behind Terraform, the infrastructure-as-code tool. It describes cloud resources declaratively: what should exist, not how to create it.

```
resource "aws_instance" "web" {
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = "t2.micro"
  tags = { Name = "WebServer" }
}
```

Terraform reads HCL files, computes the difference between the declared state and the actual state, and applies only the necessary changes. The DSL user never writes provisioning logic — they describe the desired end state.

Dockerfile

A Dockerfile describes how to build a container image, step by step. Unlike the declarative languages above, a Dockerfile is explicitly imperative — each instruction executes in sequence, and order matters.

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
```

The language is small (fewer than 20 instructions), easy to learn, and tightly coupled to its domain. Every line corresponds to a layer in the container image.

Makefile

Make, dating from 1976, uses a DSL that blends declarative dependency tracking with imperative recipe execution. Targets declare their dependencies, and Make determines what needs rebuilding.

```
app: main.o utils.o
    gcc -o app main.o utils.o

main.o: main.c
    gcc -c main.c
```

The dependency graph is declarative — Make computes the build order automatically. But the recipes (the indented lines) are imperative shell commands. This hybrid nature makes Make a fascinating case study.

CSS Selectors

CSS (Cascading Style Sheets) uses a pattern-based declarative approach: selectors match elements, and declarations describe how those elements should appear.

```
.student-card {
  background: #f5f5f5;
  border-radius: 8px;
  padding: 1rem;
}
```

CSS is purely declarative. There are no loops, no conditionals (ignoring media queries), and no variables in the original specification. The browser's rendering engine decides how to apply the rules efficiently.

Note

Preprocessors like Sass and Less add variables, loops, and functions to CSS — effectively creating a *new* DSL on top of CSS. This is a common evolution: when a DSL lacks a needed feature, users build a meta-DSL rather than abandoning the original.

Programmer's Perspective

The DSLs surveyed here have direct analogues in the Go ecosystem. Protocol Buffers (protobuf) is a serialisation DSL where you write `.proto` files and `protoc` generates Go structs and marshalling code – a pattern nearly identical to how MSD generates output from its source files. Mermaid is a diagram DSL that compiles textual descriptions into SVGs, much as MSD compiles schema descriptions into visual layouts. GraphQL schema files define API contracts that tools like `gqlgen` translate into Go interfaces and resolvers. In each case, the DSL captures domain knowledge that would be tedious and error-prone to express in raw Go code, and a code generator bridges the gap.

2.2 Classification by Paradigm

The six DSLs surveyed above fall into three broad paradigms, distinguished by how they express computation.

Declarative DSLs

A declarative DSL describes *what* the desired outcome is, leaving the *how* to the language's runtime or engine. The user specifies constraints, relationships, or properties — not the steps to achieve them.

SQL is the canonical example. The query `SELECT name FROM students WHERE age > 20` says nothing about index scans, hash joins, or sort algorithms. The query optimiser makes those decisions. Similarly, Terraform HCL declares resource configurations without specifying API call sequences, and CSS declares styles without specifying rendering algorithms.

Declarative DSLs excel when the domain has a well-understood execution model that can be optimised automatically. They are harder to design — the language author must build the "intelligence" that translates declarations into actions — but far easier to use.

Imperative DSLs

An imperative DSL describes a sequence of actions to perform, step by step. Order matters, and the user controls the flow of execution.

Dockerfiles are imperative: `FROM` sets the base image, `COPY` copies files, `RUN` executes commands, each building on the previous step. The Makefile recipe lines are also imperative — they are shell commands executed in order.

Imperative DSLs are simpler to implement (the runtime just executes instructions in sequence) and more intuitive for users with programming backgrounds. However, they are harder to optimise because the runtime has less freedom to reorder or skip steps.

Hybrid DSLs

Many real-world DSLs combine declarative and imperative elements. GraphQL is declarative for queries but uses imperative mutations for writes. Make's dependency graph is declarative, but its recipes are imperative. Even SQL has procedural extensions (`PL/pgSQL`, `T-SQL`) that add imperative control flow.

Hybrid DSLs often emerge when a purely declarative approach cannot cover all use cases. The key design challenge is keeping the two paradigms coherent — users should understand when they are in "declarative mode" and when they are in "imperative mode".

Paradigm Comparison

Aspect	Declarative	Imperative	Hybrid
Describes	What the result should be	Steps to perform	Both
Order matters?	No (usually)	Yes	Depends on context
Optimisable?	Highly	Limited	Partially
Ease of use	Higher for domain experts	Higher for programmers	Varies
Implementation	Complex runtime	Simple runtime	Mixed
Examples	SQL, CSS, Terraform HCL	Dockerfile, Make recipes	GraphQL, Make (overall)

When to choose each paradigm. Use a declarative paradigm when the domain has a well-defined notion of "desired state" and the runtime can determine the best execution strategy. Use an imperative paradigm when the domain is naturally sequential and order is semantically important. Use a hybrid approach when the domain has both aspects — but be deliberate about the boundary between them.

2.3 Classification by Complexity

Paradigm tells us *how* a DSL expresses computation. Complexity tells us *how much* it can express. We identify three tiers.

Micro-DSLs

Micro-DSLs have tiny grammars — often expressible in a few lines of EBNF — and extremely limited expressiveness. They solve one narrow problem and nothing else.

- **Regular expressions**: pattern matching on text (`\d{4}-\d{2}-\d{2}`)
- **Cron expressions**: scheduling (`0 9 * * MON-FRI`)
- **Glob patterns**: file matching (`src/**/*.py`)

Micro-DSLs typically have no variables, no types, no blocks, and no abstraction mechanisms. They are learned in minutes and mastered in hours. Their limitation is also their strength: there is almost no way to write a confusing cron expression (though regular expressions famously push this boundary).

Medium DSLs

Medium DSLs have richer grammars with multiple statement types, domain-specific types, and some form of structuring mechanism (blocks, nesting, or sections). Most of the DSLs surveyed in Section 2.1 fall into this category.

- **SQL**: multiple clauses, subqueries, joins, aggregate functions, data types
- **CSS**: selectors, properties, media queries, cascading rules
- **MSD**: entities, associations, links, attributes, types, cardinalities

Medium DSLs take hours to learn and weeks to master. They are expressive enough to model their domain completely but lack general-purpose features like arbitrary computation, user-defined functions, or module systems.

Full DSLs

Full DSLs approach the power of general-purpose languages while remaining anchored in a specific domain. They often include variables, functions, control flow, and module systems.

- **TeX**: a complete typesetting language with macros and Turing-complete expansion
- **Emacs Lisp**: a Lisp dialect specialised for text editing
- **Vim script**: an editor scripting language with its own type system

Full DSLs take weeks to learn and months to master. They offer great power but risk crossing the line into general-purpose territory, which brings the complexity costs of a GPL without the ecosystem benefits.

Complexity Comparison

Aspect	Micro-DSL	Medium DSL	Full DSL
Grammar size	5–15 rules	20–80 rules	100+ rules
Learning time	Minutes to hours	Hours to weeks	Weeks to months
Abstraction	None	Limited (blocks, names)	Functions, macros, modules
Types	None or implicit	Domain-specific	Rich type system
Turing-complete?	No	Usually no	Often yes
Examples	Regex, cron, glob	SQL, CSS, MSD	TeX, Emacs Lisp, Vim script

Tip

When designing a new DSL, start at the micro or medium level. It is far easier to add expressiveness later than to remove complexity from an overgrown language. Many successful DSLs began as micro-DSLs and grew into medium DSLs over decades (CSS is a prime example).

Programmer's Perspective

The complexity tiers map directly to implementation effort in Go. A micro-DSL like cron expressions can be parsed with a single `strings.Split` call and a few `strconv.Atoi` checks – no formal grammar needed. A medium DSL like HCL or MSD requires a proper lexer and recursive descent parser, typically 500–1500 lines of Go. A full DSL like Terraform's extended HCL (with `for_each`, conditionals, and functions) demands a type system, an evaluator, and thousands of lines of infrastructure. Go's `go/scanner` and `go/parser` packages themselves implement a medium-to-full DSL processor for the Go language, and studying their source code reveals exactly this progression from simple token scanning to complex semantic analysis.

2.4 Common Patterns Across DSLs

Despite their diversity, DSLs share recurring structural patterns. Recognising these patterns accelerates design — you do not need to reinvent them for each new language.

Block Structure

Nearly every medium or full DSL groups related declarations into blocks. The delimiters vary, but the concept is universal.

Curly braces (SQL subqueries, Terraform, MSD):

```
entity Student {
    *student_id: INT
    name: VARCHAR(100)
}
```

Indentation (Python, YAML, Makefile recipes):

```
app: main.o
    gcc -o app main.o
```

Keywords (`BEGIN/END` in PL/SQL, `do/end` in Ruby DSLs):

```
BEGIN
    INSERT INTO log VALUES (NOW(), 'started');
```

```
END;
```

Block structure creates visual hierarchy and defines scoping boundaries. When designing a DSL, choose a block style that matches your users' expectations — web developers expect braces; data analysts expect indentation or keywords.

References

DSLs that define multiple entities need a way to refer from one to another. This requires naming things and resolving those names later.

In **SQL**, table and column names are references:

```
SELECT s.name FROM students s
JOIN enrolments e ON s.id = e.student_id;
```

In **Terraform**, resources reference each other by path:

```
subnet_id = aws_subnet.main.id
```

In **MSD**, associations reference entities by name:

```
association Enrol [Student, Course]
```

Reference resolution is one of the most important (and error-prone) aspects of DSL implementation. Chapter 6 covers semantic design for references, and Chapter 10 implements resolution with typo suggestions.

Types

Even DSLs that are not "typed" in the programming-language sense often have a basic type system for validation. SQL has `INT`, `VARCHAR`, `DATE`. MSD has `INT`, `VARCHAR(n)`, `DATE`, and ten others. CSS has length units (`px`, `em`, `%`), colour values, and keywords.

Types serve two purposes in DSLs: they constrain what users can write (catching errors early) and they carry information that downstream tools need (a `VARCHAR(100)` column requires different storage from an `INT` column).

Constraints

Constraints are rules that restrict valid programs beyond what syntax alone can enforce. They are part of the *semantic* layer.

- **SQL**: foreign key constraints, `NOT NULL`, `UNIQUE`
- **MSD**: cardinality rules (a link must specify exactly one of `(0,1)`, `(0,N)`, `(1,1)`, `(1,N)`)
- **Terraform**: required attributes, valid values for `instance_type`

Constraints are what make a DSL *safe*. Without them, syntactically valid programs can produce nonsensical results.

Comments

Every DSL should allow human-readable annotations. Comment styles vary:

- **SQL**: `-- single line` and `/* multi-line */`
- **Terraform**: `#` and `//` for single-line, `/* */` for multi-line
- **MSD**: `// single line`

Comments seem trivial, but omitting them is a serious design mistake. Users *will* need to annotate their files, and without comments they will resort to ugly workarounds.

Metadata

Many DSLs include a mechanism for project-level or file-level information that does not directly affect the domain model but is needed by tools.

In **MSD**, a `project` block carries metadata:

```
project {
    name: "University Database"
    author: "Dr. Hamza"
}
```

In **Terraform**, the `terraform` block configures providers and backend settings. In **SQL**, `SET` commands configure session parameters.

Metadata is often the first thing users encounter in a file, so it should be easy to read and write.

2.5 What Makes a DSL Succeed?

Not every DSL thrives. For every SQL there are dozens of forgotten query languages. What separates the successes from the failures?

Domain Fit

The language must model its domain naturally. SQL succeeds because its core abstractions — tables, rows, columns, joins, filters — map directly onto the relational model. Users think in domain terms, and the language lets them express those terms without translation.

When a language forces users to think in implementation terms rather than domain terms, adoption suffers. The CODASYL network database model required users to navigate record-by-record through pointer chains — a faithful representation of the implementation but a poor model of *what users wanted to express*. SQL's declarative approach won because it matched how people think about data, not how machines store it.

Learnability

A well-designed DSL can be *read* before it can be *written*. A non-programmer should be able to look at a SQL `SELECT` statement or a Terraform resource block and grasp the intent, even without formal training.

This read-before-write property comes from choosing familiar vocabulary (`SELECT`, `FROM`, `WHERE`), consistent syntax, and meaningful defaults. It also comes from keeping the language small — the fewer concepts to learn, the faster the onboarding.

Tool Support

A DSL without decent error messages, editor support, or documentation is a DSL that frustrates its users into abandoning it. SQL benefits from decades of tooling: syntax highlighting, query planners, `EXPLAIN` output, IDE autocompletion. Terraform has `terraform validate`, `terraform plan`, and VS Code extensions.

Error messages deserve special attention. A DSL's error messages are often the primary way users learn the language. Saying "syntax error on line 7" teaches nothing; saying "expected attribute type after ':' on line 7, got '{' — did you forget the type?" teaches the language while fixing the problem.

Community Adoption

Languages benefit from network effects. Once SQL reached critical mass, every database had to support it, every programmer had to learn it, and every textbook had to teach it. This creates a virtuous cycle: more users produce more documentation, more tools, and more demand.

A new DSL faces a bootstrapping problem. It must be *significantly* better than the alternatives to overcome the switching cost. MSD, for instance, competes with graphical-only modelling tools by offering version control, diffability, and scriptability — advantages that a GUI cannot match.

Interoperability

A DSL that exists in isolation will remain a curiosity. Successful DSLs integrate with existing tools and workflows. SQL results feed into applications. Terraform outputs are consumed by CI/CD pipelines. CSS is embedded in HTML. MSD generates `.merisio` project files that the Merisio GUI can open, and produces SQL that databases can execute.

The CODASYL lesson. CODASYL's Data Manipulation Language was the dominant database approach in the 1970s. It failed not because it was technically inferior in all respects, but because it violated several of the principles above: it required imperative navigation rather than declarative queries (poor domain fit for most users), it was difficult to learn (low learnability), and its programs were tightly coupled to the physical data structure (poor interoperability). SQL offered a simpler mental model and won the market despite initially being slower.

2.6 Common DSL Anti-Patterns

Just as there are patterns that lead to success, there are anti-patterns that lead to failure. Recognising these early can save months of misguided design work.

The "Kitchen Sink"

Symptom: The DSL grows features until it becomes a general-purpose language, but without the tooling, documentation, or ecosystem of a real GPL.

This happens when designers respond to every user request by adding a new language feature. "We need conditionals." "We need loops." "We need user-defined functions." Each addition seems reasonable in isolation, but cumulatively they transform a focused DSL into a clumsy programming language.

Prevention: Before adding a feature, ask: "Is this a domain concept, or a general computation concept?" If it is the latter, consider providing an escape hatch to a GPL rather than implementing it in the DSL.

The "One Weird Trick"

Symptom: The syntax is so clever or unconventional that only the original author can read it fluently.

Regular expressions are sometimes cited here — the syntax is compact and powerful but notoriously difficult to read. APL-family languages take this further with symbolic operators that require specialised keyboards.

Prevention: Optimise for readability over conciseness. Remember that code is read far more often than it is written. A slightly more verbose syntax that reads like prose will always beat a terse syntax that reads like line noise.

The "Island Language"

Symptom: The DSL has no way to integrate with other tools, export its data, or compose with other languages.

An island language traps its users. If the only way to use a diagram DSL is through its proprietary GUI, users cannot version-control their models, cannot generate documentation automatically, and cannot incorporate the DSL into CI/CD pipelines.

Prevention: Design for interoperability from the start. Support standard file formats for input and output. Provide a CLI tool alongside any GUI. Make the language's files plain text so that `diff`, `grep`, and version control work naturally.

The "Error Desert"

Symptom: When users make mistakes, the language produces unhelpful, cryptic, or misleading error messages.

This is perhaps the most common anti-pattern and the most damaging. Users learn a DSL primarily through its error messages — every error is a teaching moment. An error message that says "parse error" with no location, no context, and no suggestion is a missed opportunity that drives users to frustration.

Prevention: Invest heavily in error messages. Include the file location (line and column). Describe what was expected and what was found. Suggest corrections where possible. Chapter 13 covers error handling in depth.

Warning

The "error desert" anti-pattern is particularly insidious because it often does not emerge until the language has real users making real mistakes. Designers who test only with correct inputs will never encounter their own bad error messages. Always test your DSL with *incorrect* inputs, deliberately introducing every kind of mistake you can imagine.

2.7 Exercises

Exercise 2.1 — Classify by Paradigm. Classify each of the following DSLs as declarative, imperative, or hybrid. Justify each classification in one or two sentences.

(a) JSON Schema — defines the structure that a JSON document must conform to.
(b) Ansible playbooks — describe a sequence of tasks to configure servers.
(c) HTML — describes the structure of a web page.
(d) CMake — orchestrates C/C++ build processes.
(e) PlantUML — generates UML diagrams from text descriptions.

Exercise 2.2 — Classify by Complexity. For each DSL below, determine whether it is a micro-DSL, medium DSL, or full DSL. Explain what features push it into (or keep it out of) the next complexity tier.

(a) `.gitignore` patterns
(b) YAML
(c) Lua (as embedded in game engines)
(d) CSS (including modern features like custom properties and `calc()`)
(e) MSD

Exercise 2.3 — Identify Patterns. Consider the following fragment of an imaginary DSL for defining REST APIs:

```
api UserService {
    // User management endpoints
    model User {
        id: UUID
        name: String
        email: Email
    }
```

```
    GET /users -> List<User>
    GET /users/{id} -> User
    POST /users (body: User) -> User
}
```

Identify which of the six common patterns from Section 2.4 (block structure, references, types, constraints, comments, metadata) are present in this fragment. For each pattern you identify, point to the specific syntax that implements it.

Exercise 2.4 — Success Factors. Choose a DSL you use regularly (it can be any of those mentioned in this chapter or another). Evaluate it against the five success factors from Section 2.5. Where does it excel? Where does it fall short? What one change would most improve its weakest area?

Exercise 2.5 — Anti-Pattern Detection. The following MSD fragment contains a hypothetical extension that introduces an anti-pattern. Identify the anti-pattern and explain why it is problematic.

```
entity Student {
    *student_id: INT
    name: VARCHAR(100)
    status: VARCHAR(20)

    computed average_grade {
        grades = SELECT avg(grade) FROM enrolments
                 WHERE student_id = self.student_id;
        if grades > 15 { return "excellent" }
        else { return "standard" }
    }
}
```

Exercise 2.6 — Design a Micro-DSL. Design a micro-DSL for expressing email filter rules. A rule should be able to match on sender, subject, or body content and specify an action (move to folder, mark as read, delete, or forward to an address). Write at least three example rules in your syntax, then classify your design by paradigm and argue why that paradigm is appropriate.

Summary

This chapter surveyed the landscape of external domain-specific languages, examining six influential examples — SQL, GraphQL, Terraform HCL, Dockerfile, Makefile, and CSS — and extracting general principles from their designs.

We developed two classification schemes. The **paradigm** classification distinguishes declarative DSLs (describing *what*), imperative DSLs (describing *how*), and hybrid DSLs (combining both). The **complexity** classification distinguishes micro-DSLs (tiny

grammars), medium DSLs (richer structure and types), and full DSLs (approaching general-purpose power).

We identified six **common patterns** — block structure, references, types, constraints, comments, and metadata — that recur across successful DSLs. We examined five **success factors** that determine whether a DSL thrives: domain fit, learnability, tool support, community adoption, and interoperability. And we catalogued four **anti-patterns** — the kitchen sink, the one weird trick, the island language, and the error desert — that lead to DSL failure.

In the next chapter, we narrow our focus to a specific domain. Chapter 3 introduces MERISE conceptual data modelling, analyses its requirements, and sets the stage for designing MSD — the DSL that we will build throughout the rest of this book.

Chapter 3: Requirements and Domain Analysis

A common temptation when designing a domain-specific language is to start with syntax. You imagine what code should look like, sketch a few examples, and begin writing a parser. This is exactly backwards. Syntax is the *expression* of domain knowledge — it cannot precede that knowledge. Before you write a single grammar rule, you must deeply understand the domain your language will serve.

This chapter is about the work that comes before language design. We will explore how to analyse a target domain, extract its core concepts and relationships, understand the people who will use the language, and systematically map domain knowledge onto language constructs. The techniques here apply to any DSL, but we will illustrate them throughout with the MERISE database modelling domain — the domain that motivates the MSD language we design in Chapter 7.

3.1 Understanding the Target Domain

Why Domain Analysis Must Precede Language Design

A DSL exists to serve a domain. If you misunderstand the domain, the language will be awkward at best and misleading at worst. Consider what happens when you skip domain analysis:

- You invent syntax that *looks nice* but does not map to real domain concepts.
- You omit constructs that practitioners need daily.
- You conflate distinct concepts because you did not realise they were different.
- You impose constraints that make no sense to domain experts.

The purpose of domain analysis is to build a **domain model** — a structured understanding of the concepts, relationships, and rules that govern the domain. This model becomes the blueprint for every design decision in your language.

The Danger of Designing Syntax First

When you start with syntax, you anchor yourself to surface-level decisions before understanding the deep structure. Suppose you are designing a language for cooking recipes. You might write:

```
mix flour, sugar, eggs for 5 minutes
```

This looks reasonable. But a domain expert would tell you that mixing is not a single operation — there is folding, whisking, beating, and kneading, each with different mechanics. There are also implicit requirements: ingredients must be prepared (sifted flour, room-temperature eggs) before mixing. A syntax-first approach would have missed these distinctions entirely, and retrofitting them later would distort the language.

The rule is simple: **understand first, design second**.

Working with Domain Experts vs Working Alone

If you have access to domain experts, involve them from the start. They will identify concepts you would miss, correct your misunderstandings, and validate your domain model. The collaboration typically follows this pattern:

1. **Interview**: Ask domain experts to describe their work, their vocabulary, and their pain points.
2. **Model**: Build a draft domain model and present it back to them.
3. **Refine**: Iterate until the model accurately reflects their understanding.
4. **Validate**: Show prototype syntax and ask whether it expresses their concepts naturally.

If you are working alone — perhaps because you *are* the domain expert — you must be especially disciplined. Document your assumptions explicitly. Search for edge cases. Study existing tools and standards in the domain. Your own familiarity can blind you to implicit knowledge that a language must make explicit.

Domain Modelling as the Foundation for Language Constructs

A domain model captures three things:

- **Concepts**: The nouns of the domain — the things that exist and that users reason about.
- **Operations**: The verbs — the actions performed on or between concepts.
- **Constraints**: The rules that determine which combinations of concepts and operations are valid.

Every element of your eventual language — its keywords, its syntax patterns, its validation rules — will trace back to one of these three categories. The domain model is not a preliminary step you do once and forget; it is the reference document you consult throughout the design and implementation process.

3.2 Identifying Domain Concepts

The first step in domain analysis is to identify the **concepts** — the things that exist in the domain. A systematic way to do this is to extract nouns, verbs, and adjectives from domain descriptions, expert interviews, and existing documentation.

Extracting Nouns (Entities and Objects)

Nouns correspond to the *things* in the domain — the objects, structures, and containers that users create, manipulate, and reason about. To find them, read through domain documentation and highlight every noun phrase.

For example, consider this description of MERISE conceptual modelling:

> "A conceptual data model consists of **entities** that represent real-world objects. Each entity has **attributes** that describe its properties. **Associations** represent relationships between entities. Each association is connected to entities through **links**, and each link has a **cardinality** that specifies how many instances participate."

The nouns are: entity, attribute, association, link, cardinality, model, instance. Not all nouns will become language constructs — "model" and "instance" are meta-level concepts rather than things the user defines — but this extraction gives you a starting vocabulary.

Extracting Verbs (Operations and Relationships)

Verbs tell you what users *do* with the domain concepts. They may become language constructs (if the user expresses them in the DSL) or tooling operations (if they happen outside the language). Common verbs include:

- **Define / declare**: Creating new instances of concepts (defining an entity, declaring an attribute).
- **Connect / link**: Establishing relationships between concepts.
- **Transform**: Converting from one representation to another (MCD to MLD).
- **Validate**: Checking that a model satisfies domain rules.

In the MERISE domain, the key verbs are: define (entities, associations), connect (via links), assign (cardinalities, attributes), transform (MCD to MLD to SQL), and validate (structural rules).

Extracting Adjectives (Properties, Attributes, and Constraints)

Adjectives describe the properties and qualifiers that distinguish instances of a concept. They often become attributes in your data model and parameters or modifiers in your language syntax.

For MERISE: an attribute can be a **primary key** (or not). A cardinality has a **minimum** and a **maximum**. An attribute has a **data type** and an optional **size**. An entity

has a **name**. These adjectives tell you what information the user needs to specify when defining each concept.

Creating a Domain Vocabulary

Once you have extracted nouns, verbs, and adjectives, organise them into a **domain vocabulary table**. This table becomes the authoritative reference for your language design.

Category	Term	Description
Noun	Entity	A named container representing a real-world object
Noun	Association	A named relationship between two or more entities
Noun	Link	A connection between one entity and one association
Noun	Attribute	A named property with a data type
Noun	Cardinality	The participation constraint on a link
Verb	Define	Create a new entity or association
Verb	Connect	Establish a link between an entity and an association
Verb	Assign	Give an attribute to an entity or association
Adjective	Primary key	Marks an attribute as part of the entity's identifier
Adjective	Data type	The type of data an attribute holds (INT, VARCHAR, etc.)
Adjective	Size	The length or precision of a data type
Adjective	Minimum	The lower bound of a cardinality (0 or 1)
Adjective	Maximum	The upper bound of a cardinality (1 or N)

Tip

Keep your domain vocabulary table alive throughout the project. As you learn more about the domain, add new terms. When you design syntax in later chapters, annotate each term with the language construct that expresses it. This table becomes the traceability link between the domain and the language.

Programmer's Perspective

Requirements gathering for a DSL closely mirrors API design in Go. When you design a Go package, you start by defining the types and method signatures that callers will use – effectively building the user's mental model before writing any implementation. The same discipline applies to DSLs: write ten realistic example files in your proposed syntax before implementing the lexer. This is the DSL equivalent of dogfooding. Go's own standard library follows this pattern rigorously; the `io.Reader` and `io.Writer` interfaces were designed around how users think about I/O, not around how the kernel implements it. Your DSL's keywords and constructs should similarly mirror how domain experts already think, not how your parser will process them.

3.3 Relationships and Constraints

How Domain Concepts Relate to Each Other

Concepts do not exist in isolation. The structure of a domain is defined as much by the *relationships* between concepts as by the concepts themselves. Understanding these relationships is critical because they determine the nesting, ordering, and grouping patterns in your language syntax.

In the MERISE domain, the key relationships are:

- An **entity** *contains* zero or more **attributes**.
- An **association** *contains* zero or more **carrying attributes**.
- A **link** *connects* exactly one **entity** to exactly one **association**.
- A **link** *has* exactly one **cardinality** (a minimum-maximum pair).
- An **attribute** *has* exactly one **data type**.

These relationships naturally suggest a hierarchical structure: entities and associations are top-level constructs, attributes are nested within them, and links express connections between the top-level constructs.

Cardinality of Relationships

Each relationship between domain concepts has a cardinality — how many of one concept can relate to another. This matters for language design because it determines whether a syntax element appears once, optionally, or in a list.

Relationship	Cardinality	Language implication
Entity contains attributes	One-to-many	Attribute list within entity block

Relationship	Cardinality	Language implication
Association contains attributes	One-to-many	Attribute list within association block
Link connects entity to association	One-to-one on each side	Link references exactly one of each
Attribute has data type	One-to-one	Type annotation is required, not a list
Cardinality has min and max	One-to-one	Single value pair, not a list

Constraints That Restrict Valid Combinations

Beyond structure, every domain has **constraints** — rules about what constitutes a valid model. Some constraints are structural (enforced by the syntax itself), while others are semantic (enforced by validation logic after parsing).

For the MERISE domain, the key constraints include:

1. Every entity must have a unique name within the model.
2. Every entity must have at least one primary key attribute.
3. Every association must be connected to at least two entities (via links).
4. Cardinalities must come from the set {(0,1), (0,N), (1,1), (1,N)}.
5. Attribute names must be unique within their owning entity or association.
6. An entity that is not connected to any association is an orphan (warning).

Which Constraints Belong in the Language vs in the Tooling?

Not every constraint should be enforced by the language grammar. There is a spectrum:

- **Lexical constraints** are enforced by the tokeniser. Example: an identifier must start with a letter.
- **Syntactic constraints** are enforced by the parser. Example: an entity block must contain braces.
- **Semantic constraints** are enforced by a post-parsing validation pass. Example: entity names must be unique.
- **Tooling constraints** are enforced by the environment. Example: a model should have at least one entity (but an empty file is still syntactically valid).

The general principle is: **push constraints as early as possible in the pipeline, but not earlier than they naturally belong**. Cardinalities being from a fixed set is naturally a lexical or syntactic constraint (the parser can reject invalid values). Entity name uniqueness is naturally a semantic constraint (the parser sees each entity independently). A model having no orphan entities is naturally a tooling-level warning.

Warning

Encoding too many constraints into the grammar makes the language rigid and the parser complex. Encoding too few makes invalid models silently accepted. Strike a balance: use the grammar for structural rules, and use semantic analysis for cross-referencing rules.

Programmer's Perspective

Backward compatibility is a critical concern even for small DSLs, and Go's approach to compatibility offers a useful model. Go 1's compatibility promise guarantees that valid Go 1.0 code compiles with Go 1.22, and Go modules use explicit version declarations (`go 1.21` in `go.mod`) to manage feature availability. If your DSL gains adoption, users will have existing files that must continue to work when the tool is updated. Design your DSL with a version field from day one (as MSD does with its `project {}` block), and treat syntax changes as seriously as you would treat breaking API changes in a Go library. Adding a keyword is straightforward; removing or redefining one is a migration nightmare.

3.4 User Personas and Workflow Analysis

Who Will Use the DSL?

A language is only as good as its fit for the people who use it. Before designing syntax, identify your **user personas** — the distinct categories of people who will interact with the language. For each persona, understand:

- Their technical background (programmer, domain expert, student, or mixed).
- Their familiarity with text-based languages (comfortable with code, or prefer GUIs).
- Their goals (authoring new models, reviewing existing ones, automating tasks).
- Their tolerance for complexity (will they learn a full syntax, or do they need simplicity?).

What Tasks Will They Perform?

Users rarely interact with a DSL in only one way. Typical tasks include:

- **Authoring**: Writing new models from scratch.
- **Editing**: Modifying existing models.
- **Reviewing**: Reading and understanding models written by others.
- **Transforming**: Converting models to other formats (SQL, diagrams, documentation).

- **Validating**: Checking models for correctness.
- **Versioning**: Tracking changes to models over time (version control).

Each task places different demands on the language. Authoring favours conciseness. Reviewing favours readability. Versioning favours line-oriented syntax with clean diffs.

What Tools Do They Already Use?

Your DSL does not exist in a vacuum. Users have existing workflows, editors, and tools. A DSL that integrates smoothly into these workflows will see adoption; one that demands a completely new environment will face resistance.

Consider: do your users work in text editors, IDEs, command-line terminals, or graphical applications? Do they use version control (Git)? Do they work in CI/CD pipelines? Each of these contexts influences language design decisions — a language intended for Git workflows should produce clean diffs; a language intended for CLI pipelines should support streaming input.

Example: Merisio User Personas

For the MSD language, we identified three primary personas:

Persona	Background	Primary task	Tool preference
CS student	Learning databases	Authoring MCD models	Text editor, GUI app
University teacher	Database expert	Reviewing student work	Text editor, CLI
Software developer	Programmer	Automating model generation	CLI, scripts, CI/CD

The **CS student** is learning MERISE as part of a database course. They need syntax that is easy to learn, with clear error messages that guide them when they make mistakes. They may use either the Merisio GUI or write MSD files directly.

The **university teacher** reviews dozens of student models. They need the language to be readable at a glance and to support batch validation. A CLI tool that validates a directory of `.msd` files and reports errors is essential for this persona.

The **software developer** wants to generate MERISE models programmatically — perhaps from an existing database schema or as part of a code generation pipeline. They need a machine-writable format that is also human-readable for debugging.

These three personas pull in different directions. The student wants simplicity; the developer wants precision; the teacher wants readability. A well-designed DSL finds the balance point that serves all three.

3.5 From Domain Model to Language Constructs

Once you have a domain model and an understanding of your users, you can systematically map domain concepts to language constructs. This mapping is the bridge between analysis and design.

The Mapping Principles

The mapping follows a consistent pattern:

- **Each noun** in the domain vocabulary becomes a **keyword** or **construct type** in the language.
- **Each relationship** between concepts becomes a **syntax pattern** — typically nesting, referencing, or sequencing.
- **Each constraint** becomes a **validation rule** — enforced at the lexical, syntactic, or semantic level.
- **Each adjective** becomes a **modifier**, **annotation**, or **parameter** in the syntax.

Design Table: Domain Concept to Language Construct

The most useful tool at this stage is a **design table** that traces each domain concept through to its language representation. Here is the design table for the MERISE MCD domain:

Domain concept	Language construct	Proposed syntax	Validation rule
Entity	`entity` keyword	`entity Name { ... }`	Name must be unique across all entities
Association	`association` keyword	`association Name { ... }`	Name must be unique across all associations
Attribute	Attribute declaration	`name: TYPE` or `name: TYPE(size)`	Name unique within owning block
Primary key	`*` prefix marker	`*id: INT`	At least one per entity
Link	`link` statement	`link Entity (0,N) Association`	References must resolve to defined names
Cardinality	Inline notation	`(0,1)`, `(0,N)`, `(1,1)`, `(1,N)`	Must be one of the four valid pairs
Data type	Type name	`INT`, `VARCHAR(100)`, `DATE`	Must be a recognised type name

This table serves multiple purposes. During design, it ensures that every domain concept has a corresponding language feature. During implementation, it tells the parser what to expect and the semantic analyser what to check. During testing, it defines what valid and invalid inputs look like.

Resolving Design Tensions

The mapping is rarely one-to-one. You will encounter tensions:

- **A domain concept is too complex for a single construct.** Solution: decompose it. The "link" concept in MERISE combines a connection and a cardinality. In the language, we express this as a single statement with two parts: the entity-association reference and the cardinality annotation.
- **Two domain concepts are similar but not identical.** Solution: decide whether they share syntax or have distinct syntax. Entities and associations both contain attributes, but entities have primary keys and associations do not. We use a shared attribute syntax inside both, but only allow the * marker inside entity blocks.
- **A constraint spans multiple constructs.** Solution: handle it in semantic analysis, not in the grammar. "Every entity must be connected to at least one association" cannot be checked until all entities and links have been parsed.

Note

The design table is a living document. You will revise it repeatedly as you work through lexical design (Chapter 4), syntactic design (Chapter 5), and semantic design (Chapter 6). Do not expect to get it right on the first pass.

3.6 Case Study: The MERISE Modelling Domain

What Is MERISE?

MERISE is a systems analysis and design methodology that originated in France in the late 1970s. It remains widely taught in French-speaking universities and is used in practice for database design across Europe and North Africa. MERISE provides a structured approach to information system design, moving from abstract conceptual models down to concrete physical implementations.

The Three Abstraction Levels

MERISE defines three levels of data modelling:

1. **MCD (Modele Conceptuel de Donnees)** — The conceptual data model. This is the most abstract level, describing *what* data exists and how it relates, without any concern for implementation. The MCD uses entities, associations, and links with cardinalities.

2. **MLD (Modele Logique de Donnees)** — The logical data model. This is derived mechanically from the MCD by applying transformation rules (e.g., a many-to-many association becomes a junction table). The MLD uses tables, columns, primary keys, and foreign keys.
3. **MPD (Modele Physique de Donnees)** — The physical data model. This is the MLD expressed as concrete SQL for a specific database engine (PostgreSQL, MySQL, etc.), including data types, indexes, and constraints.

The key insight for DSL design is that the MCD level is where human modelling decisions are made. The MLD and MPD levels are derived — they can be generated automatically from a well-specified MCD. This makes the MCD the natural target for a domain-specific language: capture the human decisions in text, then generate everything else.

The MCD Level in Detail

An MCD consists of:

- **Entities**: Named containers representing real-world objects or concepts. Each entity has one or more attributes, at least one of which is marked as the primary key. Example: a `Student` entity with attributes `student_id` (primary key), `name`, and `date_of_birth`.
- **Associations**: Named relationships between entities. An association may carry its own attributes (called *carrying attributes*). Example: an `Enrol` association between `Student` and `Course`, carrying a `grade` attribute.
- **Links**: Connections between entities and associations, each annotated with a cardinality. A cardinality is a pair (min, max) drawn from the set {(0,1), (0,N), (1,1), (1,N)}. Example: `Student` is linked to `Enrol` with cardinality (0,N), meaning a student may be enrolled in zero or more courses.

Why This Domain Is Well-Suited for a DSL

Several characteristics make MERISE an excellent candidate for a DSL:

- **Bounded complexity**: The domain has a small, well-defined set of concepts (entities, associations, links, attributes, cardinalities). A DSL can cover the entire domain without becoming sprawling.
- **Formal structure**: The MCD has precise rules about what constitutes a valid model. These rules map naturally to language constraints.
- **Repetitive patterns**: Every entity follows the same pattern (name, attributes, primary keys). Every link follows the same pattern (entity, cardinality, association). Repetitive structure is exactly what a DSL excels at expressing concisely.
- **Derivable outputs**: The MLD and SQL can be generated mechanically from the MCD. A text-based MCD format enables automation that a graphical-only tool cannot.

- **Version control need**: Graphical MERISE tools produce binary or complex JSON project files that are opaque to Git. A text-based DSL produces clean, diffable, reviewable files.

Existing Tools

Before designing a new DSL, it is essential to survey existing tools — both to understand what works and to identify gaps:

- **AnalyseSI** (Java): A desktop application widely used in French computer science education. Provides a graphical MCD editor with MLD generation. No text-based input format. Project files are XML.
- **Mocodo** (Python): A text-based tool that uses a minimal DSL for defining MCDs. Mocodo's syntax is compact but limited — it lacks explicit data types, has no primary key markers, and provides minimal error messages. It is designed for diagram generation rather than full database modelling.
- **Merisio** (Python/Qt): A visual MCD editor with MLD generation and PostgreSQL SQL export. Merisio stores projects as `.merisio` JSON files. It provides the graphical editing experience but lacked a text-based authoring format.

What Merisio Needed

The gap that motivated MSD was clear: Merisio needed a **text-based authoring format** that would enable:

- **Version control**: MCD models stored as plain text files in Git repositories, with meaningful diffs.
- **Code review**: Teachers and team leads reviewing model changes in pull requests.
- **Automation**: CI/CD pipelines that validate models, generate SQL, and export diagrams without launching the GUI.
- **Batch processing**: The `merisio-cli` tool processing multiple model files from the command line.
- **Round-trip fidelity**: A text file imported into Merisio's GUI, edited visually, and exported back to text should preserve the model faithfully.

This is the problem that MSD — the Merisio Schema Definition language — was designed to solve. In Chapter 7, we will design MSD in full, applying the lexical, syntactic, and semantic design principles from Chapters 4 through 6. The domain analysis in this chapter provides the foundation for that design.

3.7 Exercises

Exercise 3.1 — Domain Vocabulary Extraction

Choose a domain you know well — cooking recipes, board game rules, network configurations, knitting patterns, or music notation. Write a one-paragraph description of the domain as a domain expert would explain it. Then extract all nouns, verbs, and adjectives from your description and organise them into a domain vocabulary table with three columns: Category (noun/verb/adjective), Term, and Description.

Exercise 3.2 — Relationship Analysis

Using the domain you chose in Exercise 3.1, identify the relationships between concepts. For each relationship, state its cardinality (one-to-one, one-to-many, or many-to-many) and describe its language design implication (nesting, referencing, listing, or annotation).

Exercise 3.3 — Constraint Classification

List five constraints from the MERISE domain. For each one, classify it as lexical, syntactic, semantic, or tooling-level. Justify your classification by explaining *why* it belongs at that level rather than an adjacent one. If you disagree with the classification suggested in Section 3.3, argue your case.

Exercise 3.4 — User Persona Design

Imagine you are designing a DSL for defining REST API endpoints (routes, HTTP methods, request/response schemas). Identify three distinct user personas for this language. For each persona, describe their background, primary task, tool preferences, and what they would value most in the language (conciseness, readability, strictness, flexibility).

Exercise 3.5 — Design Table

Create a design table (as in Section 3.5) for a DSL that describes chess game positions. Your domain concepts should include at minimum: board, square, piece, colour, and position. Map each concept to a proposed language construct, syntax, and validation rule.

Exercise 3.6 — Existing Tool Survey

Choose an existing DSL you use regularly (SQL, CSS, Dockerfile, Makefile, or another). Identify three domain concepts that the language expresses well and one concept that it handles awkwardly or not at all. For the awkward concept, sketch an alternative syntax that would express it more naturally, and explain what trade-off the original language designers likely faced.

Chapter 4: Lexical Design

Every language begins as a stream of characters. Before a parser can build structure, before a semantic analyser can check meaning, the raw text must be broken into meaningful pieces. This process – **lexical analysis** – is the first stage of any language implementation, and the decisions you make at this stage shape the entire character of your DSL.

This chapter explores the principles of lexical design: how to choose the tokens, keywords, symbols, and conventions that define the look and feel of a domain-specific language. We use MSD as our primary case study throughout, but draw comparisons from SQL, Python, CSS, GraphQL, and other languages to illustrate the breadth of choices available to a language designer.

4.1 Tokens as the Atoms of a Language

A **token** (sometimes called a **lexeme**) is the smallest meaningful unit of a language. Just as chemistry reduces matter to atoms, lexical analysis reduces source text to tokens. A **lexer** (or **tokeniser** or **scanner**) is the component responsible for this transformation: it reads a stream of characters and produces a stream of tokens.

Consider the following MSD fragment:

```
entity Student {
    *student_id: INT
}
```

To a human reader, this clearly defines an entity called `Student` with a primary key attribute `student_id` of type `INT`. But to a parser, this is meaningless until the lexer has done its work. The lexer transforms the character stream into a token stream:

Token Type	Value
ENTITY	`entity`
IDENTIFIER	`Student`
LBRACE	`{`
NEWLINE	`\n`
STAR	`*`
IDENTIFIER	`student_id`
COLON	`:`
IDENTIFIER	`INT`

Token Type	Value
NEWLINE	`\n`
RBRACE	`}`
NEWLINE	`\n`
EOF	

Several things are worth noting. First, whitespace has been consumed – it served only to separate tokens and carries no further meaning. Second, the keyword `entity` has been recognised as a special token type (ENTITY) rather than a generic identifier. Third, each token records its position (line and column), which will be essential for error reporting later.

Why does lexical design matter? Because the tokens you choose define the **look and feel** of your language. They determine what users type, what they read, and how they think about the language. A language that uses `begin`/`end` for blocks feels fundamentally different from one that uses `{`/`}`, even if the two are semantically identical. The lexical layer is where aesthetics meets engineering.

Tip

Think of lexical design as the typography of your language. Just as a book's typeface affects readability without changing the content, your token choices affect usability without changing the language's expressive power.

4.2 Keywords

Keywords (or **reserved words**) are identifiers that have been given special meaning by the language. When the lexer encounters the character sequence `entity`, it does not produce a generic IDENTIFIER token – it produces an ENTITY token, because `entity` is a keyword.

How Many Keywords?

The number of keywords is a strong signal of a language's complexity. Fewer keywords generally mean a simpler language that is faster to learn:

Language	Approximate Keyword Count	Examples
MSD	4	`project`, `entity`, `association`, `link`
CSS	50+	`color`, `margin`, `display`, `flex`, ...
Python	35	`def`, `class`, `if`, `for`, `import`, ...

Language	Approximate Keyword Count	Examples
SQL	300+	`SELECT`, `FROM`, `WHERE`, `JOIN`, `ALTER`, ...

MSD has exactly four keywords. This is remarkably few, even for a DSL, and it reflects the language's narrow focus: MSD describes only entities, associations, links between them, and project metadata. There is no need for control flow (`if`, `while`), data manipulation (`SELECT`, `INSERT`), or module structure (`import`, `package`). Every keyword maps directly to a concept in the MERISE modelling domain.

Keyword Reservation

An important design decision is whether keywords are **reserved** – that is, whether users can use a keyword as an identifier name. In MSD, keywords are reserved: you cannot name an entity `entity` or an attribute `link`. This keeps the lexer simple and avoids ambiguity, but it does restrict the user's naming freedom.

Some languages take a different approach. In PL/I (a historically significant case), no words were reserved at all, which meant that statements like `IF IF = THEN THEN THEN = ELSE ELSE ELSE = IF` were syntactically valid. This is an extreme example, but it illustrates the spectrum: full reservation at one end, no reservation at the other.

For DSLs, full keyword reservation is almost always the right choice. The keyword count is small, and the simplicity it brings to the lexer and parser far outweighs the loss of a handful of identifier names.

Case Sensitivity for Keywords

Should `Entity`, `ENTITY`, and `entity` all be recognised as the same keyword? Languages differ:

- **Case-insensitive keywords**: SQL (`SELECT` and `select` are equivalent), MSD (`Entity` and `entity` are equivalent)
- **Case-sensitive keywords**: Python (`def` is a keyword, `Def` is not), Go, Rust, C

MSD chose case-insensitive keywords for a pragmatic reason: users coming from different backgrounds have different conventions. A French computer science student accustomed to writing `ENTITE` in uppercase might naturally type `ENTITY` or `Entity`. By accepting all casings, MSD removes a source of unnecessary errors.

The implementation is straightforward – the lexer normalises the word to lowercase before checking the keyword table:

```
keyword_type = KEYWORDS.get(word.lower())
```

Note

Case-insensitive keywords do not imply case-insensitive identifiers. In MSD, `entity` and `ENTITY` are the same keyword, but `Student` and `student` are different entity names. This split approach – case-insensitive keywords, case-sensitive identifiers – is the same strategy SQL uses for its keywords versus table and column names.

Programmer's Perspective

Token design in a DSL mirrors the naming conventions you apply when designing a Go API. Just as Go's exported identifiers start with an uppercase letter and unexported ones with lowercase, your DSL's keywords should follow intuitive conventions that reduce cognitive load. Go's `go/token` package is an excellent reference: it defines every token type as a named constant (`token.FUNC`, `token.RETURN`, `token.LBRACE`) and uses a lookup table to map identifier strings to keyword tokens – exactly the pattern MSD uses. If you are implementing a DSL in Go, modelling your `TokenType` enum and keyword map after `go/token` gives you a battle-tested design that handles reserved words, operator precedence metadata, and string representations cleanly.

Choosing Keywords That Read Naturally

Good keywords read like natural language in context. Consider MSD's `link` keyword:

```
link Student Enrol (1,N)
```

This reads almost like an English sentence: "link Student to Enrol with cardinality (1,N)." Compare this with a hypothetical alternative:

```
rel Student Enrol (1,N)
```

The abbreviation `rel` saves two characters but sacrifices readability. In a DSL where files are typically short and written infrequently, readability should almost always win over brevity.

Design guideline: Choose keywords that, when read aloud in a typical statement, sound like a description of what the statement does.

4.3 Identifiers

Identifiers are user-defined names: entity names, attribute names, association names, variable names. They are the tokens that the user has full control over, and the rules you set for identifiers determine what names are legal in your language.

Character Set Rules

Most languages allow identifiers to contain letters, digits, and underscores. The almost-universal restriction is on the **first character**: it must be a letter or under-

score, not a digit. This rule exists to keep the lexer simple – if a token starts with a digit, it is a number; if it starts with a letter or underscore, it is an identifier or keyword.

MSD follows this convention:

```
# Start of an identifier
if ch.isalpha() or ch == "_":
    start = col
    while col < length and (line_text[col].isalnum() or line_text[col] == "_"):
        col += 1
```

This means the following are all valid MSD identifiers:

```
Student
student_id
_internal
VARCHAR
```

But these are not:

```
2ndYear       # starts with a digit
student-id    # contains a hyphen
first.name    # contains a dot
```

Case Sensitivity for Identifiers

While MSD keywords are case-insensitive, MSD identifiers are **case-sensitive**. The entity `Student` and the entity `student` are two different things. This matches the convention of most programming languages (C, Java, Python, Go) and avoids a class of subtle bugs where `studentID` and `StudentId` might accidentally refer to the same thing.

Some DSLs choose case-insensitive identifiers – SQL is the most prominent example, where `SELECT * FROM Students` and `SELECT * FROM students` query the same table (unless quoting is used). This can be convenient but introduces its own complications: which casing should be used in output? How do you handle user-defined names that differ only in case?

Unicode Identifiers

Should your DSL allow Unicode characters in identifiers? Python 3, Java, and Swift all allow Unicode identifiers (`nombre_d_etudiant`, `nombre_d_etudiant`, or even `nombre_d_etudiant` with accented characters). This can be important for internationalisation – a French user might want to write `Etudiant` with an accent (`Etudiant` vs `Etudiant`).

MSD currently restricts identifiers to ASCII letters, digits, and underscores. This is a deliberate simplification: MERISE models are typically defined with ASCII identifiers even in French-language contexts, and supporting Unicode would complicate the lexer, parser, and output generator (particularly for SQL generation, where not all databases handle Unicode identifiers identically).

Naming Conventions

The lexer does not enforce naming conventions – it accepts any legal identifier regardless of style. But language documentation and examples establish conventions that users tend to follow:

- **PascalCase** (`StudentEnrolment`): MSD entity and association names
- **snake_case** (`student_id`): MSD attribute names
- **camelCase** (`studentId`): Common in JavaScript, Java
- **SCREAMING_SNAKE_CASE** (`MAX_SIZE`): Constants in many languages

MSD's examples consistently use PascalCase for entity and association names and snake_case for attributes. This mirrors SQL's common convention (PascalCase table names, snake_case column names) and reinforces MSD's role as a database modelling language.

4.4 Literals

Literals are values that appear directly in source code: numbers, strings, booleans. Different DSLs need different kinds of literals, and some DSLs need very few.

Integer Literals

MSD supports integer literals for a single purpose: specifying sizes in parameterised data types.

```
name: VARCHAR(100)
price: DECIMAL(10,2)
```

The numbers `100`, `10`, and `2` are integer literals. MSD does not need floating-point numbers, hexadecimal notation, or numeric separators – the only use of numbers in the language is to specify sizes and precisions, which are always small positive integers.

The lexer scans integer literals with a simple loop:

```
if ch.isdigit():
    start = col
    while col < length and line_text[col].isdigit():
        col += 1
    tokens.append(Token(TokenType.INTEGER, line_text[start:col], line_num, start + 1))
```

String Literals

String literals are values enclosed in quotation marks. Different languages use different quoting strategies:

- **Double quotes**: `"hello"` (C, Java, JSON)
- **Single quotes**: `'hello'` (SQL, Python)
- **Backticks**: `` `hello` `` (Go raw strings, SQL identifier quoting)
- **No quotes**: Values are captured without delimiters

MSD takes an unusual approach: inside `project {}` blocks, values after the colon are captured as raw strings without any quoting:

```
project {
    name: University Database
    author: Dr. Smith
    version: 2.1
}
```

The values `University Database`, `Dr. Smith`, and `2.1` are all captured as STRING_VALUE tokens without requiring quotation marks. This works because MSD's project block has simple, single-line key-value semantics – the value is everything from the colon to the end of the line (minus any trailing comment).

This design eliminates a common source of user errors: forgetting to close a string, using the wrong quote character, or needing to escape quotes within a value. Since project metadata values never need to span multiple lines or contain colons, the quoting-free approach is both simpler and less error-prone.

When Does a DSL Need Literals?

Not every DSL needs every kind of literal. Consider:

DSL	Integer	Float	String	Boolean	Other
MSD	Yes	No	Yes*	No	–
SQL	Yes	Yes	Yes	Yes	Date, NULL
CSS	Yes	Yes	Yes	No	Colours, units (px, em)
Dockerfile	No	No	Yes	No	–
Makefile	No	No	Yes	No	–

*MSD string literals are unquoted rest-of-line values, used only inside `project {}` blocks.

The lesson is clear: include only the literal types your domain requires. Each additional literal type adds complexity to the lexer and cognitive load for the user. MSD needs integers for type sizes and strings for project metadata – nothing more.

4.5 Symbols and Punctuation

Symbols and punctuation are the structural glue of a language. They delimit blocks, separate items, annotate types, and mark special attributes. Getting symbols right is largely a matter of choosing the right balance between familiarity and domain specificity.

Single-Character Tokens

MSD uses seven single-character symbols:

Symbol	Token Type	Meaning
`{`	LBRACE	Opens a block (entity body, project body)
`}`	RBRACE	Closes a block
`(`	LPAREN	Opens a group (cardinality, type size)
`)`	RPAREN	Closes a group
`:`	COLON	Separates name from type, key from value
`,`	COMMA	Separates items (cardinality min from max)
`*`	STAR	Marks primary key attributes

Each symbol was chosen for its familiarity. Braces `{}` for blocks come from C, Java, JavaScript, and Go. Parentheses `()` for grouping are universal across mathematics and programming. The colon `:` for type annotation follows Python, TypeScript, and Pascal. The asterisk `*` for marking primary keys follows the convention used in database modelling diagrams, where primary key attributes are traditionally underlined or starred.

Multi-Character Tokens

Many languages use multi-character symbols:

Symbol	Languages	Meaning
`//`	C, Java, MSD	Line comment
`->`	C, Rust, PHP	Arrow (dereference, return)
`<=`	Many	Less than or equal
`!=`	Many	Not equal
`::`	Rust, Haskell	Path separator, type annotation
`=>`	JS, PHP	Arrow function, array mapping
`...`	JS, Python	Spread operator, ellipsis

MSD has only one multi-character token: `//` for line comments. This reflects the language's simplicity – there are no operators, no comparisons, no arrows. MSD is a declarative data definition language, not a computational one.

Choosing Symbols: Familiarity vs Domain Conventions

When selecting symbols, favour conventions that your target audience already knows. MSD's users are primarily computer science students and database practitioners who are familiar with C-family syntax. Using `{}` for blocks and `:` for type annotations leverages their existing knowledge.

However, domain conventions can override programming conventions. MSD uses `*` for primary keys because this is how primary keys are conventionally marked in MERISE diagrams – with an asterisk or underline. A programmer unfamiliar with MERISE might expect `*` to mean multiplication or pointer dereference, but MSD's target audience immediately recognises its meaning.

Here is a table of common symbols and their typical meanings across DSLs:

Symbol	Database DSLs	Programming Languages	Configuration DSLs
`{}`	Block delimiters	Block delimiters	Block delimiters
`()`	Parameter lists, sizes	Function calls, grouping	–
`:`	Type annotation	Type annotation, slice	Key-value separator
`,`	List separator	List separator	–
`;`	Statement terminator	Statement terminator	–
`*`	Primary key, wildcard	Multiplication, pointer	Glob wildcard
`=`	–	Assignment	Value assignment

Warning

Avoid reusing a symbol with two very different meanings in the same language. If `*` means "primary key" in one context and "wildcard" in another, users will be confused. MSD uses `*` exclusively for primary keys, avoiding ambiguity.

Programmer's Perspective

Ambiguity in token design has a direct impact on parser complexity, and Go's language design illustrates this beautifully. Go deliberately chose unambiguous tokens: the walrus operator `:=` for short variable declarations is syntactically distinct from `=` for assignment, and the language avoids overloading symbols with context-dependent meanings. This is why Go's parser is fast and its error messages are clear. When designing your DSL, resist the temptation to overload punctuation. MSD's choice to use `*` exclusively for primary keys and `()` for both cardinalities and type sizes works only because these appear in entirely different grammatical positions. If they could appear adjacently, the lexer would need context-sensitive rules that complicate both implementation and user comprehension.

4.6 Whitespace and Indentation

How a language treats whitespace is one of the most visible and debated aspects of its design. There are two fundamental approaches.

Insignificant Whitespace

In languages with **insignificant whitespace**, spaces, tabs, and blank lines serve only to separate tokens. The parser ignores them entirely. This means the following MSD fragments are all equivalent:

```
entity Student { *student_id: INT name: VARCHAR(100) }
```

```
entity Student {
    *student_id: INT
    name: VARCHAR(100)
}
```

```
entity    Student
{
    *student_id:    INT
    name:           VARCHAR(100)
}
```

C, Java, JavaScript, Go, SQL, and MSD all use insignificant whitespace. The advantage is flexibility: users can format their code however they prefer, and the language works the same regardless. Tools like auto-formatters can reformat code without changing its meaning.

Significant Whitespace

In languages with **significant whitespace**, indentation carries structural meaning. Python is the most prominent example:

```python
def greet(name):
    if name:
        print(f"Hello, {name}")
    else:
        print("Hello, world")
```

Removing the indentation changes the program's meaning – or more likely, causes a syntax error. YAML, Haskell (in its layout-sensitive mode), and Makefile (which requires tabs for recipe lines) also use significant whitespace.

Comparing the Two Approaches

Aspect	Insignificant Whitespace	Significant Whitespace
Visual structure	Optional (by convention)	Mandatory (by grammar)

Aspect	Insignificant Whitespace	Significant Whitespace
Parser complexity	Simpler	More complex (indent tracking)
Formatting freedom	High	Constrained
Copy-paste safety	Robust	Fragile (indentation lost)
Delimiter characters	Needed (`{}`, `begin/end`)	Not needed
Readability	Depends on discipline	Enforced consistency

The MSD Choice

MSD uses insignificant whitespace. Indentation is conventional – the examples in this book indent block contents by four spaces – but the lexer does not enforce it. This decision was made for several reasons:

1. **Simplicity**: The lexer does not need to track indentation levels or emit INDENT/DEDENT tokens.
2. **Robustness**: Users can copy-paste MSD fragments without worrying about indentation.
3. **Familiarity**: MSD's target audience (computer science students) is accustomed to brace-delimited blocks from C and Java.

That said, MSD's lexer does emit NEWLINE tokens. These are not strictly necessary for a language with insignificant whitespace, but they serve a practical purpose: they allow the parser to provide accurate line numbers in error messages and to handle the line-oriented nature of the `project {}` block, where each key-value pair occupies a single line.

4.7 Comments

Comments are text that the lexer recognises and discards. They carry no meaning in the language itself, but they are essential for users: comments explain intent, provide documentation, temporarily disable code, and annotate models with domain knowledge.

Line Comments

A **line comment** starts with a special marker and extends to the end of the line. Different languages use different markers:

Marker	Languages
#	Python, Shell, Ruby, YAML, TOML, MSD

Marker	Languages
`//`	C, C++, Java, JavaScript, Go, Rust, MSD
`--`	SQL, Haskell, Lua, Ada
`%`	LaTeX, Erlang, Prolog

MSD supports **both** `#` and `//` line comments:

```
# This is a comment using the hash style
entity Student {
    *student_id: INT   // This is a comment using the double-slash style
    name: VARCHAR(100) # Both styles work
}
```

Why offer two styles? The reasoning is practical rather than theoretical. MSD is a data definition language used in educational contexts. Students coming from Python or Shell scripting instinctively reach for `#`. Students coming from C or Java instinctively reach for `//`. By accepting both, MSD meets users where they are, reducing friction and avoiding a class of "why doesn't my comment work?" errors.

The implementation is straightforward – the lexer checks for both markers:

```
# Comments: # or //
if ch == "#":
    break  # skip rest of line
if ch == "/" and col + 1 < length and line_text[col + 1] == "/":
    break  # skip rest of line
```

Block Comments

Block comments start with one marker and end with another, potentially spanning multiple lines:

```
/* This is a
   multi-line block comment
   in C */

{- This is a
   multi-line block comment
   in Haskell -}
```

MSD does not support block comments. This is a deliberate omission: MSD files are typically short (tens of lines, not hundreds), and line comments suffice for all practical purposes. Block comments add complexity to the lexer (they require tracking nesting depth if you want to support nested block comments) without providing proportional value.

Documentation Comments

Some languages provide a special comment syntax for documentation that can be extracted by tools:

```
/**
 * Represents a student in the university system.
 * @param name The student's full name
 */

/// Represents a student in the university system.
///
/// # Examples
/// ...
```

MSD does not have a documentation comment syntax. However, since MSD files describe database schemas, the `project {}` block serves a similar purpose – it provides structured metadata (name, author, version) that tools can extract and display. If a future version of MSD needed richer documentation, adding a `///` doc-comment syntax would be a natural extension.

4.8 Context-Sensitive Lexing

In most languages, the lexer is **context-free**: the same character sequence always produces the same token, regardless of where it appears. The character `{` is always LBRACE, the character `:` is always COLON, and the word `entity` is always the ENTITY keyword.

MSD mostly follows this rule, with one notable exception: **the colon token behaves differently inside `project {}` blocks**.

The Problem

Consider these two MSD fragments:

```
project {
    name: University Database
}

entity Student {
    *student_id: INT
}
```

In the entity block, the colon separates an attribute name from its type: `student_id` COLON `INT`. The parser expects an IDENTIFIER token after the colon.

In the project block, the colon separates a metadata key from a free-form value: `name` COLON `University Database`. The value is not a single identifier – it is an arbitrary string that may contain spaces, punctuation, and anything else.

If the lexer treated the colon the same way in both contexts, it would tokenise `University Database` as two separate IDENTIFIER tokens (`University` and `Database`), and the parser would need complex rules to reassemble them. A simpler approach is for the lexer to recognise the context.

The Solution

MSD's lexer tracks whether it is currently inside a `project {}` block. When it encounters a colon inside such a block, it captures everything after the colon (up to a comment marker or end of line) as a single STRING_VALUE token:

```
# Colon -- context-sensitive in project blocks
if ch == ":":
    tokens.append(Token(TokenType.COLON, ":", line_num, col + 1))
    col += 1

    if in_project_block:
        # Capture rest of line as STRING_VALUE
        rest = line_text[col:].strip()
        # Strip trailing comment
        for comment_start in ("#", "//"):
            idx = rest.find(comment_start)
            if idx >= 0:
                rest = rest[:idx].rstrip()
        if rest:
            tokens.append(Token(TokenType.STRING_VALUE, rest, line_num, col + 1))
        break  # consumed rest of line
    continue
```

The `in_project_block` flag is set when the lexer encounters the `project` keyword and cleared when the matching `}` is reached.

Keep Context Sensitivity Minimal

Context-sensitive lexing is powerful but dangerous. Each piece of context the lexer must track is a piece of state that can go wrong, and it makes the lexer harder to reason about. MSD has exactly one context-sensitive rule, and it is well-contained: the flag is set on one keyword and cleared on one brace.

Info

Context-sensitive lexing is more common than you might think. JavaScript's template literals (`` `Hello, ${name}` ``) require the lexer to switch between "string mode" and "expression mode". Python's f-strings have similar complexity. Ruby's heredocs, Perl's regular expression literals, and XML's CDATA sections are all examples of context-sensitive lexing in production languages. The key is to use it sparingly and to keep the context-switching rules simple.

Other DSL designs might avoid context sensitivity entirely. Section 4.10 includes an exercise asking you to redesign MSD's project block without context-sensitive lexing – for instance, by requiring quoted strings or a different block syntax.

4.9 Design Trade-Offs

Every lexical decision involves trade-offs. This section summarises the key tensions and how MSD navigates them.

Familiarity vs Precision

Using symbols and conventions from well-known languages makes your DSL easier to learn. But familiar symbols can also mislead: if * means "pointer dereference" in C and "primary key" in MSD, a C programmer might be momentarily confused.

MSD leans toward familiarity for structural elements (`{}`, `()`, `:`) and uses domain-specific conventions only where the domain meaning is strong (* for primary keys). This hybrid approach works because MSD's target audience – computer science students studying databases – is familiar with both worlds.

Brevity vs Readability

Short tokens are faster to type but harder to read. Consider these alternatives for marking a primary key:

Syntax	Style	Example
`*`	Symbol	`*student_id: INT`
`@pk`	Decorator	`@pk student_id: INT`
`primary_key`	Keyword	`primary_key student_id: INT`

MSD chose * because primary key marking is extremely common in MSD files (almost every entity has one), and a single character is the most concise option. The meaning is unambiguous because * has no other meaning in MSD. For a less frequently used feature, a more verbose syntax might be preferable.

Consistency vs Convenience

A consistent language applies the same rules everywhere. A convenient language makes exceptions when they improve usability. MSD's context-sensitive colon is a convenience that breaks consistency – the same character means different things in different contexts. The alternative (requiring quoted strings in project blocks) would be more consistent but less convenient for users.

Summary of MSD's Lexical Choices

Choice	MSD Decision	Rationale
Number of keywords	4	Minimal; one per domain concept
Keyword case sensitivity	Case-insensitive	Accepts `entity`, `Entity`, `ENTITY`
Identifier case sensitivity	Case-sensitive	`Student` and `student` are different
Identifier characters	ASCII letters, digits, underscore	Simple; sufficient for database identifiers
Block delimiters	`{ }`	Familiar to C/Java audience
Type annotation	`:`	Familiar from Python/TypeScript
Primary key marker	*	Concise; domain convention from MERISE diagrams
Comment styles	`#` and `//`	Dual support for Python and C-family users
String literals	Unquoted (project block only)	Eliminates quoting errors; sufficient for metadata
Whitespace	Insignificant	Flexible formatting; simpler lexer
Context sensitivity	Colon in project blocks only	Minimal; enables unquoted string values

4.10 Exercises

Exercise 4.1 – *Keyword Design*

Design the keyword set for a DSL that describes cooking recipes. The language should express ingredients, quantities, preparation steps, cooking times, and temperatures. What keywords would you choose? How many do you need? Should keywords be case-sensitive? Write three example "programs" in your proposed syntax.

Exercise 4.2 – *Eliminating Context Sensitivity*

The MSD lexer uses context-sensitive behaviour for the colon token: inside `project {}` blocks, it triggers rest-of-line capture as a STRING_VALUE; outside, it is a simple separator. Propose an alternative design that avoids this context sensitivity entirely. Your design must still allow project metadata values that contain spaces (e.g., `University Database`). What trade-offs does your design introduce? Consider at least two alternatives (e.g., quoted strings, a different separator, a different block syntax).

Exercise 4.3 – *Symbol Overloading*

MSD uses parentheses `()` for two purposes: cardinalities in link statements (`(1,N)`) and type sizes in attribute declarations (`VARCHAR(100)`). Could these two uses ever be ambiguous? If so, design an alternative syntax for one of the uses that eliminates the ambiguity. If not, explain why the two uses can always be distinguished.

Exercise 4.4 – *Comment Style Survey*

Survey at least five DSLs or configuration formats you have used (e.g., SQL, CSS, JSON, YAML, TOML, Dockerfile, Makefile, .ini files). For each, document: (a) what comment syntax it supports, (b) whether it supports block comments, and (c) whether comments can appear anywhere or only in specific positions. What patterns do you observe?

Exercise 4.5 – *Significant Whitespace Redesign*

Redesign MSD's syntax to use significant whitespace (Python-style indentation) instead of braces. Write the lexical rules for your redesigned language, including how INDENT and DEDENT tokens would be generated. Rewrite the following MSD fragment in your new syntax:

```
entity Student {
    *student_id: INT
    name: VARCHAR(100)
    email: VARCHAR(255)
}

association Enrol {
    enrolment_date: DATE
}

link Student Enrol (1,N)
link Course Enrol (0,N)
```

What are the advantages and disadvantages of your redesign compared to the brace-based original?

Exercise 4.6 – *Unicode Identifiers*

MSD currently restricts identifiers to ASCII characters. Suppose you were internationalising MSD for use in universities worldwide, where entity names might include accented characters (e.g., `Etudiant`, `Universite`), Arabic script, or Chinese characters. What changes would you need to make to the lexer? What new problems would Unicode identifiers introduce for downstream stages (SQL generation, file I/O, display)? Would you allow all of Unicode, or restrict to specific categories (e.g., Unicode letters only)?

Exercise 4.7 – *Token Design for a Query DSL*

Design the complete token set (keywords, symbols, identifier rules, literal types, comment syntax) for a DSL that queries a music library. The language should support queries like "find all albums by Artist X released after 2010 with rating above 4 stars." List every token type, give examples of each, and justify your choices. Pay particular

attention to how you handle string values that contain spaces (artist names, album titles).

Chapter 5: Syntactic Design

The previous chapter established the lexical layer — the vocabulary of a language. Tokens are the words, but words alone do not make sentences. *Syntax* is the grammar that arranges tokens into meaningful structures. It is the difference between a bag of words and a well-formed program.

This chapter explores the principles and techniques of syntactic design for domain-specific languages. We shall formalise grammars using EBNF notation, understand why LL(1) grammars are particularly well suited to hand-written parsers, and examine the design decisions that shape how users experience a language: block structure, line orientation, nesting depth, and delimiter choice. Throughout, we use MSD as our running case study, but the principles apply to any DSL you might design.

5.1 From Tokens to Structure

A lexer produces a flat stream of tokens. The parser's job is to impose *structure* on that stream — to recognise which tokens belong together, how they nest, and what they mean in relation to one another.

Consider the following MSD fragment:

```
entity Student {
    *student_id: INT
    name: VARCHAR(100)
}
```

The lexer produces tokens: `ENTITY`, `IDENTIFIER("Student")`, `LBRACE`, `STAR`, `IDENTIFIER("student_id`
`COLON`, `IDENTIFIER("INT")`, `IDENTIFIER("name")`, `COLON`, `IDENTIFIER("VARCHAR")`, `LPAREN`, `INTEGER("100")`, `RPAREN`, `RBRACE`. But this flat list tells us nothing about which attributes belong to which entity, or which identifier is a name and which is a type. The parser discovers that structure.

Why syntax matters beyond correctness. Syntax is not merely a technical concern — it determines how users *think* about the language. A language with deeply nested, bracket-heavy syntax feels different from one with clean, line-oriented declarations. The syntax of SQL (`SELECT ... FROM ... WHERE ...`) encourages users to think in terms of data queries. The syntax of CSS (`selector { property: value; }`) encourages users to think in terms of styling rules applied to elements. Your syntactic choices shape the mental model that users build.

The relationship between grammar and parser. A grammar is a *specification* — a precise description of which token sequences are valid. A parser is an *implementation* — a program that checks whether a given token sequence conforms to the grammar

and, if so, builds a structured representation. The grammar comes first; the parser follows. In practice, the two influence each other: certain grammar styles are easier to parse than others, and practical parsing constraints may feed back into grammar design.

5.2 EBNF Notation

To specify a grammar precisely, we need a notation. The standard choice is **Extended Backus-Naur Form** (EBNF), a notation developed from the original BNF used to define ALGOL 60. EBNF adds constructs for optional elements, repetition, and grouping, making grammars more concise and readable.

Notation Reference

Symbol	Meaning	Example
`=`	Definition (is defined as)	`entity_block = ...`
`|`	Alternative (or)	`"0" | "1"`
`[ ]`	Optional (zero or one)	`[ "*" ]`
`{ }`	Repetition (zero or more)	`{ attribute }`
`( )`	Grouping	`( "0" | "1" )`
`" "`	Terminal (literal token)	`"entity"`
`;`	End of production	`entity_block = ... ;`

Non-terminals (rule names) are written in lowercase with underscores. Terminals are quoted strings representing literal tokens or token types.

Example: An Entity Block in MSD

Let us write the EBNF grammar for an MSD entity block:

```
entity_block = "entity" IDENTIFIER "{" { attribute } "}" ;

attribute    = [ "*" ] IDENTIFIER ":" type_expr ;

type_expr    = TYPE_NAME [ "(" INTEGER ")" ] ;
```

Reading these productions aloud:

- An **entity block** is the keyword `entity`, followed by an identifier (the entity name), followed by a left brace, followed by zero or more attributes, followed by a right brace.

- An **attribute** is an optional asterisk (marking a primary key), followed by an identifier (the attribute name), followed by a colon, followed by a type expression.
- A **type expression** is a type name, optionally followed by a size in parentheses.

Writing EBNF Productions

When writing EBNF, follow these guidelines:

1. **Start with the top-level rule.** What is the outermost structure of a valid file?
2. **Break complex rules into sub-rules.** If a production is growing unwieldy, extract a named sub-rule.
3. **Use repetition { } for lists.** Avoid recursive definitions when iteration suffices.
4. **Use alternatives | for choices.** Each alternative should begin with a distinct token to aid parsing.
5. **Name rules after what they represent**, not how they are parsed.

Why Formal Grammars Matter

An informal specification such as "an entity has a name and some attributes inside braces" is ambiguous. Are braces required? Can an entity have zero attributes? Can the name be omitted? A formal EBNF grammar answers all of these questions unambiguously. It serves as a contract between the language designer and the parser implementer — and when you are both, it serves as a contract between your design self and your implementation self.

Tip

Write your EBNF grammar *before* you write your parser. The grammar is your blueprint. Changing a grammar rule is far cheaper than refactoring a parser.

Programmer's Perspective

EBNF notation is to parser implementation what an interface definition is to Go code. Just as a Go `interface` specifies the contract a type must satisfy without dictating the implementation, an EBNF grammar specifies which token sequences are valid without dictating how the parser processes them. Go's own grammar is published formally in the language specification, and the `go/parser` package is a direct translation of that grammar into recursive descent code. When you design your DSL, write the EBNF first and treat it as a binding contract: every production rule becomes a function, every alternative becomes a `switch` or `if` block, and every repetition becomes a `for` loop. This mechanical correspondence is what makes recursive descent parsers so maintainable.

5.3 LL(1) Grammars and Why They Matter

What LL(1) Means

The notation **LL(1)** stands for:

- **L** — Left-to-right scanning of the input (we read tokens from left to right)
- **L** — Leftmost derivation (we always expand the leftmost non-terminal first)
- **1** — One token of lookahead (we decide which production to apply by examining only the next token)

An LL(1) grammar is one where, at every point in parsing, looking at the single next token is sufficient to determine which production rule to apply. There is never any ambiguity or backtracking required.

Why LL(1) Is Ideal for Hand-Written Parsers

When you write a recursive descent parser — a parser where each grammar rule becomes a function — the LL(1) property is invaluable. Each parsing function can inspect the next token and immediately decide what to do:

```
def _parse_top_level(self):
    tok = self._peek()
    if tok.type == TokenType.ENTITY:
        self._parse_entity_block()
    elif tok.type == TokenType.ASSOCIATION:
        self._parse_association_block()
    elif tok.type == TokenType.LINK:
        self._parse_link_statement()
    elif tok.type == TokenType.PROJECT:
        self._parse_project_block()
```

One token of lookahead. No backtracking. No ambiguity. This is the direct consequence of MSD having an LL(1) grammar.

Checking LL(1): FIRST and FOLLOW Sets

To verify that a grammar is LL(1), compiler theory defines two functions:

- **FIRST(A)**: the set of tokens that can appear at the beginning of any string derived from rule A.
- **FOLLOW(A)**: the set of tokens that can appear immediately after a string derived from A.

A grammar is LL(1) if, for every rule with alternatives `A = B | C`, the FIRST sets of B and C are disjoint. Intuitively: if two alternatives can both start with the same token, the parser cannot tell them apart with one token of lookahead.

For MSD's top-level rule:

```
program      = { top_level } ;
top_level    = project_block | entity_block | association_block | link_stmt ;
```

The FIRST sets are:

- FIRST(project_block) = { `"project"` }
- FIRST(entity_block) = { `"entity"` }
- FIRST(association_block) = { `"association"` }
- FIRST(link_stmt) = { `"link"` }

These four sets are completely disjoint — each alternative starts with a unique keyword. The grammar is LL(1) at the top level.

When a Grammar Is Not LL(1)

Two common problems prevent a grammar from being LL(1):

Left recursion. A rule that refers to itself as its first element:

```
expr = expr "+" term | term ;
```

This is left-recursive because `expr` appears as the first symbol on the right-hand side of its own definition. A recursive descent parser would call `_parse_expr()`, which would immediately call `_parse_expr()` again, creating infinite recursion.

Common prefixes. Two alternatives that begin with the same tokens:

```
stmt = "if" expr "then" stmt ;
     | "if" expr "then" stmt "else" stmt ;
```

Both alternatives start with `"if"`, so looking at the next token (`if`) does not tell the parser which rule to apply.

Transforming Non-LL(1) Grammars

Removing left recursion. The left-recursive rule:

```
expr = expr "+" term | term ;
```

is transformed into:

```
expr      = term { "+" term } ;
```

The recursive call is replaced by iteration, which a hand-written parser handles with a `while` loop.

Left factoring. The common-prefix rule:

```
stmt = "if" expr "then" stmt [ "else" stmt ] ;
```

The shared prefix `"if" expr "then" stmt` is factored out, and the `"else"` clause becomes optional. The parser consumes the common prefix, then checks whether an `"else"` follows.

Note

MSD's grammar is naturally LL(1) without any transformations. Each top-level construct starts with a unique keyword (`project`, `entity`, `association`, `link`), and within blocks, the structure is regular enough that one token of lookahead always suffices. This is not an accident — it is a deliberate design choice that simplifies the parser enormously.

Programmer's Perspective

Go's grammar is deliberately LL(1) for the same reason MSD's is: fast, predictable parsing with no backtracking. This is why Go compiles faster than almost any other compiled language at scale. The Go specification explicitly avoids ambiguous constructs – there is no ternary operator, no implicit type conversions, and statement-ending semicolons are inserted automatically by the lexer so that the parser never has to guess where a statement ends. If you are designing a DSL in Go, you can leverage Go's own parsing infrastructure directly: the `go/scanner` package handles LL(1) token scanning with automatic semicolon insertion, and its source code is a concise, readable example of how to build a production-quality lexer for an LL(1) grammar.

5.4 Block-Structured Syntax

Nearly every language groups related declarations into *blocks*. The question is: how are those blocks delimited?

Curly-Brace Blocks

Used by: C, C++, Java, Go, JavaScript, CSS, Terraform HCL, MSD.

```
// MSD
entity Student {
    *student_id: INT
    name: VARCHAR(100)
}
```

```
// Terraform HCL
resource "aws_instance" "web" {
    ami           = "ami-0c55b159cbfafe1f0"
    instance_type = "t2.micro"
}
```

Advantages: Explicit scope boundaries. The parser can always identify where a block begins and ends. Indentation is for humans, not the machine — misindented code still parses correctly.

Disadvantages: Visual noise from brace characters. Debates about brace placement style (same line vs next line).

Indentation-Based Blocks

Used by: Python, YAML, Haskell (layout rule), Nim, CoffeeScript.

```yaml
# YAML
student:
    id: 1
    name: "Alice"
    enrolled_in:
        - "Mathematics"
        - "Physics"
```

Advantages: Clean visual appearance. What you see is what the compiler sees — indentation is the structure.

Disadvantages: Whitespace sensitivity creates subtle bugs (tabs vs spaces, inconsistent indentation). Copy-pasting code can break structure. Parsing is more complex because the lexer must track indentation levels and emit synthetic INDENT/DEDENT tokens.

Keyword-Based Blocks

Used by: Pascal (`begin`/`end`), Ruby (`do`/`end`), SQL (`BEGIN`/`END`), Ada, VHDL.

```pascal
// Pascal
procedure Greet;
begin
    WriteLn('Hello');
end;
```

Advantages: Very explicit. Good for languages where blocks are infrequent.

Disadvantages: Verbose. The keywords `begin` and `end` consume visual space without adding information beyond what braces provide more concisely.

MSD's Choice: Curly Braces

MSD uses curly braces for blocks:

```
entity Student {
    *student_id: INT
    name: VARCHAR(100)
}

association enrolled_in {
    enrollment_date: DATE
}
```

The reasons are:

1. **Familiarity.** Most MSD users will have encountered C-family languages, JSON, or CSS. Curly braces are immediately understood.
2. **Explicit structure.** The parser can unambiguously find block boundaries without tracking whitespace.
3. **Easy parsing.** `LBRACE` and `RBRACE` are single-character tokens. No indentation tracking required.
4. **Error recovery.** When the parser encounters an error inside a block, it can skip forward to the closing brace and resume parsing the next construct.

5.5 Line-Oriented vs Expression-Based Syntax

Line-Oriented Syntax

In a line-oriented language, each line is a self-contained statement. The newline character acts as an implicit statement terminator.

MSD attributes are line-oriented:

```
*student_id: INT
name: VARCHAR(100)
date_of_birth: DATE
```

Each attribute occupies exactly one line. There is no mechanism for an attribute to span multiple lines, and no need for one — attributes are simple enough that a single line always suffices.

Other line-oriented DSLs include:

```
# Makefile
build: main.o utils.o
    gcc -o program main.o utils.o
```

```
# .ini file
[database]
host = localhost
port = 5432
```

Advantages: Simple mental model. Users can reason about each line independently. Parsing is straightforward — the lexer emits NEWLINE tokens that act as statement boundaries.

Disadvantages: Long declarations must be broken awkwardly. Not suitable for deeply nested or compositional structures.

Expression-Based Syntax

In expression-based languages, constructs can nest arbitrarily and span multiple lines:

```
;; Lisp
(define (factorial n)
  (if (<= n 1)
      1
      (* n (factorial (- n 1)))))
```

```
-- Haskell
factorial n
  | n <= 1    = 1
  | otherwise = n * factorial (n - 1)
```

Advantages: Highly compositional. Complex structures emerge naturally from nesting simple ones.

Disadvantages: Can become difficult to read without careful formatting. Parsing is more involved.

MSD's Approach

MSD is primarily line-oriented within blocks, but flexible between constructs:

```
entity Student {
    *student_id: INT
    name: VARCHAR(100)
}

link Student (0,N) enrolled_in
link Student (1,1) belongs_to
```

Attributes within a block are one-per-line. Link statements are one-per-line. But the blank lines between top-level constructs are optional — the parser skips newlines between constructs freely. This gives users the freedom to organise their files visually without the parser being rigid about layout.

Info

MSD's parser uses `_skip_newlines()` liberally between constructs. Inside blocks, NEWLINE tokens separate attributes. Between blocks, they are ignored. This two-level treatment — significant newlines inside blocks, insignificant newlines between blocks — gives users formatting freedom while keeping the grammar simple.

5.6 Nesting, Grouping, and Delimiters

Nesting Depth

Every language has a maximum practical nesting depth. In general-purpose languages, nesting can be arbitrarily deep (functions within classes within modules

within packages). In DSLs, nesting is typically shallow by design.

MSD has exactly two levels of nesting:

```
file
  └── entity/association/project block
        └── attribute
```

A file contains blocks. Blocks contain attributes. Attributes do not contain further blocks. This flat structure is intentional — a conceptual data model does not require deep nesting, and limiting depth keeps the grammar and parser simple.

Compare with a more deeply nested DSL like Terraform:

```
resource "aws_instance" "web" {
    provisioner "remote-exec" {
        inline = [
            "echo 'Hello'"
        ]
    }
}
```

Here we have file, resource block, provisioner block, and list literal — four levels. The additional depth is justified by the domain's complexity, but it comes at a cost in grammar and parser complexity.

Guideline: Choose the minimum nesting depth that your domain requires. Every additional level adds grammar rules, parser complexity, and cognitive load for users.

Grouping with Parentheses

Parentheses serve two distinct purposes in MSD:

1. **Cardinalities in links:** `(0,N)`, `(1,1)`
2. **Type sizes:** `VARCHAR(255)`, `DECIMAL(10)`

These two uses are unambiguous because they appear in different syntactic contexts. In a link statement, parentheses follow an entity name; in a type expression, they follow a type name. The parser always knows which interpretation applies.

Delimiter Choices

When a block contains multiple items, how are they separated?

Strategy	Example	Used By
Commas	`[1, 2, 3]`	JSON, Python lists
Semicolons	`int x; int y;`	C, Java, CSS
Newlines	`name: TEXT` (next line) `age: INT`	MSD, YAML, .ini

Strategy	Example	Used By
Nothing (juxtaposition)	`(+ 1 2)`	Lisp, Haskell

MSD uses **newlines** as delimiters between attributes. No commas or semicolons are required:

```
entity Student {
    *student_id: INT
    name: VARCHAR(100)
    email: VARCHAR(255)
}
```

This choice reduces visual noise. In a DSL where each item fits on one line, newlines are the natural separator — they are already there for readability. Adding commas or semicolons would be redundant punctuation that serves no purpose beyond satisfying the parser, and a well-designed parser should not require them.

5.7 Ambiguity and How to Avoid It

What Syntactic Ambiguity Means

A grammar is **ambiguous** if there exists at least one input string that can be parsed in two or more structurally different ways. Ambiguity is a property of the grammar, not the input. An ambiguous grammar is problematic because the parser cannot determine which interpretation the user intended.

The Classic Example: Dangling Else

Consider this grammar fragment:

```
stmt = "if" expr "then" stmt
     | "if" expr "then" stmt "else" stmt
     | other_stmt ;
```

Given the input `if a then if b then s1 else s2`, there are two valid parse trees:

1. `if a then (if b then s1 else s2)` — the `else` belongs to the inner `if`
2. `if a then (if b then s1) else s2` — the `else` belongs to the outer `if`

Both are valid according to the grammar. Most languages resolve this by convention (the `else` binds to the nearest `if`), but the grammar itself is ambiguous.

How Ambiguity Arises in DSLs

Common sources of ambiguity in DSL design:

- **Overloaded keywords.** If the same keyword introduces two different constructs, the parser may not be able to distinguish them.
- **Optional clauses.** When multiple optional elements can be omitted, the parser may not know which ones are present.
- **Shared prefixes.** Two constructs that begin identically force the parser to look ahead further to disambiguate.

Strategies for Avoiding Ambiguity

1. **Unique keywords for each construct.** MSD uses `entity`, `association`, `link`, and `project` — four distinct keywords for four distinct constructs. No ambiguity is possible at the top level.
2. **Explicit delimiters.** Curly braces mark block boundaries unambiguously. There is no question about where a block ends.
3. **Fixed structure within constructs.** MSD link statements follow a rigid pattern — `link EntityName (min,max) AssociationName` — with no optional parts that could shift the meaning of subsequent tokens.
4. **Context-free alternatives.** Where alternatives exist (e.g., an attribute may or may not be a primary key), the distinguishing token (*) appears first, before any shared structure.

MSD's Ambiguity-Free Design

The complete MSD grammar has no ambiguity. Every valid MSD file has exactly one parse tree. This is achieved through the combination of unique top-level keywords, brace-delimited blocks, and fixed-structure statements:

```
program     = { top_level } ;
top_level   = project_block | entity_block | assoc_block | link_stmt ;

project_block = "project" "{" { project_prop } "}" ;
project_prop  = IDENTIFIER ":" STRING_VALUE ;

entity_block  = "entity" IDENTIFIER "{" { attribute } "}" ;
assoc_block   = "association" IDENTIFIER "{" { attribute } "}" ;

attribute     = [ "*" ] IDENTIFIER ":" type_expr ;
type_expr     = TYPE_NAME [ "(" INTEGER ")" ] ;

link_stmt     = "link" IDENTIFIER "(" card_min "," card_max ")" IDENTIFIER ;
card_min      = "0" | "1" ;
card_max      = "1" | "N" ;
```

Each rule begins with a unique token. Each block has explicit delimiters. Each statement has a fixed structure. Ambiguity simply has no room to enter.

5.8 Designing for Readability vs Parseability

The Tension

The most readable syntax is not always the easiest to parse, and the easiest syntax to parse is not always the most readable. Language design involves navigating this tension.

A perfectly parseable syntax might look like:

```
ENTITY Student LBRACE ATTR PK student_id COLON INT ATTR name COLON VARCHAR LPAREN 100 RPAREN
```

Every token type is explicit. Parsing is trivial. But no human would want to write this.

A perfectly readable syntax might look like:

```
Student has attributes:
    student_id (primary key, integer)
    name (text, up to 100 characters)
```

This reads like natural English. But parsing natural language is one of the hardest problems in computer science.

MSD navigates between these extremes:

```
entity Student {
    *student_id: INT
    name: VARCHAR(100)
}
```

It borrows familiar conventions (curly braces from C, `key: value` from YAML, `*` for special marking) while maintaining a structure that is straightforward to parse.

Case Study: MSD Cardinalities

MSD's link syntax illustrates the readability-parseability tension well:

```
link Student (0,N) enrolled_in
```

This reads naturally: "link Student, with cardinality zero to many, to enrolled_in." But the cardinality `(0,N)` presents a parsing subtlety: the minimum is always an INTEGER token (`0` or `1`), whilst the maximum can be either an INTEGER (`1`) or an IDENTIFIER (`N`). The parser must accept both token types in the maximum position:

```
max_tok = self._expect(TokenType.INTEGER, TokenType.IDENTIFIER)
```

A more parseable design might have been `(0,*)` (using a symbol instead of a letter) or `0..N` (using a range operator). But `(0,N)` was chosen because it is the standard MERISE notation that every user of the language already knows. The small parsing complication is worth the domain familiarity.

The 80/20 Rule

Optimise your syntax for the 80% case — the constructs that users will write most often. Make those constructs as clean and readable as possible, even if it means slightly more complex parsing. For the 20% of less common constructs, ensure they are *possible* but accept that they may be slightly more verbose.

In MSD, entities and attributes are the 80% case. They are clean and minimal. Link statements are slightly more complex (the cardinality syntax), but links are written less frequently than attributes — you write one link per relationship, but multiple attributes per entity.

Trade-offs in MSD

Design Decision	Readability	Parseability
`entity Name { ... }`	Familiar, clear block structure	Simple: keyword, name, braces
`*attr: TYPE`	Concise primary key marker	Simple: optional star prefix
`link E (0,N) A`	Reads like MERISE notation	Mixed token types in cardinality
Newlines as separators	No visual noise from commas	Must handle flexible whitespace
No semicolons	Cleaner appearance	Newlines carry structural weight

Warning

Do not sacrifice readability for parsing convenience. You write the parser once; users write in the language every day. A few extra lines in the parser are always preferable to a syntax that frustrates users.

5.9 Exercises

Exercise 5.1 — Write an EBNF grammar for a DSL that describes REST API endpoints. Support path parameters (`/users/{id}`), HTTP methods (`GET`, `POST`, `PUT`, `DELETE`), and request/response type annotations. For example:

```
endpoint GET /users/{id} {
    response: User
}
```

```
endpoint POST /users {
    request: CreateUserInput
    response: User
}
```

Your grammar should handle the path segment syntax, including parameters with optional type constraints (e.g., `{id:int}`).

Exercise 5.2 — The MSD grammar uses `{ }` for blocks. Redesign the syntax using Python-style indentation. Write the EBNF grammar for your indentation-based version. What additional token types must the lexer produce? What parsing challenges does this create that the brace-based grammar avoids?

Exercise 5.3 — Is the following grammar LL(1)? If not, identify the problem and transform it into an equivalent LL(1) grammar.

```
stmt = "if" expr "then" stmt
     | "if" expr "then" stmt "else" stmt
     | IDENTIFIER "=" expr ;
```

Show the FIRST sets for each alternative of the original grammar to demonstrate the conflict, then write the transformed grammar.

Exercise 5.4 — Consider a DSL for defining state machines:

```
state Idle {
    on Click -> Active
    on Timeout -> Idle
}

state Active {
    on Release -> Idle
    on Error -> Failed
}
```

Write the complete EBNF grammar. Verify that it is LL(1) by examining the FIRST sets of each alternative at every decision point.

Exercise 5.5 — MSD uses newlines to separate attributes within blocks. Redesign the attribute syntax to use commas instead:

```
entity Student {
    *student_id: INT, name: VARCHAR(100), email: VARCHAR(255)
}
```

Write the EBNF for this variant. What are the trade-offs? Does this change affect the LL(1) property? Consider the impact on error recovery when a user makes a syntax

error in the middle of a comma-separated list versus a newline-separated list.

Exercise 5.6 — Remove the left recursion from the following grammar and rewrite it using EBNF repetition:

```
expr   = expr "+" term | expr "-" term | term ;
term   = term "*" factor | term "/" factor | factor ;
factor = INTEGER | "(" expr ")" ;
```

Verify that your transformed grammar is LL(1).

Exercise 5.7 — Design the syntax for a DSL that describes database migration steps. It should support creating tables, adding columns, dropping columns, and renaming columns. For each operation, the user should specify the table name, column details, and data types. Write the EBNF grammar and explain three design decisions you made, justifying each in terms of readability and parseability.

Summary

Syntactic design is where a language takes shape. The choices you make — block delimiters, statement structure, nesting depth, delimiter style — determine not only how easy the language is to parse, but how users think about and interact with it.

Key principles from this chapter:

- **Formalise your grammar in EBNF before writing a parser.** The grammar is the specification; the parser is the implementation.
- **Aim for LL(1).** A grammar where one token of lookahead suffices is a grammar that yields a clean, simple, maintainable recursive descent parser.
- **Choose block structure deliberately.** Curly braces, indentation, and keywords each have trade-offs. Pick the style that best serves your domain and your users.
- **Prefer shallow nesting.** Every level of nesting adds complexity. Use the minimum depth your domain requires.
- **Eliminate ambiguity by design.** Unique keywords, explicit delimiters, and fixed-structure statements prevent ambiguity from arising in the first place.
- **Optimise for readability.** The parser is written once; the language is used daily. When readability and parseability conflict, favour the user.

In the next chapter, we move beyond syntax to *semantics* — the meaning of well-formed programs. A syntactically valid MSD file can still contain errors: references to entities that do not exist, duplicate attribute names, or cardinalities that make no sense. Semantic analysis catches what syntax cannot.

Chapter 6: Semantic Design

Chapters 4 and 5 addressed how a DSL's input is broken into tokens and assembled into structure. A lexer can tell you that `VARCHAR` is a valid identifier. A parser can tell you that `name: VARCHAR(100)` is a well-formed attribute declaration. But neither can tell you whether `VARCHAR` is a legitimate data type, whether `100` is a sensible size for it, or whether the entity containing this attribute has already been declared under the same name. These questions belong to the **semantic** layer — the layer that assigns *meaning* to syntactically valid programs.

This chapter examines the design decisions that shape a DSL's semantics: type systems, name binding, reference resolution, uniqueness rules, validation constraints, and error message quality. We continue to use MSD as our primary case study, drawing comparisons with SQL, Terraform, and other DSLs where they illuminate a general principle.

6.1 Beyond Syntax: What Makes a Program Meaningful

A grammar defines what *shapes* are legal. Semantics defines what those shapes *mean*. The distinction is easy to state in the abstract, but its practical consequences are profound. Consider this MSD fragment:

```
entity Student {
    *student_id: INT
    name: VARCHAR(100)
}

link Student (0,N) Studnt
```

The parser will accept this without complaint. The `link` statement has the correct structure: an identifier, a cardinality in parentheses, and another identifier. Syntactically, it is flawless. Semantically, it is broken — there is no association called `Studnt`. The user almost certainly meant to write the name of an association that exists elsewhere in the file, but a parser that works purely on structure has no way to know this.

This is the fundamental job of the semantic layer: to bridge the gap between *what the user wrote* and *what the tool should do*. The parser produces an intermediate representation — a faithful record of the input's structure. The semantic analyser examines that structure and asks: does it make sense?

Semantic analysis typically involves several concerns:

1. **Type checking** — are values of the right kind?
2. **Name resolution** — do referenced names actually exist?
3. **Uniqueness enforcement** — are declarations distinct where they must be?

4. **Constraint validation** — are domain-specific rules satisfied?
5. **Error reporting** — when something is wrong, can we explain it clearly?

In a general-purpose language like Java, the semantic layer is vast: type inference, method overload resolution, generic type bounds, access control, definite assignment analysis. In a DSL, the semantic layer is smaller but no less important. A DSL that catches errors early and explains them well is a pleasure to use; one that silently produces wrong output is dangerous.

6.2 Type Systems for DSLs

Why DSLs Need Types

Even the simplest DSLs have types, though they may not call them that. A Dockerfile's `EXPOSE` instruction expects a port number, not an arbitrary string. A Makefile's target name is a filename, not a shell command. Whenever a language restricts what values are valid in a given context, it has a type system — even if it is implicit.

Making types explicit in a DSL serves three purposes:

- **Validation**: the tool can reject invalid input before it causes downstream failures.
- **Documentation**: the type annotations tell the user what is expected, even without reading a manual.
- **Code generation**: the type information drives output. MSD translates `INT` into `INTEGER` in PostgreSQL DDL; it cannot do this without knowing the type.

Simple Types

The most basic type system is a fixed enumeration of type names. MSD defines 13 data types:

```
INT, BIGINT, SMALLINT
VARCHAR, CHAR, TEXT
BOOLEAN
DATE, TIME, TIMESTAMP
DECIMAL, FLOAT, DOUBLE
```

This is a **closed set** — the user cannot define new types. Every attribute in an MSD file must use one of these names (case-insensitively). If the user writes `BLOB`, the parser rejects it immediately and lists the valid alternatives. This design is deliberate: MSD targets PostgreSQL, and supporting only types that map cleanly to PostgreSQL DDL avoids ambiguity in code generation.

Compare this with Terraform, which has a richer but still finite type system: `string`, `number`, `bool`, `list(type)`, `map(type)`, `set(type)`, and `object({...})`. Terraform's types are composable — you can nest them — but the base set is still closed.

Enumerated Types

Some DSL values are not data types in the traditional sense but are drawn from a fixed set nonetheless. MSD cardinalities are an example. A cardinality minimum must be `0` or `1`; a cardinality maximum must be `1` or `N`. These are enumerated values, and MSD validates them during parsing:

```
link Student (3,N) Enrol
```

This is rejected with: `invalid minimum cardinality: '3' (expected 0 or 1)`. The parser does not need a type system in the academic sense to enforce this — a simple membership check suffices — but the principle is the same: values are constrained to a known set.

Parameterised Types

Some types accept arguments that refine their meaning. In MSD, `VARCHAR` requires a size parameter, `CHAR` accepts one, and `DECIMAL` accepts a precision:

```
entity Product {
    *product_id: INT
    name: VARCHAR(255)
    sku: CHAR(12)
    price: DECIMAL(10)
}
```

MSD distinguishes between types that accept sizes (`VARCHAR`, `CHAR`, `DECIMAL`) and those that do not. Writing `INT(10)` triggers a warning: `data type 'INT' does not accept a size parameter`. This is a *parameterised type system* — not in the sense of Java generics, but in the sense that certain type constructors take arguments.

SQL follows the same pattern: `VARCHAR(n)`, `NUMERIC(p,s)`, `CHAR(n)`. The parallel is intentional — MSD's type system mirrors the subset of SQL that its users already know.

When to Validate Types

A key design decision is *when* type validation occurs. There are two common strategies:

- **Parse-time validation**: check types as they are parsed. This is MSD's approach — an unknown type like `BLOB` causes an immediate error, and parsing of that attribute aborts via panic-mode recovery.
- **Build-time validation**: accept any identifier as a type during parsing and validate it later, during semantic analysis. This is more flexible — it allows type aliases or user-defined types — but delays error reporting.

MSD chose parse-time validation because its type set is closed and small. There is no benefit to deferring the check, and rejecting early means the error message can point precisely at the offending token.

Tip

If your DSL has a fixed set of types, validate them during parsing. If users can define their own types, you must defer validation to a later pass, because the type definition may appear after its first use.

Programmer's Perspective

Go's `go/types` package is the gold standard for semantic analysis in a production language, and its architecture maps directly onto DSL semantic design. The `types.Config.Check()` function takes a parsed AST, resolves all identifiers, checks type compatibility, and returns a `types.Info` struct containing the resolved type of every expression. MSD's builder follows the same pattern at a smaller scale: it takes parsed data structures, resolves name references, checks uniqueness constraints, and produces a validated model. If you are building a DSL in Go and need to enforce semantic constraints, studying `go/types` will show you how to implement symbol tables, handle forward references via multi-pass resolution, and produce error messages that include source positions and suggestions.

6.3 Name Binding and Scoping Rules

Names Are the Glue

Every non-trivial language uses names to refer to things. In MSD, entities have names, associations have names, attributes have names, and links refer to entities and associations by name. The rules governing how names are declared, how they are looked up, and where they are visible constitute the language's **scoping rules**.

Flat Scope

The simplest scoping model is **flat scope**: all names live in a single global namespace, and every name is visible everywhere. This is MSD's model. There is no nesting of entity definitions inside other entities, no modules, no imports. If you declare an entity called `Student`, that name is available throughout the entire file.

SQL tables follow the same model within a schema — all table names are globally visible, and any table can reference any other table in a foreign key constraint.

Flat scope has the virtue of simplicity. The user never needs to think about which names are "in scope" — everything is. The cost is that name collisions are more likely, because there is only one namespace to hold everything.

Block Scope and Lexical Scope

For contrast, consider how other languages handle scope:

- **Block scope** (C, Java): names declared inside a `{ ... }` block are local to that block. A variable declared inside a `for` loop is invisible outside it.
- **Lexical scope** (Python, JavaScript): nested blocks can see names from their enclosing blocks, but not vice versa. An inner function can read variables from the outer function that defined it.

These models are more powerful but more complex. A DSL designer should adopt them only when the domain genuinely requires nested definitions. A configuration language for infrastructure (like Terraform's `resource` blocks) benefits from block scope because resources are naturally grouped. A data modelling language like MSD, where everything is inherently global, does not.

MSD's Scoping Model

MSD uses three separate flat namespaces:

1. **Entity names** — each entity must have a unique name.
2. **Association names** — each association must have a unique name, and must not collide with any entity name.
3. **Attribute names** — attributes are scoped to their enclosing entity or association. Two different entities may each have an attribute called `name` without conflict, but a single entity may not have two attributes called `name`.

This is a pragmatic middle ground. The global namespaces for entities and associations ensure that link references are unambiguous. The entity-local namespace for attributes avoids needless uniqueness constraints on common attribute names like `id`, `name`, or `date`.

Note

MSD's cross-namespace uniqueness rule (associations cannot share names with entities) is a deliberate design choice, not a technical necessity. It ensures that every identifier in a `link` statement can be resolved unambiguously without needing the parser to know whether the target is an entity or an association. We explore the trade-offs of this decision in Section 6.5.

6.4 Reference Resolution and Forward References

What Is a Reference?

A **reference** is the use of a name that was defined elsewhere. In MSD, the `link` statement contains two references — one to an entity and one to an association:

```
link Student (0,N) Enrol
```

Here, `Student` refers to an entity defined in an `entity Student { ... }` block, and `Enrol` refers to an association defined in an `association Enrol { ... }` block. The

semantic analyser must *resolve* these names — that is, it must find the actual entity and association objects that the names refer to.

The Forward Reference Problem

Consider this MSD file:

```
link Student (0,N) Enrol

entity Student {
    *student_id: INT
    name: VARCHAR(100)
}

association Enrol {
    enrolment_date: DATE
}
```

The `link` statement appears *before* the `entity` and `association` declarations it references. This is a **forward reference** — a use of a name before its definition. Should the language allow this?

There are two common strategies:

- **Single-pass, reject forward references**: process the file top to bottom, and if a name is used before it is declared, report an error. This is simple to implement but forces the user to arrange declarations in dependency order, which can be tedious.
- **Multi-pass, collect then resolve**: first pass collects all declarations; second pass resolves all references. Forward references work naturally because all names are known before resolution begins.

MSD uses the multi-pass approach. The parser's job is to collect all declarations — entities, associations, links — into a `ParseResult` intermediate representation. The builder then processes this intermediate representation, first registering all entity and association names, then resolving link references against the complete set of known names. This is why the parser produces `ParsedEntity`, `ParsedAssociation`, and `ParsedLink` data classes rather than final model objects — the intermediate representation exists precisely to enable multi-pass processing.

The benefit is clear: the user can write MSD files in whatever order feels natural. The `link` statements can appear before, after, or interleaved with the entity and association declarations they reference.

Info

The two-phase design (parse, then build) is a common pattern in language implementation. Compilers call it "declaration before use" vs "use before declaration." Languages like C require declaration before use (hence forward declarations for functions). Languages like Java and Python allow use before declaration within the same file. For DSLs, allowing forward references is almost always the friendlier choice — it avoids forcing users to think about declaration order.

Resolution in Practice

MSD's builder resolves references with a straightforward dictionary lookup. After building all entities and associations, it maintains two dictionaries: `entity_names` mapping names to `Entity` objects, and `assoc_names` mapping names to `Association` objects. For each parsed link, it looks up the entity name in `entity_names` and the association name in `assoc_names`. If either lookup fails, it reports an error — and, as we shall see in Section 6.7, suggests a correction.

In SQL, reference resolution follows a similar pattern. A `FOREIGN KEY` clause references a table by name, and the database engine must verify that the referenced table exists. The difference is that SQL operates on a persistent catalogue of known tables, whereas MSD resolves everything within a single file.

6.5 Uniqueness and Conflict Rules

Duplicate Declarations

What should happen when two declarations share the same name? In MSD, the answer is straightforward: it is an error.

```
entity Student {
    *student_id: INT
    name: VARCHAR(100)
}

entity Student {
    *id: INT
    email: VARCHAR(255)
}
```

The builder rejects this with `duplicate entity name: 'Student'`. The second declaration is skipped, and any links referencing `Student` will resolve to the first declaration. This is a **first-wins** strategy: the first declaration of a name "owns" it.

Alternative strategies exist:

- **Last-wins**: the second declaration silently replaces the first. CSS follows this model — later rules override earlier ones. This is useful for languages where overriding is a feature, but it can mask errors.

- **Merge**: the two declarations are combined. Protocol Buffers allows extending message types across files. This is powerful but complex.
- **Namespace**: allow duplicate names in different namespaces. Java allows a class and a package to share a name because they occupy different namespaces.

For MSD, strict uniqueness with an error on duplicates is the right choice. A data model with two entities named `Student` is almost certainly a mistake, not an intentional override.

Cross-Namespace Conflicts

MSD goes further: an association may not have the same name as an entity.

```
entity Enrol {
    *id: INT
}

association Enrol {
    date: DATE
}
```

This produces: `association name 'Enrol' conflicts with an entity of the same name`. The rule exists to ensure that every name appearing in a `link` statement can be resolved unambiguously. Without this rule, the name `Enrol` in a link could refer to either the entity or the association, and the tool would need additional context (or syntax) to distinguish them.

This is a design trade-off. Terraform, for instance, allows a `resource` and a `data` source to share the same logical name because they are distinguished by their block type (`resource "aws_instance" "web"` vs `data "aws_instance" "web"`). MSD could have adopted a similar scheme — perhaps `link Student (0,N) association:Enrol` — but the added syntax complexity was not justified for the domain.

Warning

Cross-namespace conflicts are easy to overlook during design. If your DSL has multiple kinds of named things (types, variables, functions, modules), decide early whether names must be globally unique or only unique within their kind. Ambiguous resolution rules will confuse users and complicate your implementation.

Programmer's Perspective

DSL semantic checks are essentially business logic validation moved into the language layer, and this shift catches entire categories of bugs at authoring time rather than at runtime. Consider how Terraform's `terraform validate` command catches invalid resource references, type mismatches, and missing required attributes before any infrastructure is provisioned. In Go, the compiler's semantic analysis rejects programs where a variable is declared but never used, where an interface is not satisfied, or where a type assertion is provably impossible. Your DSL's semantic layer should aim for the same goal: every constraint that can be checked statically should be, because an error caught by the DSL tool costs the user seconds to fix, whilst the same error discovered in production can cost hours or worse.

6.6 Validation Rules and Constraint Checking

Beyond the Grammar

Many rules that a DSL must enforce cannot be expressed in a context-free grammar. Grammars describe structure; they cannot count, compare, or look things up in a symbol table. The rules that remain after parsing are **semantic constraints**, and they form the heart of the semantic layer.

MSD's Semantic Validations

MSD enforces the following semantic rules, each checked in the builder:

Rule	Severity	Stage
Duplicate entity name	Error	Build
Duplicate association name	Error	Build
Entity/association name conflict	Error	Build
Unknown entity in link	Error	Build
Unknown association in link	Error	Build
Entity without primary key	Warning	Build
Unknown data type	Error	Parse
Invalid cardinality value	Error	Parse
Size on non-sized type	Warning	Parse

Notice that some validations occur during parsing (types, cardinalities) while others occur during building (names, references). The division follows a simple principle: validate *locally* during parsing (a type is valid or not, independent of anything else

in the file) and validate *relationally* during building (a reference is valid only if the referenced name exists).

Hard Constraints vs Soft Constraints

MSD distinguishes between **errors** and **warnings**:

- **Errors** indicate that the input is definitely wrong and the tool cannot produce correct output. A link referencing a non-existent entity is an error — the tool cannot generate SQL for a foreign key to a table that does not exist.
- **Warnings** indicate that the input is probably wrong but the tool can still proceed. An entity without a primary key is a warning — the tool can still generate a CREATE TABLE statement (though it will lack a PRIMARY KEY clause), and some modelling scenarios genuinely omit primary keys during early design.

The classification matters for tooling. MSD's CLI uses exit code 1 for errors and exit code 0 for warnings, allowing CI pipelines to distinguish between fatal problems and advisory notices. The GUI displays errors in red and warnings in amber.

The Error Budget

How many validations should a DSL enforce? There is no universal answer, but a useful heuristic is the **error budget**: each validation you add has a cost (implementation complexity, maintenance burden, potential for false positives) and a benefit (catching user mistakes). Add validations where the benefit clearly outweighs the cost.

MSD's builder enforces nine validations — a modest number, but each one catches a common and consequential mistake. A more complex DSL might enforce dozens. Terraform, for example, validates resource types against provider schemas, checks variable type constraints, detects dependency cycles, and warns about deprecated features — easily hundreds of distinct checks.

The key is to prioritise. Start with the validations that catch the most common mistakes and the most dangerous ones (those that produce silently wrong output). Add more as users report confusion or bugs.

6.7 Error Messages as a Design Concern

Error Messages Are User Interface

If your DSL's error messages are an afterthought, your DSL's user experience is an afterthought. For many users, error messages are the primary way they learn the language. A clear error message that explains what went wrong and how to fix it is worth more than a page of documentation.

Principles of Good Error Messages

A well-designed error message is:

1. **Specific**: it identifies exactly what is wrong. Not "invalid input" but "unknown data type: 'BLOB'."
2. **Located**: it points to where the problem is. MSD errors include the filename, line number, and column number — enough for any editor to jump to the exact location.
3. **Actionable**: it tells the user what to do. Listing valid types after rejecting an unknown one gives the user everything they need to fix the problem without consulting a reference manual.
4. **Contextual**: it shows what was expected alongside what was found. "Expected IDENTIFIER, got RBRACE ('}')" tells the user that a closing brace appeared where a name was expected — suggesting a missing attribute or an extra brace.

Anti-Patterns

Common error message anti-patterns include:

- **"Syntax error"** — tells the user nothing about what is actually wrong.
- **"Error at line 7"** — locates the problem but does not describe it.
- **"E0042"** — error codes without explanations require the user to look up the code in a separate document.
- **"Invalid token"** — too vague. Which token? Why is it invalid? What should it be?

MSD's Error Message Design

MSD's error messages follow a consistent pattern: `<location>: <severity>: <description>`. For example:

```
schema.msd:12: error: unknown data type: 'BLOB' (valid types: BIGINT, BOOLEAN,
    CHAR, DATE, DECIMAL, DOUBLE, FLOAT, INT, SMALLINT, TEXT, TIME, TIMESTAMP,
    VARCHAR)
```

This message is specific (it names the offending type), located (file and line), and actionable (it lists every valid alternative). The user can fix the problem without leaving their editor.

For reference resolution failures, MSD goes a step further:

```
schema.msd:24: error: unknown entity: 'Tourits' (did you mean 'Tourist'?)
```

The suggestion is computed using **Levenshtein distance** — the minimum number of single-character edits (insertions, deletions, substitutions) needed to transform one string into another. MSD's implementation compares the unknown name against all known entity names (case-insensitively) and suggests the closest match, provided the distance is at most 3.

```python
def _suggest(name: str, candidates: list, max_distance: int = 3) -> Optional[str]:
    """Find the closest match by Levenshtein distance."""
```

```
    best = None
    best_dist = max_distance + 1
    for c in candidates:
        d = _levenshtein(name.lower(), c.lower())
        if d < best_dist:
            best_dist = d
            best = c
    return best if best_dist <= max_distance else None
```

The threshold of 3 is a pragmatic choice. A distance of 1 catches single-character typos (the most common case). A distance of 2 catches transpositions and double typos. A distance of 3 extends to slightly more garbled names. Beyond 3, suggestions become unreliable — the "closest" match may bear little resemblance to the intended name, and a misleading suggestion is worse than no suggestion at all.

When to Stop Processing

A final error message design question: when an error is detected, should the tool stop immediately or continue and report additional errors?

MSD takes a **continue-and-collect** approach. The parser uses panic-mode recovery to skip past errors and resume parsing at the next well-defined boundary (a top-level keyword or a closing brace). The builder processes all declarations even when some fail. The result is that a single invocation can report multiple errors:

```
schema.msd:5: error: duplicate entity name: 'Student'
schema.msd:12: error: unknown entity: 'Tourits' (did you mean 'Tourist'?)
schema.msd:15: warning: entity 'Module' has no primary key
```

This is far more useful than stopping at the first error. A user fixing a file with three problems would otherwise need three separate edit-run cycles to discover them all. Multi-error reporting respects the user's time.

The trade-off is complexity. After an error, the tool's state may be partially invalid, and subsequent errors may be *cascading* — caused not by real problems in the input but by the incomplete state left by the first error. MSD mitigates this by skipping the offending declaration entirely (using `continue` after recording a duplicate name error, for instance), so that subsequent processing does not attempt to use the broken declaration.

Tip

Design your error messages before you implement your semantic analyser. Write down the exact text you want users to see for each kind of mistake. This forces you to think about what information you need to include and ensures consistency across all your error paths.

6.8 Exercises

Exercise 6.1 — Consider the following MSD fragment:

```
entity Student {
    *student_id: INT
    name: VARCHAR(100)
    name: VARCHAR(200)
}
```

Should MSD reject duplicate attribute names within the same entity? What are the arguments for treating this as an error vs a warning? If it is an error, should the first or second declaration win?

Exercise 6.2 — MSD uses Levenshtein distance with a maximum of 3 to suggest corrections for unknown names. What are the trade-offs of this threshold? Give an example where a maximum distance of 3 produces a misleading suggestion. Give an example where a maximum distance of 1 would fail to suggest a useful correction.

Exercise 6.3 — Should MSD allow an entity and an association to have the same name? Argue *for* this change (what flexibility does it add?) and argue *against* it (what problems does it create?). If you allowed it, how would the `link` statement syntax need to change to remain unambiguous?

Exercise 6.4 — Design the semantic rules for a DSL that describes board game rules. The language allows defining **spaces** (with names and types like "property", "chance", "tax"), **cards** (with effects like "move to space X" or "pay Y"), and **rules** (like "if a player lands on an unowned property, they may buy it"). What names need to be unique? What cross-references need to be resolved? What should be an error vs a warning?

Exercise 6.5 — Terraform allows forward references: a resource can refer to another resource that is defined later in the file. Terraform resolves this by building a dependency graph and processing resources in topological order. Compare this approach to MSD's two-pass strategy (parse, then build). What are the advantages and disadvantages of each? In what situations would topological ordering be necessary but two-pass resolution would not suffice?

Exercise 6.6 — Consider adding an `import` statement to MSD that allows one `.merisio` file to reference entities defined in another file:

```
import "shared_types.msd"

link Student (0,N) Enrol
```

What new semantic rules would this feature require? How would it affect name resolution, duplicate detection, and error reporting? What scoping model would you choose — all imported names are global, or imported names must be qualified (e.g., `shared_types.Student`)?

Exercise 6.7 — Rewrite the following error messages to follow the principles described in Section 6.7. For each, explain what information was missing and what you

added.

1. `Error: bad type`
2. `Line 14: syntax error`
3. `E0017: validation failed`
4. `Unknown reference on line 9`

Chapter 7: Designing MSD

The previous four chapters gave us a systematic framework for designing domain-specific languages: domain analysis to identify what concepts the language must express, lexical design to choose tokens, syntactic design to arrange tokens into structures, and semantic design to give those structures meaning. Each chapter explored one layer in isolation.

Now we bring all four layers together. This chapter applies the entire design process to a single, real problem: creating **MSD** (Merisio Schema Definition), a text-based language for MERISE conceptual data models. By the end, we will have a complete language specification — every keyword chosen, every grammar rule written, every semantic constraint defined — ready to be implemented in Part III.

7.1 Applying the Design Process

Chapters 3 through 6 established a four-stage design process:

1. **Domain analysis** (Chapter 3): Identify the concepts, relationships, and constraints in the target domain. Produce a domain vocabulary table mapping concepts to language constructs.
2. **Lexical design** (Chapter 4): Choose keywords, identifier rules, comment syntax, and symbolic tokens. Decide on case sensitivity. Handle context-sensitive scanning where necessary.
3. **Syntactic design** (Chapter 5): Arrange tokens into grammatical structures. Write the grammar in EBNF. Ensure the grammar is unambiguous and parseable by a recursive descent parser.
4. **Semantic design** (Chapter 6): Define what the syntactic structures mean. Establish type systems, naming rules, reference resolution, and validation constraints. Design error messages.

These stages are not strictly sequential. Design decisions at one layer influence the others — a choice of keyword at the lexical layer shapes the grammar, and a semantic constraint may require syntactic support. In practice, the designer iterates between layers, refining each as the language takes shape.

For MSD, our goal is specific: design a DSL that Merisio can **parse** into an intermediate representation, **validate** against MERISE modelling rules, **lay out** as a visual diagram, and **import** into a graphical project. The language must be human-writable in a text editor, machine-parseable without ambiguity, and expressive enough to capture any valid MERISE conceptual data model.

7.2 Domain Analysis: MERISE Conceptual Data Modelling

The Domain

MERISE is a database design methodology widely used in French computer science education and industry. Its first artefact is the **MCD** (Modele Conceptuel de Donnees) — a conceptual data model that describes the information structure of a system without reference to any particular database technology.

An MCD contains three kinds of elements:

- **Entities**: real-world objects or concepts with attributes. Each entity has a name and one or more attributes, at least one of which is a primary key. For example, a `Student` entity might have attributes `student_id` (primary key), `first_name`, `last_name`, and `date_of_birth`.
- **Associations**: named relationships between entities. An association may carry its own attributes — data that belongs to the relationship itself rather than to either entity. For example, an `enrolled_in` association between `Student` and `Course` might carry a `grade` attribute.
- **Links**: connections between entities and associations, annotated with **cardinalities**. A cardinality is a pair (min, max) that specifies how many times an entity can participate in an association. The four valid cardinalities in MERISE are (0,1), (0,N), (1,1), and (1,N).

Domain Vocabulary Table

The first concrete output of domain analysis is a mapping from domain concepts to language constructs. For MSD:

Domain Concept	Language Construct	Example
Entity	`entity` block	`entity Student { ... }`
Association	`association` block	`association enrolled_in { ... }`
Link with cardinality	`link` statement	`link Student (0,N) enrolled_in`
Attribute	Name-type pair inside a block	`first_name: VARCHAR(100)`
Primary key	* prefix on attribute	`*student_id: INT`
Data type	Type name after :	`VARCHAR`, `INT`, `DATE`
Project metadata	`project` block	`project { name: My Project }`

This table is the bridge between the domain expert's vocabulary and the language

designer's constructs. Every domain concept has exactly one corresponding language construct, and every language construct maps back to a domain concept. There is no construct without a purpose.

Domain Constraints

Beyond the vocabulary, the domain imposes constraints that the language must enforce:

- **Unique names**: No two entities may share a name. No two associations may share a name. No entity and association may share a name.
- **Valid cardinalities**: The minimum must be 0 or 1. The maximum must be 1 or N. This gives exactly four valid cardinalities: (0,1), (0,N), (1,1), (1,N).
- **Valid data types**: Attributes must have types drawn from a fixed set of database-oriented types.
- **Entity completeness**: Every entity must have at least one attribute (an entity with no attributes has no reason to exist).
- **Link validity**: Every link must reference an existing entity and an existing association.

Note

The domain analysis does not prescribe *how* these constraints are enforced — only *what* they are. Whether a missing primary key is an error or a warning, for instance, is a semantic design decision made later.

Programmer's Perspective

Domain modelling for a DSL is essentially the same discipline as designing Go structs for your domain. In MSD, the domain concepts – entities, associations, links, attributes, cardinalities – map directly to data structures in the implementation. This mirrors how Go developers model domains: a `Note` struct with fields for pitch, duration, and octave; a `Rest` struct with a duration; a `Bar` struct containing a slice of events. The DSL's syntax should read like a natural serialisation of these structs. If you find yourself writing a Go struct that feels awkward to represent in your DSL syntax, or DSL syntax that does not map cleanly to a Go struct, the mismatch is a signal that your domain model needs refinement.

7.3 Lexical Choices

With the domain analysis complete, we turn to the lexical layer: what tokens will MSD use, and what rules govern them?

Keywords

MSD has exactly four keywords, one for each top-level domain concept:

- `project` — introduces a metadata block
- `entity` — introduces an entity definition
- `association` — introduces an association definition
- `link` — introduces a link statement

Four keywords is remarkably few. SQL has over a hundred. Even Dockerfile has seventeen. A small keyword count is a sign that the language is well-scoped — each keyword carries significant weight.

Case-Insensitive Keywords

MSD keywords are **case-insensitive**: `entity`, `Entity`, `ENTITY`, and `eNtItY` are all recognised as the entity keyword. This was a deliberate decision influenced by the domain.

Database professionals are the primary audience for MSD. They are accustomed to SQL, where `SELECT`, `select`, and `Select` are equivalent. Making keywords case-insensitive respects this expectation. A user who habitually writes `ENTITY` or `Entity` should not receive an error.

The alternative — case-sensitive keywords like most programming languages — was considered and rejected. Programming language conventions (lowercase keywords) conflict with SQL conventions (uppercase keywords), and choosing one would frustrate users accustomed to the other.

Case-Sensitive Identifiers

While keywords are case-insensitive, **identifiers are case-sensitive**: `Student` and `student` are different names. This follows programming language convention rather than SQL convention, and the reasoning is practical.

Entity and association names in MSD flow directly into SQL table names during code generation. If the user writes `Student`, the generated table should be `Student`, not `student` or `STUDENT`. Preserving the original casing requires case-sensitive identifiers.

Tip

When a DSL bridges two domains with conflicting conventions (SQL's case-insensitivity and programming's case-sensitivity), apply each convention where it makes the most sense: case-insensitive for the part that resembles one domain (keywords), case-sensitive for the part that resembles the other (identifiers).

The * Prefix for Primary Keys

Primary key attributes are marked with a * prefix: `*student_id: INT`. This notation was borrowed from entity-relationship diagram conventions, where primary key at-

tributes are traditionally underlined or marked with a special symbol.

The asterisk was chosen over alternatives for several reasons:

- **Brevity**: A single character, versus a keyword like `primary` or `pk`
- **Visual distinctiveness**: The * stands out at the beginning of a line, making primary keys easy to spot when scanning a file
- **Convention**: ER diagrams and many database design tools use * or a similar marker
- **Composability**: Multiple attributes can be marked as primary key for composite keys — simply prefix each with *

The alternative of a `primary key` keyword was rejected because it would require either a keyword pair (complicating the grammar) or a special attribute modifier syntax. The * prefix keeps the grammar simple: an attribute is optionally preceded by *, and that is all.

Comment Syntax

MSD supports two comment styles:

- `#` — single-line comment (everything after `#` to end of line is ignored)
- `//` — single-line comment (everything after `//` to end of line is ignored)

Supporting both styles was a deliberate choice. The `#` comment is standard in data-oriented formats (YAML, TOML, shell scripts, Python) and will feel natural to users working with configuration files. The `//` comment is standard in C-family programming languages and will feel natural to developers. Since MSD's audience includes both database designers and software developers, supporting both styles avoids forcing either group to adopt an unfamiliar convention.

Block comments (`/* ... */`) were considered and rejected. MSD files are typically short (tens to low hundreds of lines), and single-line comments suffice. Block comments introduce lexical complexity (nesting, unterminated comments) with little practical benefit.

Context-Sensitive Colon

The colon `:` serves different roles in different contexts. Inside an `entity` or `association` block, it separates an attribute name from its type:

```
first_name: VARCHAR(100)
```

Inside a `project` block, it separates a metadata key from a free-form value:

```
name: School Management System
```

In the second case, the value `School Management System` contains spaces and is not a single identifier. Rather than requiring quotation marks — which would be unfamiliar to users and add lexical complexity — MSD uses **context-sensitive lexing**: when the lexer encounters a `:` inside a `project` block, it captures the entire remainder of the

line (stripped of leading whitespace and trailing comments) as a single string value token.

This means the following works without any quoting:

```
project {
    name: My Database Project with Spaces
    author: Dr. H
    description: A model for the university registration system
}
```

The lexer tracks brace depth and a flag (`in_project_block`) to know when to apply this rule. This is a small amount of lexer state, but it eliminates the need for string literal syntax — a significant simplification for both the grammar and the user experience.

Token Summary

The complete set of token types in MSD:

Category	Tokens
Keywords	`PROJECT`, `ENTITY`, `ASSOCIATION`, `LINK`
Symbols	`{`, `}`, `(`, `)`, `:`, `,`, `*`
Literals	`IDENTIFIER`, `INTEGER`, `STRING_VALUE`
Structural	`NEWLINE`, `EOF`

Fourteen token types in total. For comparison, a typical programming language lexer handles 40–60 token types. MSD's small token vocabulary is a direct consequence of its narrow domain scope.

7.4 Syntactic Choices

Block Structure

MSD uses a block structure with curly braces to group related declarations:

```
entity Student {
    *student_id: INT
    first_name: VARCHAR(100)
}
```

Braces were chosen over indentation-based grouping (Python-style) for a practical reason: indentation sensitivity is fragile in copy-paste scenarios. A user pasting an entity definition from a web page or an email may lose its indentation, turning valid MSD into a parse error. Braces are explicit and survive formatting changes.

The block keyword (`entity`, `association`, `project`) appears before the opening brace, followed by the block name (for entities and associations) or nothing (for project).

This follows the pattern established by C, Java, Go, and CSS — a familiar structure that requires no explanation.

Attribute Syntax

Attributes use a name-colon-type pattern:

```
[*]name: TYPE[(size)]
```

This reads like a type annotation in TypeScript, Python, or Kotlin: the name comes first, then a colon, then the type. It is more natural for reading than the C-style alternative (`TYPE name`) because the eye finds the attribute name first — which is what users scan for most often.

The optional * prefix for primary keys appears before the name, making it the very first character on the line. This placement ensures that primary keys are immediately visible when scanning a list of attributes.

The optional size parameter — `VARCHAR(100)`, `DECIMAL(10)` — uses parentheses after the type name, matching SQL's syntax exactly. A database professional reading `VARCHAR(100)` will immediately understand its meaning.

Link Syntax

Links use a sentence-like syntax:

```
link Entity (min,max) Association
```

This was deliberately designed to read as a natural-language statement: "link Student (zero, many) enrolled_in" — meaning "the Student entity participates in the enrolled_in association with a cardinality of zero to many." The cardinality appears between the entity and the association, visually situated on the connection between them, much as it appears on an MCD diagram.

The alternative of placing cardinalities inside a block was considered:

```
# Rejected alternative
link {
    entity: Student
    association: enrolled_in
    cardinality: (0,N)
}
```

This is more verbose, requires four lines instead of one, and does not read as naturally. Since links carry no additional data beyond entity, association, and cardinality, a single-line syntax is both sufficient and preferred.

No Commas, No Semicolons

Within blocks, attributes are separated by newlines — no commas or semicolons are needed:

```
entity Student {
    *student_id: INT
    first_name: VARCHAR(100)
    last_name: VARCHAR(100)
}
```

This follows the precedent of Go (struct fields), TOML (key-value pairs), and HCL (block contents). Each attribute occupies exactly one line, making the newline a natural separator. Commas and semicolons would be visual noise without adding information.

The parser emits `NEWLINE` tokens to track line boundaries, but these tokens serve only as optional separators — consecutive newlines and blank lines are harmlessly skipped.

The Project Block

The `project` block uses a key-value format without quoting:

```
project {
    name: School Management System
    author: Achraf SOLTANI
    description: A school database model
}
```

The `project` block is optional. When present, it must appear at most once. Its keys are a fixed set: `name`, `author`, and `description`. Unknown keys produce a warning rather than an error, allowing forward compatibility if new metadata fields are added in future versions.

Complete Grammar in EBNF

The complete MSD grammar can be expressed in six productions:

```
program       = { project_block | entity_block | assoc_block | link_stmt } ;

project_block = "project" "{" { IDENTIFIER ":" STRING_VALUE } "}" ;

entity_block  = "entity" IDENTIFIER "{" { attribute } "}" ;

assoc_block   = "association" IDENTIFIER "{" { attribute } "}" ;

attribute     = [ "*" ] IDENTIFIER ":" type_expr ;

type_expr     = IDENTIFIER [ "(" INTEGER ")" ] ;

link_stmt     = "link" IDENTIFIER "(" INTEGER|IDENTIFIER "," INTEGER|IDENTIFIER ")" IDE
```

Six productions for an entire language. The grammar is LL(1) — at every decision point, a single token of lookahead suffices to determine which production to apply.

This makes it straightforward to implement with recursive descent parsing, which we will do in Chapter 9.

Warning

The EBNF above is a design-level specification, not a formal parser specification. It omits newline handling, comment stripping, and context-sensitive lexing — all of which are handled at the lexer level before the parser sees the token stream.

Programmer's Perspective

MSD's six-production grammar demonstrates a key principle: the DSL syntax should mirror the domain expert's vocabulary, not the programmer's implementation vocabulary. In music notation DSLs, you would write `note C4 quarter` rather than `Event{type: NOTE, pitch: 60, duration: 0.25}`, because musicians think in note names and durations, not MIDI numbers and floating-point fractions. Go's own syntax follows this philosophy – `go func() { ... }()` reads as "go run this function concurrently," mapping directly to the mental model of goroutines rather than exposing thread creation details. When designing your DSL, always ask: would a domain expert recognise this syntax without reading the implementation?

7.5 Semantic Choices

The lexical and syntactic layers define the *form* of MSD. The semantic layer defines its *meaning* — what is legal, what is illegal, and what the structures represent.

Data Types

MSD supports 13 data types, chosen to match the types in Merisio's data dictionary and to cover the most common PostgreSQL column types:

Category	Types
Integer	`INT`, `BIGINT`, `SMALLINT`
String	`VARCHAR`, `CHAR`, `TEXT`
Boolean	`BOOLEAN`
Date/Time	`DATE`, `TIME`, `TIMESTAMP`
Numeric	`DECIMAL`, `FLOAT`, `DOUBLE`

Three of these types accept a size parameter in parentheses:

- `VARCHAR(n)` — variable-length string with maximum length n
- `CHAR(n)` — fixed-length string of length n

- `DECIMAL(p)` — decimal number with precision *p*

All other types used with a size parameter produce a warning. The type name is case-insensitive during parsing (internally normalised to uppercase), so `varchar(100)`, `VARCHAR(100)`, and `Varchar(100)` are all equivalent.

Thirteen types is a carefully chosen set. It is large enough to model any realistic database schema, but small enough to memorise. Types like `SERIAL`, `UUID`, `JSON`, and `ARRAY` were deliberately excluded — they are PostgreSQL-specific extensions that do not belong in a conceptual model. MSD models the *logical* structure of data, not its physical storage.

Cardinality Constraints

Cardinalities in link statements are constrained to four valid combinations:

- `(0,1)` — zero or one (optional, singular)
- `(0,N)` — zero or more (optional, multiple)
- `(1,1)` — exactly one (mandatory, singular)
- `(1,N)` — one or more (mandatory, multiple)

The minimum must be `0` or `1`. The maximum must be `1` or `N` (where `N` is case-insensitive — both `n` and `N` are accepted). Any other value is a parse error.

This constraint set matches MERISE's definition exactly. Some ER modelling tools allow arbitrary numeric cardinalities like `(2,5)`, but MERISE does not. MSD enforces the domain's rules, not a superset of them.

Uniqueness Rules

MSD enforces three uniqueness rules:

1. **Entity names must be unique**: Defining two entities named `Student` is an error.
2. **Association names must be unique**: Defining two associations named `enrolled_in` is an error.
3. **No entity-association name conflicts**: An entity and an association cannot share a name. If entity `Course` exists, association `Course` is an error.

These rules exist because names serve as identifiers in link statements. If two entities shared the name `Student`, a link referencing `Student` would be ambiguous. The cross-category uniqueness rule (rule 3) exists for the same reason and also because entity and association names both become table names in the generated SQL.

Forward References

Link statements may reference entities and associations that are defined later in the file:

```
# Links come first — this is valid
link Student (0,N) enrolled_in
```

```
link Course (1,N) enrolled_in

entity Student {
    *student_id: INT
    name: VARCHAR(100)
}

entity Course {
    *course_id: INT
    title: VARCHAR(200)
}

association enrolled_in {
    grade: DECIMAL(10)
}
```

Forward references are supported because the parser and the semantic analyser are separate stages. The parser collects all declarations first; the semantic analyser resolves references afterward, when all names are known. This two-pass approach allows declarations to appear in any order, which is a significant usability benefit — users can organise their files by logical grouping rather than being forced into declaration-before-use ordering.

Primary Key Enforcement

An entity without a primary key attribute produces a **warning**, not an error:

```
entity Temporary {
    data: TEXT    # Warning: entity 'Temporary' has no primary key
}
```

This is a deliberate choice. A missing primary key is almost certainly unintentional, but it does not prevent the model from being valid at the conceptual level. Treating it as a warning allows the user to import an incomplete model and fix it in the graphical editor, rather than being blocked entirely. The severity distinction between errors (which may block import) and warnings (which allow it) is a usability decision informed by the workflow: text files are often used for rapid prototyping, and prototypes are intentionally incomplete.

Error Suggestions via Levenshtein Distance

When a link references an unknown entity or association name, MSD computes the Levenshtein distance between the unknown name and all known names. If a close match is found (distance of 3 or less), the error message includes a suggestion:

```
school.msd:15: error: unknown entity: 'Studnet' (did you mean 'Student'?)
```

This is a semantic design decision, not merely an implementation detail. The language specification declares that error messages *should* include typo suggestions

when possible. This commitment at the design level ensures that any conforming implementation provides this feature, rather than leaving it as an optional nicety.

Info

Levenshtein distance measures the minimum number of single-character edits (insertions, deletions, substitutions) needed to transform one string into another. A maximum distance of 3 means MSD will suggest corrections for common typos — transposed letters, missing letters, or wrong letters — without suggesting unrelated names.

7.6 The Complete MSD Specification

We now have all the pieces. Here is a complete, working MSD file that demonstrates every language construct:

```
# School Management System
# MSD file for the university registration database

project {
    name: School Management System
    author: Achraf SOLTANI
    description: A school database model
}

// Entities

entity Student {
    *student_id: INT
    first_name: VARCHAR(100)
    last_name: VARCHAR(100)
    date_of_birth: DATE
    email: VARCHAR(255)
}

entity Course {
    *course_id: INT
    title: VARCHAR(200)
    credits: INT
}

// Associations

association enrolled_in {
    grade: DECIMAL(10)
    enrollment_date: DATE
}
```

```
// Links

link Student (0,N) enrolled_in
link Course (1,N) enrolled_in
```

Let us walk through this file, line by line, observing how each design decision manifests.

Lines 1–2: Comments using the `#` style. These are stripped by the lexer and never reach the parser. They serve as documentation for the human reader.

Lines 4–8: The `project` block. The keyword `project` opens a brace-delimited block. Inside, each line is a key-value pair separated by `:`. The values — `School Management System`, `Achraf SOLTANI`, `A school database model` — contain spaces but require no quoting, thanks to context-sensitive lexing. The lexer recognises that it is inside a project block and captures the remainder of each line after `:` as a single `STRING_VALUE` token.

Line 10: A comment using the `//` style. Both comment styles work anywhere in the file.

Lines 12–18: The `Student` entity. The keyword `entity` followed by the identifier `Student` and an opening brace. Inside, five attributes are listed, one per line. The first attribute — `*student_id: INT` — is marked as a primary key by the `*` prefix. The remaining attributes use `VARCHAR(100)`, `VARCHAR(255)`, and `DATE` types. No commas separate the attributes; newlines suffice.

Lines 20–24: The `Course` entity. Same structure, different attributes. The `credits` attribute uses the unsized type `INT`.

Lines 28–31: The `enrolled_in` association. Uses the `association` keyword instead of `entity`, but the internal syntax is identical — attributes with name-colon-type notation. The `grade` attribute uses `DECIMAL(10)`, a sized type. Note that association attributes are never primary keys (associations do not have primary keys in MERISE).

Lines 35–36: Two link statements. Each reads as a sentence: "link Student (zero, many) enrolled_in" and "link Course (one, many) enrolled_in." The cardinalities `(0,N)` and `(1,N)` specify that a student may be enrolled in zero or more courses, and a course must have at least one enrolled student. The link syntax places the cardinality between the entity and the association, mirroring its position on an MCD diagram.

The entire file is 36 lines, including comments and blank lines. It fully specifies a two-entity, one-association conceptual data model with typed attributes, explicit primary keys, and cardinalities. Merisio can parse this file, validate it, generate a visual layout, and import it as a complete project — all from plain text.

7.7 Comparison with Mocodo

MSD is not the first DSL for MERISE modelling. **Mocodo** is an existing tool that uses its own text-based format to describe MCDs. Comparing MSD with Mocodo's syntax

illuminates the design trade-offs and explains why MSD made different choices.

Syntax Comparison

A simple model in Mocodo's format:

```
ENROLLED_IN, 0N STUDENT, 1N COURSE
STUDENT: student_id, first_name, last_name
COURSE: course_id, title, credits
```

The same model in MSD:

```
entity Student {
    *student_id: INT
    first_name: VARCHAR(100)
    last_name: VARCHAR(100)
}

entity Course {
    *course_id: INT
    title: VARCHAR(200)
    credits: INT
}

association enrolled_in {
    grade: DECIMAL(10)
}

link Student (0,N) enrolled_in
link Course (1,N) enrolled_in
```

Feature Comparison

Feature	Mocodo	MSD
Primary keys	Implicit (first attribute)	Explicit (* prefix)
Composite primary keys	Not supported	Multiple * attributes
Data types	None	Mandatory, 13 types
Type sizes	None	`VARCHAR(n)`, `CHAR(n)`, `DECIMAL(p)`
Syntax style	Single-line, comma-separated	Block-structured, one attribute per line
Comments	None	`#` and `//`
Project metadata	None	`project { ... }` block

Feature	Mocodo	MSD
Error messages	Minimal	Line numbers, column numbers, suggestions
Layout information	Embedded in syntax (line/column position)	Automatically computed
Carrying attributes	On the association line	In the `association` block

Why MSD Made Different Choices

Explicit primary keys over implicit ones. In Mocodo, the first attribute listed for an entity is implicitly the primary key. This is concise but fragile — reordering attributes silently changes the primary key. MSD requires the * prefix, making primary keys explicit and immune to reordering. The principle at work: **explicit is better than implicit**, especially for structural information.

Mandatory data types. Mocodo does not require data types for attributes. This keeps the syntax minimal but means that Mocodo cannot generate SQL DDL without additional information. MSD requires types because one of its primary use cases is SQL generation. Requiring types at the modelling stage catches type mismatches early and produces complete SQL output without human intervention.

Block structure over single-line syntax. Mocodo fits an entire entity on one line: `STUDENT: student_id, first_name, last_name`. This is compact for small models but becomes unwieldy for entities with many attributes. A ten-attribute entity in Mocodo is a single line exceeding 100 characters. In MSD, the same entity is ten readable lines. Block structure scales better with model complexity.

Separate link statements. In Mocodo, cardinalities are embedded in the association line: `ENROLLED_IN, 0N STUDENT, 1N COURSE`. This couples the association definition with its links. In MSD, the association and its links are independent constructs. This separation allows an association to be defined once and linked to any number of entities without repeating the association's attributes — a cleaner decomposition that mirrors the MERISE domain model itself, where associations and links are distinct concepts.

Comments and metadata. Mocodo has no comment syntax and no metadata format. MSD supports both because real-world files benefit from documentation and machine-readable metadata. Comments explain design decisions; metadata enables tooling (e.g., displaying the project name in a window title bar).

7.8 Exercises

Exercise 7.1: Domain Analysis for a New DSL

Design a DSL for network topology definitions. A topology consists of nodes (with names and types such as `router`, `switch`, `host`), connections (between pairs of nodes

with bandwidth and latency attributes), and subnets (grouping nodes under a CIDR block). Apply the domain analysis process from Section 7.2: list the domain concepts, create a domain vocabulary table mapping each concept to a language construct, and enumerate the domain constraints.

Exercise 7.2: Alternative Primary Key Syntax

MSD uses the * prefix for primary keys. Consider three alternative designs: (a) a `@pk` annotation before the attribute, (b) a separate `primary key(attr1, attr2)` statement after the attribute list, (c) a `key` keyword used as a modifier (`key student_id: INT`). For each alternative, write example syntax, assess its impact on the grammar, and evaluate its readability compared to the * prefix. Which would you choose, and why?

Exercise 7.3: Block Syntax vs Single-Line Syntax

Rewrite the complete MSD example from Section 7.6 using a hypothetical single-line syntax where each entity, association, and link occupies exactly one line (similar to Mocodo). Compare the two versions for readability at three scales: a model with 2 entities, a model with 10 entities (5 attributes each), and a model with 50 entities. At what point does block syntax become clearly superior? Is there a model size where single-line syntax is preferable?

Exercise 7.4: Adding Inheritance

MSD does not support inheritance between entities (e.g., `Person` as a parent of `Student` and `Teacher`). Design syntax and semantic rules for an inheritance feature. Consider: What keyword would you use? How would inherited attributes be represented? Would the child entity repeat the parent's primary key or inherit it implicitly? Write the EBNF productions for your design. Then evaluate: does the added complexity justify the feature, given that MERISE itself has limited support for inheritance?

Exercise 7.5: Forward References and Ordering

MSD supports forward references in link statements. Redesign MSD *without* forward references — link statements may only reference entities and associations defined earlier in the file. How does this restriction affect file organisation? Write two versions of the school management model: one ordered for the no-forward-reference version, and one ordered for maximum logical clarity. Which is more readable? What does this tell you about the value of forward references in DSL design?

Exercise 7.6: Error Message Design

Consider the following invalid MSD file:

```
entity Student {
    *student_id INT
    name: VARCAHR(100)
}

link Studnet (0,N) enrolled
```

There are three errors: a missing colon on line 2, a misspelled type on line 3, and misspelled names on line 6. For each error, write the ideal error message — including

line number, column number, severity, and a helpful suggestion. Then explain which errors can be caught at the lexical stage, which at the syntactic stage, and which at the semantic stage.

This chapter has demonstrated the complete design process for a domain-specific language. Starting from MERISE conceptual data modelling as a domain, we systematically derived MSD's lexical tokens, syntactic grammar, and semantic rules. Each decision was grounded in a principle: brevity where possible, explicitness where it matters, familiarity drawn from both SQL and programming languages, and useful error messages as a first-class design concern.

The result is a six-production grammar, four keywords, thirteen data types, and a handful of semantic constraints — a complete language specification small enough to fit in this chapter, yet expressive enough to capture any MERISE conceptual data model.

In Part III, we turn from design to implementation. Chapter 8 will build a lexer that transforms MSD source text into the token stream described in Section 7.3. Chapter 9 will build a recursive descent parser for the grammar in Section 7.4. And Chapter 10 will implement the semantic rules from Section 7.5 — including the Levenshtein-based typo suggestions. The language is designed. Now we build it.

Chapter 8: Building a Lexer

Part II gave us a complete language design for MSD: its tokens, its grammar, and its semantic rules. Now we build the machinery that turns text into meaning. The first stage in that machinery is the **lexer** — the component that reads raw characters and produces a stream of tokens.

This chapter begins with the general architecture of a lexer, introduces token representation as a concept, compares scanning strategies, and examines techniques for keyword recognition and context sensitivity. We then walk through the complete MSD lexer, tracing it character by character through a concrete example.

8.1 General Lexer Architecture

A lexer (also called a *tokeniser* or *scanner*) is the first stage in any language processing pipeline. Its job is conceptually simple: transform a flat string of characters into a structured list of tokens. Each token represents the smallest meaningful unit in the language — a keyword, an identifier, a symbol, a literal value.

The lexer's interface has two inputs and two outputs:

Inputs:

- **Source text**: the raw string to tokenise (the contents of a `.msd` file, for instance).
- **Filename**: an optional string used solely for error reporting — it never affects the tokenisation logic.

Outputs:

- **Token list**: an ordered sequence of `Token` objects representing every meaningful element in the source.
- **Error list**: a (possibly empty) list of errors encountered during scanning.

This interface establishes a clean contract. The lexer handles *characters* — whitespace, symbols, the boundaries of words and numbers. The parser, which comes next in the pipeline, handles *structure* — which tokens can follow which, how blocks nest, what constitutes a valid declaration. The lexer does not know that `entity` must be followed by an identifier and then a brace. It simply recognises `entity` as a keyword and emits the corresponding token. Structural validation belongs to the parser.

Every token carries a **source location**: the line number and column number where it was found. This information is not used by the lexer itself — it is carried forward through the pipeline so that the parser and semantic analyser can produce error

messages that point to the exact position in the source file. A lexer that discards location information cripples every downstream stage.

Note

Some lexers produce tokens lazily, yielding one at a time as the parser requests them. Others produce the entire list eagerly, up front. For small DSLs like MSD, eager tokenisation is simpler and perfectly adequate — the entire token list fits comfortably in memory. Lazy tokenisation becomes important for languages with very large source files or streaming input.

Programmer's Perspective

Go provides two ready-made approaches for building lexers. For simple DSLs, `text/scanner` offers a configurable character-class scanner that handles identifiers, integers, floats, and strings out of the box – you configure which token types to recognise and it does the character-level work. For full control, `bufio.Scanner` with a custom `SplitFunc` lets you define your own tokenisation logic whilst handling buffered I/O efficiently. Go's own lexer in `go/scanner` is a hand-written character-by-character scanner that tracks line/column positions and inserts semicolons automatically. For a DSL like MSD, a hand-written lexer of 100–200 lines of Go gives you complete control over error reporting, context sensitivity, and token position tracking – all critical for producing the kind of precise error messages that make a DSL pleasant to use.

8.2 Token Representation

A token is a small data object with four fields:

- **Type**: an enumeration value identifying the kind of token (`ENTITY`, `LBRACE`, `IDENTIFIER`, etc.).
- **Value**: the original text fragment from the source (`"entity"`, `"{"`, `"Student"`).
- **Line**: the 1-based line number where the token appears.
- **Column**: the 1-based column number where the token starts.

In Python, this is naturally expressed as a dataclass:

```python
@dataclass
class Token:
    type: TokenType
    value: str
    line: int
    column: int
```

The `type` field is the most important. It tells the parser *what kind* of token this is, without requiring the parser to inspect the raw text. The parser never needs to check whether a string equals `"entity"` — it checks whether the type equals `TokenType.ENTITY`. This separation between classification (the lexer's job) and structure recognition (the parser's job) keeps both components simple.

Token type design

Token types are best represented as an enumeration. Each member names a distinct kind of token:

```
class TokenType(Enum):
    # Keywords
    PROJECT = auto()
    ENTITY = auto()
    ASSOCIATION = auto()
    LINK = auto()

    # Symbols
    LBRACE = auto()      # {
    RBRACE = auto()      # }
    LPAREN = auto()      # (
    RPAREN = auto()      # )
    COLON = auto()       # :
    COMMA = auto()       # ,
    STAR = auto()        # *

    # Literals
    IDENTIFIER = auto()  # names: Student, id, VARCHAR, etc.
    INTEGER = auto()     # numeric literals: 100, 255, etc.
    STRING_VALUE = auto() # rest-of-line value inside project {}

    # Structural
    NEWLINE = auto()     # emitted after each source line
    EOF = auto()         # marks end of input
```

MSD has **17 token types** organised into four categories:

Category	Token Types	Count
Keywords	`PROJECT`, `ENTITY`, `ASSOCIATION`, `LINK`	4
Symbols	`LBRACE`, `RBRACE`, `LPAREN`, `RPAREN`, `COLON`, `COMMA`, `STAR`	7
Literals	`IDENTIFIER`, `INTEGER`, `STRING_VALUE`	3
Structural	`NEWLINE`, `EOF`	2

Why use an enum rather than plain strings? Three reasons. First, the compiler (or type checker) catches misspellings — `TokenType.ENTTY` is an error, but `"ENTTY"` is a

valid string. Second, enums group related values, making it easy to see the full set of token types at a glance. Third, enum comparisons are faster than string comparisons, though for a DSL this size the difference is negligible.

Tip

When designing token types for your own DSL, start by listing every distinct character sequence the lexer must recognise. Group them into categories (keywords, symbols, literals, structural). If two sequences serve different purposes, they need different token types — even if they look similar. MSD's `COLON` and `STAR` are both single-character symbols, but they serve completely different roles in the grammar.

8.3 Line-by-Line vs Character-by-Character Scanning

There are two fundamental approaches to scanning source text.

Character-by-character scanning

The traditional approach, used by most compiler textbooks, processes the entire input as a single character stream. The lexer maintains a position index into the source string and advances through it one character at a time:

```
# Character-by-character approach (simplified)
pos = 0
while pos < len(source):
    ch = source[pos]
    if ch == '\n':
        line_num += 1
        col = 0
    # ... match tokens ...
```

This approach is universal — it works for any language, regardless of whether the language is line-oriented. However, it requires explicit newline tracking, and handling line-level constructs (comments, rest-of-line captures) requires scanning forward to find the end of the line.

Line-by-line scanning

The alternative approach splits the input into lines first, then processes each line with an inner character loop:

```
# Line-by-line approach (simplified)
lines = source.split("\n")
for line_num, line_text in enumerate(lines, start=1):
    col = 0
```

```
    while col < len(line_text):
        ch = line_text[col]
        # ... match tokens ...
```

For DSLs, line-by-line scanning offers several advantages:

1. **Natural comment handling.** When the lexer encounters a comment marker (`#` or `//`), it simply `breaks` out of the inner loop. The rest of the line is skipped automatically. No need to scan forward searching for a newline character.
2. **Automatic NEWLINE emission.** After the inner loop completes for each line, the lexer emits a `NEWLINE` token unconditionally. Line numbers come for free from the outer loop's `enumerate`.
3. **Trivial rest-of-line capture.** When the lexer needs to capture everything from the current position to the end of the line (as MSD does for project metadata values), it simply slices the current line string: `line_text[col:]`. In a character-by-character scanner, this requires scanning forward to find the next `\n`.
4. **Simpler column tracking.** The column resets to zero at the start of each line automatically.

MSD uses line-by-line scanning. The outer loop iterates over source lines; the inner loop iterates over characters within each line:

```
def tokenize(self, source: str, filename: str = "") -> Tuple[List[Token], List[MSDError
    tokens: List[Token] = []
    errors: List[MSDError] = []

    lines = source.split("\n")
    brace_depth = 0
    in_project_block = False

    for line_num, line_text in enumerate(lines, start=1):
        col = 0
        length = len(line_text)

        while col < length:
            ch = line_text[col]
            # ... matching logic ...

        # Emit newline token after each source line
        tokens.append(Token(TokenType.NEWLINE, "\\n", line_num, length + 1))

    tokens.append(Token(TokenType.EOF, "", len(lines), 0))
    return tokens, errors
```

The line-by-line approach is not suitable for every language. Languages with multi-line string literals, block comments that span many lines, or significant indentation may be easier to handle with character-by-character scanning. But for line-oriented DSLs like MSD — where comments consume the rest of a line, metadata values span

a single line, and no construct crosses a line boundary — line-by-line scanning is the natural choice.

8.4 Keyword Recognition via Identifier-to-Keyword Mapping

MSD has four keywords: `project`, `entity`, `association`, and `link`. How should the lexer recognise them?

The naive approach is to check for each keyword explicitly before checking for identifiers — a chain of `if line_text[col:col+6] == "entity"` tests. This works but is fragile: it requires careful length management, can accidentally match prefixes (is `entities` a keyword followed by `s`?), and scales poorly as the keyword count grows.

The standard technique, used by virtually every hand-written lexer, is simpler:

1. **Lex everything as an identifier.** When the lexer encounters a letter or underscore, it consumes all subsequent alphanumeric characters and underscores, producing a word string.
2. **Look up the word in a keyword table.** If the word is found, emit the corresponding keyword token. If not, emit an `IDENTIFIER` token.

The keyword table is a simple dictionary:

```
KEYWORDS = {
    "project": TokenType.PROJECT,
    "entity": TokenType.ENTITY,
    "association": TokenType.ASSOCIATION,
    "link": TokenType.LINK,
}
```

The lexer performs a case-insensitive lookup using `word.lower()`:

```
# Identifiers and keywords
if ch.isalpha() or ch == "_":
    start = col
    while col < length and (line_text[col].isalnum() or line_text[col] == "_"):
        col += 1
    word = line_text[start:col]

    # Case-insensitive keyword matching
    keyword_type = KEYWORDS.get(word.lower())
    if keyword_type:
        tokens.append(Token(keyword_type, word, line_num, start + 1))
        if keyword_type == TokenType.PROJECT:
            in_project_block = True
    else:
```

```
        tokens.append(Token(TokenType.IDENTIFIER, word, line_num, start + 1))
    continue
```

This approach has three important properties:

Keywords are reserved. Because identifiers are checked against the keyword table, you cannot name an entity `entity` or an attribute `link`. The word will always be classified as a keyword. This is a deliberate design choice — allowing keywords as identifiers creates ambiguity that complicates both the lexer and the parser.

Adding a keyword requires only one line. To add a new keyword `constraint`, you add one entry to the `KEYWORDS` dictionary and one member to the `TokenType` enum. No changes to the scanning logic are needed.

Case-insensitive matching is centralised. The `word.lower()` call in the keyword lookup is the only place where case sensitivity matters. The identifier scanning loop itself is case-agnostic — it simply consumes alphanumeric characters. If you later decide to make keywords case-sensitive, you change one line (remove `.lower()`).

8.5 Handling Context Sensitivity

Most tokens in MSD are context-free: a `{` is always a `LBRACE`, an integer is always an `INTEGER`, and the word `Student` is always an `IDENTIFIER`, regardless of where they appear. But MSD has one context-sensitive case: the **colon inside a project block**.

The problem

Consider these two colons:

```
project {
    name: My Application    // colon inside project block
}

entity Student {
    *id: INT                // colon inside entity block
}
```

In the entity block, the colon separates an attribute name from its data type. After the colon, the lexer should continue scanning normally — it will find an `IDENTIFIER` token (`INT`).

In the project block, the colon separates a metadata key from its value. After the colon, *everything until the end of the line* is the value — including spaces, special characters, and anything that would normally be separate tokens. `My Application` should not become two `IDENTIFIER` tokens; it should become a single `STRING_VALUE` token with the value `"My Application"`.

The solution

The lexer tracks two pieces of state:

- **`in_project_block`**: a boolean flag, set to `True` when the `project` keyword is encountered.
- **`brace_depth`**: an integer counter, incremented on `{` and decremented on `}`.

When `brace_depth` returns to zero, `in_project_block` is cleared:

```
if ch == "}":
    tokens.append(Token(TokenType.RBRACE, "}", line_num, col + 1))
    brace_depth -= 1
    if brace_depth == 0:
        in_project_block = False
    col += 1
    continue
```

When the lexer encounters a colon, it always emits a `COLON` token. Then, if `in_project_block` is `True`, it captures the rest of the line as a `STRING_VALUE`:

```
# Colon — context-sensitive in project blocks
if ch == ":":
    tokens.append(Token(TokenType.COLON, ":", line_num, col + 1))
    col += 1

    if in_project_block:
        # Capture rest of line as STRING_VALUE (strip leading whitespace)
        rest = line_text[col:].strip()
        # Strip trailing comment
        for comment_start in ("#", "//"):
            idx = rest.find(comment_start)
            if idx >= 0:
                rest = rest[:idx].rstrip()
        if rest:
            tokens.append(Token(TokenType.STRING_VALUE, rest, line_num, col + 1))
        break  # consumed rest of line
    continue
```

Notice the `break` after capturing the `STRING_VALUE`. Since the rest of the line has been consumed, the inner character loop ends, and the lexer moves to the next line.

Also notice the comment stripping: before emitting the `STRING_VALUE`, the lexer scans for `#` or `//` within the rest-of-line text and strips everything after the comment marker. This means you can write:

```
project {
    name: University Database  # this is a comment
}
```

And the `STRING_VALUE` will be `"University Database"`, not `"University Database  # this is a comment"`.

Warning

Context sensitivity should be minimised. Every context-sensitive rule adds state to the lexer, making it harder to reason about and harder to test. MSD has exactly one context-sensitive case. If your DSL design calls for three or more, consider whether a syntax redesign could eliminate some of them. A common alternative is to use quoted strings (`name: "My Application"`) instead of rest-of-line capture, which removes the need for context sensitivity entirely — but at the cost of requiring users to type quotation marks around every metadata value.

Programmer's Perspective

Error reporting with precise line and column numbers is what transforms a frustrating DSL into a productive one, and Go's `token.Position` type shows the standard approach. Every token in Go's lexer carries a `token.Pos` value that can be resolved to a filename, line, and column through the `token.FileSet`. For your DSL, storing `(line, column)` on every token is cheap – two integers per token – and it pays dividends throughout the pipeline. The parser uses positions for error messages, the semantic analyser uses them for "did you mean?" suggestions, and IDE integrations use them for syntax highlighting and jump-to-definition. If you skip position tracking to save implementation effort, you will regret it the moment your first user reports a confusing error with no source location.

8.6 Error Detection and Reporting at the Lexical Level

The lexer is responsible for exactly one kind of error: **unexpected characters**. If the lexer encounters a character that does not begin any valid token — say, `@` or `$` — it records an error and moves on.

```
# Unknown character
errors.append(MSDError(
    message=f"unexpected character: '{ch}'",
    line=line_num,
    column=col + 1,
    filename=filename,
))
col += 1
```

Two design principles are at work here.

The lexer does not stop on error. After recording the unexpected character, it advances by one character and continues scanning. This means a single lexer pass can report multiple errors. If the user accidentally writes `entity Foo @@ { *id: INT }`, the lexer will report two "unexpected character" errors (one for each `@`) but will still produce valid tokens for `entity`, `Foo`, `{`, `*`, `id`, `:`, `INT`, and `}`. The parser can then

analyse the token stream and may succeed despite the lexer errors — or it may find further problems.

The lexer does not validate. It does not check whether `VARCAHR` is a valid data type — that is the parser's job (or the semantic analyser's). It does not check whether braces are balanced — that is the parser's job. It does not check whether an entity has a primary key — that is the semantic analyser's job. The lexer's sole concern is: "Does this character start a valid token?" If yes, consume the token. If no, report the error and skip the character.

This minimal error scope keeps the lexer simple and maintainable. Each pipeline stage handles a specific category of error:

Stage	Errors detected
Lexer	Unexpected characters
Parser	Structural errors: missing braces, wrong token order, unknown types
Semantic analyser	Domain errors: duplicate names, missing primary keys, unresolved references

---

8.7 MSD Lexer: Complete Walkthrough

With all the pieces in place, let us trace through the complete `MSDLexer.tokenize()` method. The inner loop tests characters in the following order:

1. **Whitespace**: spaces and tabs are skipped silently.
2. **Comments**: `#` or `//` cause a `break` — the rest of the line is ignored.
3. **Symbols**: `{`, `}`, `(`, `)`, `,`, `*` each emit their corresponding token.
4. **Colon** (context-sensitive): emits `COLON`, then captures `STRING_VALUE` if inside a project block.
5. **Integers**: sequences of digits are consumed and emitted as `INTEGER`.
6. **Identifiers/keywords**: sequences starting with a letter or underscore are consumed, then checked against the keyword table.
7. **Error**: any character that reaches this point is unexpected.

This ordering matters. Comments must be checked before symbols, because `//` starts with `/`, which might otherwise be treated as an unexpected character. Symbols must be checked before identifiers, because `*` might otherwise be misinterpreted. The integer check must come before the identifier check, because digits are not valid identifier-start characters — but if a digit appeared after the identifier check, it would not match `ch.isalpha()` and would fall through to the error case.

Token trace

Let us trace the lexer through this input:

```
entity Foo {
    *id: INT
}
```

The input has three lines. The lexer processes them one by one.

Line 1: `entity Foo {`

Position	Character(s)	Action	Token emitted
col 0–5	`entity`	Identifier consumed, matches keyword table	`ENTITY("entity", 1, 1)`
col 6		Whitespace skipped	—
col 7–9	`Foo`	Identifier consumed, not a keyword	`IDENTIFIER("Foo", 1, 8)`
col 10		Whitespace skipped	—
col 11	`{`	Symbol matched	`LBRACE("{", 1, 12)`
end of line	—	Automatic emission	`NEWLINE("\\n", 1, 12)`

Line 2: `*id: INT`

Position	Character(s)	Action	Token emitted
col 0–3		Whitespace skipped	—
col 4	`*`	Symbol matched	`STAR("*", 2, 5)`
col 5–6	`id`	Identifier consumed, not a keyword	`IDENTIFIER("id", 2, 6)`

Position	Character(s)	Action	Token emitted
col 7	`:`	Colon matched; `in_project_bl` is `False`, so no rest-of-line capture	`COLON(":", 2, 8)`
col 8		Whitespace skipped	—
col 9–11	`INT`	Identifier consumed, not a keyword	`IDENTIFIER("INT", 2, 10)`
end of line	—	Automatic emission	`NEWLINE("\\n", 2, 12)`

Line 3: }

Position	Character(s)	Action	Token emitted
col 0	`}`	Symbol matched; `brace_depth` decrements to 0	`RBRACE("}", 3, 1)`
end of line	—	Automatic emission	`NEWLINE("\\n", 3, 2)`

End of input:

The lexer appends `EOF("", 3, 0)`.

The complete token stream is:

```
ENTITY("entity", 1, 1)
IDENTIFIER("Foo", 1, 8)
LBRACE("{", 1, 12)
NEWLINE("\\n", 1, 12)
STAR("*", 2, 5)
IDENTIFIER("id", 2, 6)
COLON(":", 2, 8)
IDENTIFIER("INT", 2, 10)
NEWLINE("\\n", 2, 12)
```

```
RBRACE("}", 3, 1)
NEWLINE("\\n", 3, 2)
EOF("", 3, 0)
```

Twelve tokens from three lines of source code. Notice that `INT` is an `IDENTIFIER`, not a keyword — the lexer does not know that `INT` is a data type. That classification happens in the parser, which checks identifiers against a set of valid type names. The lexer's vocabulary is deliberately smaller than the language's vocabulary.

Context-sensitive trace

Now consider this input:

```
project {
    name: My App
}
```

Line 1: `project {`

The lexer consumes `project`, finds it in the keyword table, emits `PROJECT`, and sets `in_project_block = True`. It then emits `LBRACE` and increments `brace_depth` to 1.

Line 2: `name: My App`

The lexer skips whitespace, consumes `name` as an `IDENTIFIER`, then hits the colon. It emits `COLON`. Because `in_project_block` is `True`, it captures the rest of the line: `"My App"` becomes a `STRING_VALUE` token. The inner loop `breaks`.

Line 3: `}`

The lexer emits `RBRACE`, decrements `brace_depth` to 0, and clears `in_project_block`.

The token stream:

```
PROJECT("project", 1, 1)
LBRACE("{", 1, 9)
NEWLINE("\\n", 1, 10)
IDENTIFIER("name", 2, 5)
COLON(":", 2, 9)
STRING_VALUE("My App", 2, 11)
NEWLINE("\\n", 2, 15)
RBRACE("}", 3, 1)
NEWLINE("\\n", 3, 2)
EOF("", 3, 0)
```

Without the context-sensitive colon handling, `My` and `App` would have been emitted as two separate `IDENTIFIER` tokens, and the parser would have had to piece them back together — a far messier approach.

8.8 Design Decisions

Several design choices in the MSD lexer deserve explicit justification, as they represent trade-offs that every DSL author must consider.

Why line-by-line scanning?

Three features of MSD make line-by-line scanning the natural choice:

- **Comments** consume the rest of a line — `break` is the simplest possible implementation.
- **NEWLINE tokens** are emitted once per source line — the outer loop's end provides the natural emission point.
- **Context-sensitive colon** captures the rest of a line as a `STRING_VALUE` — `line_text[col:]` gives us exactly the right slice.

A character-by-character scanner would need to search forward for `\n` in each of these three cases.

Why not regular expressions?

Many lexer tutorials use regular expressions to define token patterns. For MSD, manual character matching is preferable for two reasons.

First, MSD is simple enough that the matching logic is a short sequence of `if` statements. Regular expressions would add syntactic overhead (compiling patterns, managing match groups) without reducing complexity.

Second, regular expressions cannot handle context sensitivity. The decision to capture the rest of a line as `STRING_VALUE` depends on the `in_project_block` flag, which is set by earlier tokens. A regex-based lexer would need to switch between different regex sets based on state — which is essentially a hand-written state machine with regex syntax layered on top, combining the worst of both approaches.

Why emit NEWLINE tokens?

MSD does not use newlines as statement terminators (unlike Python, which uses them heavily). So why emit them?

The answer lies in the parser. MSD's parser uses a `_skip_newlines()` helper to advance past any NEWLINE tokens before checking for the next meaningful token. This approach is simpler than having the lexer suppress newlines entirely, because the lexer would then need to know *which* newlines matter and which do not — a structural question that belongs to the parser, not the lexer.

By emitting every newline, the lexer stays simple (emit one after each line, unconditionally) and the parser stays simple (skip them whenever they appear). The two stages agree on a protocol: the lexer emits newlines liberally, the parser ignores them uniformly.

Why track column numbers?

Column numbers are not used by the MSD parser or semantic analyser — they are carried through the pipeline purely for error reporting. An error message that says `line 7, column 15: unexpected character: '$'` allows a text editor to jump directly to the problem. Without column numbers, the user must scan the entire line to find the offending character.

For a language like MSD where lines tend to be short, column numbers are a convenience. For languages with long lines (minified CSS, for instance), they are essential.

8.9 Exercises

Exercise 8.1 — Block comments. Extend the MSD lexer to support block comments (`/* ... */`) that can span multiple lines. What additional state must the lexer track? How does this interact with the line-by-line scanning approach? Write the modified scanning logic and explain how the lexer handles an unterminated block comment (one that starts with `/*` but never has a matching `*/`).

Exercise 8.2 — Case sensitivity. The MSD lexer uses `word.lower()` for keyword matching, making keywords case-insensitive: `Entity`, `ENTITY`, and `entity` are all recognised as the `ENTITY` keyword. What would change if keywords were case-sensitive? Would this simplify or complicate the lexer? What impact would it have on the user experience? Argue for one approach or the other.

Exercise 8.3 — Calculator lexer. Write a lexer for a simple calculator language that supports `+`, `-`, `*`, `/`, parentheses, integers, and floating-point numbers (e.g., `3.14`). Use the line-by-line scanning approach. Your lexer should handle multi-digit numbers, decimal points, and produce meaningful errors for unexpected characters. What token types do you need?

Exercise 8.4 — Token trace. Trace the MSD lexer through this input and list all tokens produced, including their types and values:

```
project {
    name: My App
}
entity X {
    *id: INT
}
```

Pay particular attention to the colon in the project block versus the colon in the entity block. How many tokens does the project block's colon produce compared to the entity block's colon?

Exercise 8.5 — String literals. Suppose MSD required quoted string values in project blocks (`name: "My Application"`) instead of using rest-of-line capture. Rewrite the colon-handling code to remove context sensitivity, and add a new token type

`QUOTED_STRING` with a scanning rule that handles opening and closing double quotes. What happens if the closing quote is missing? How would you report that error?

Exercise 8.6 — Performance. The MSD lexer calls `source.split("\n")` at the start, creating a list of all lines. For a 10,000-line MSD file, this is perfectly fine. At what scale might this become problematic? Describe an alternative approach that processes the source incrementally without splitting it into lines up front, while still preserving the benefits of line-by-line scanning.

Exercise 8.7 — Token location accuracy. The MSD lexer reports column numbers as 1-based (the first character of a line is column 1). Some tools use 0-based columns instead. What are the arguments for each convention? Which convention do most text editors and IDEs expect? Write a helper function that converts between the two representations.

Chapter 9: Building a Recursive Descent Parser

The lexer from Chapter 8 transforms raw source text into a stream of tokens. Each token carries a type, a value, and a position. But a flat sequence of tokens is not yet a *structure* — it does not tell us that `student_id` is an attribute of the `Student` entity, or that `(0,N)` is the cardinality of a particular link. That structural understanding is the parser's job.

This chapter builds a complete recursive descent parser for MSD. We start with the technique itself — why it works, why it is the most popular approach for hand-written parsers — and then construct every parsing method, from top-level dispatch down to individual type expressions. Along the way, we introduce intermediate representations, token stream navigation helpers, and a robust error recovery strategy that allows the parser to report multiple errors in a single pass.

By the end of this chapter, you will have a working parser that accepts MSD token streams and produces a clean intermediate representation ready for semantic analysis.

9.1 The Recursive Descent Technique

One Function Per Grammar Production

The core idea of recursive descent parsing is disarmingly simple: **write one function for each production rule in the grammar**. If the grammar has a rule for `entity_block`, you write a method called `_parse_entity_block()`. If it has a rule for `attribute`, you write `_parse_attribute()`. If it has a rule for `cardinality`, you write `_parse_cardinality()`.

Each function is responsible for recognising exactly the tokens described by its grammar rule. When a rule references another rule, the function calls the corresponding method. This is where the "recursive" in "recursive descent" comes from — a method can call other methods, and those methods can call further methods, naturally following the grammar's structure from top to bottom.

Consider this simplified fragment of the MSD grammar in EBNF:

```
entity_block   = "entity" IDENTIFIER "{" { attribute } "}" ;
attribute      = [ "*" ] IDENTIFIER ":" type_expr ;
type_expr      = IDENTIFIER [ "(" INTEGER ")" ] ;
```

The recursive descent translation is almost mechanical:

```
def _parse_entity_block(self):
    self._expect(TokenType.ENTITY)
    name_tok = self._expect(TokenType.IDENTIFIER)
    self._expect(TokenType.LBRACE)
    attributes = []
    while not self._check(TokenType.RBRACE):
        attributes.append(self._parse_attribute())
    self._expect(TokenType.RBRACE)

def _parse_attribute(self):
    is_pk = False
    if self._check(TokenType.STAR):
        self._advance()
        is_pk = True
    name_tok = self._expect(TokenType.IDENTIFIER)
    self._expect(TokenType.COLON)
    data_type, size = self._parse_type_expr()
    ...

def _parse_type_expr(self):
    type_tok = self._expect(TokenType.IDENTIFIER)
    size = None
    if self._check(TokenType.LPAREN):
        self._advance()
        size_tok = self._expect(TokenType.INTEGER)
        self._expect(TokenType.RPAREN)
    ...
```

Notice the direct correspondence. The EBNF `{ attribute }` (zero or more) becomes a `while` loop. The EBNF `[ "*" ]` (optional) becomes an `if` statement. The EBNF `"entity"` (terminal) becomes a call to `_expect()`. This mechanical mapping is precisely what makes recursive descent so appealing.

Top-Down Parsing

Recursive descent is a **top-down** parsing technique. It begins with the start symbol — the root of the grammar — and works downward towards the terminals (individual tokens). The parser never needs to "guess and backtrack" because MSD's grammar is designed to be LL(1): at every decision point, a single token of lookahead is sufficient to determine which production to apply.

The main parsing loop embodies this top-down approach:

```
def parse(self, source: str, filename: str = "") -> ParseResult:
    lexer = MSDLexer()
    tokens, lex_errors = lexer.tokenize(source, filename)

    self._tokens = tokens
    self._pos = 0
```

```
    self._result = ParseResult(filename=filename)
    self._result.errors.extend(lex_errors)

    self._skip_newlines()

    while not self._at_end():
        try:
            self._parse_top_level()
        except _ParsePanic:
            self._recover_to_top_level()

    return self._result
```

The loop repeatedly calls `_parse_top_level()` until every token has been consumed. Each call to `_parse_top_level()` peeks at the current token and dispatches to the appropriate handler — `_parse_entity_block()`, `_parse_association_block()`, `_parse_link_statement()`, or `_parse_project_block()`. Those handlers, in turn, call lower-level methods. The entire parse tree is constructed from the top down, one function call at a time.

Why Recursive Descent Is Popular

Recursive descent is the most widely used technique for hand-written parsers. GCC, Clang, the Go compiler, the V8 JavaScript engine, and the Rust compiler all use variants of recursive descent. The reasons are practical:

1. **Simplicity.** The code reads like a direct translation of the grammar. Anyone who understands the grammar can read the parser, and vice versa.
2. **Control.** Every parsing decision is explicit code. You can add context-sensitive logic, custom error messages, and recovery strategies exactly where they are needed.
3. **No dependencies.** The parser is pure application code. There is no parser generator tool to learn, no build step to maintain, and no generated code to debug.
4. **Excellent error messages.** Because you control every decision point, you can craft error messages that explain exactly what the parser expected and what it found.
5. **Debuggability.** You can set breakpoints, step through the parser, and inspect the token stream at any point. Try doing that with a parser table.

Tip

If your grammar is LL(1) — meaning each decision can be made by looking at a single token ahead — recursive descent is almost certainly the right choice. MSD's grammar is LL(1) by design: the first token of any top-level declaration (`entity`, `association`, `link`, `project`) immediately identifies the construct type.

Programmer's Perspective

Recursive descent parsing is the technique used by the Go compiler itself, and `go/parser` is the canonical reference implementation. Each grammar rule in the Go specification – `FunctionDecl`, `IfStmt`, `ForStmt` – has a corresponding `parseXxx` method in the parser source code. The Go team chose recursive descent over parser generators like `yacc` precisely because it gives full control over error messages and recovery. For your DSL, the same principle applies: each grammar production becomes a Go function, each `|` alternative becomes an `if/switch`, each `{ }` repetition becomes a `for` loop, and each `[ ]` optional becomes an `if`. Pratt parsing (operator precedence parsing) extends this approach when your DSL has expressions with infix operators – it handles precedence and associativity elegantly within the recursive descent framework.

9.2 Intermediate Representations

Why Not Build the Final Output Directly?

A natural question arises: why not construct the final model objects — `Entity`, `Association`, `Link` — directly during parsing? Why introduce an intermediate layer?

The answer involves separation of concerns. The parser's job is to recognise syntactic structure and report syntax errors. The builder's job (Chapter 10) is to resolve references, check semantic constraints, generate UUIDs, and construct the final model. Mixing these responsibilities creates several problems:

- **UUID generation during parsing.** Final model objects in Merisio carry UUIDs. Generating UUIDs during parsing means that a syntax error in one entity could leave orphaned UUIDs in the model. The intermediate representation avoids this: no UUIDs exist until the entire parse succeeds.
- **Dependency on the model layer.** If the parser imports and constructs `Entity` objects directly, it becomes tightly coupled to the model layer. Changes to the model (adding fields, changing constructors) break the parser. The intermediate dataclasses are simple, stable, and owned by the parser module.
- **Error recovery.** When the parser encounters an error and skips tokens, a partially constructed model object is in an inconsistent state. An intermediate dataclass can safely represent partial data — it is just a container of strings and integers, with no invariants to violate.
- **Testability.** Testing the parser in isolation is straightforward when it produces simple dataclasses. You do not need to set up a full project model, mock UUID generators, or wire up layout algorithms just to verify that an entity block parses correctly.

The Parsed Dataclasses

The MSD parser produces four intermediate dataclasses, each mirroring a grammar construct:

```
@dataclass
class ParsedAttribute:
    name: str
    data_type: str
    size: Optional[int] = None
    is_primary_key: bool = False
    line: int = 0
    column: int = 0

@dataclass
class ParsedEntity:
    name: str
    attributes: List[ParsedAttribute] = field(default_factory=list)
    line: int = 0
    column: int = 0

@dataclass
class ParsedAssociation:
    name: str
    attributes: List[ParsedAttribute] = field(default_factory=list)
    line: int = 0
    column: int = 0

@dataclass
class ParsedLink:
    entity_name: str
    cardinality_min: str = "0"
    cardinality_max: str = "N"
    association_name: str = ""
    line: int = 0
    column: int = 0

@dataclass
class ParsedMetadata:
    name: str = ""
    author: str = ""
    description: str = ""
```

Notice several deliberate design choices. First, every dataclass carries `line` and `column` fields. These are not needed for the data model — they exist purely so that later stages (the builder, the error reporter) can point back to the source location. Second, `ParsedLink` stores entity and association names as **strings**, not as object references or UUIDs. Name resolution happens later, in the builder. Third, cardinalities are stored as strings (`"0"`, `"1"`, `"N"`), not as enums or integers. The parser validates

that the values are legal, but it does not interpret them further.

ParseResult: The Container

All parsed data flows into a single container:

```
@dataclass
class ParseResult:
    entities: List[ParsedEntity] = field(default_factory=list)
    associations: List[ParsedAssociation] = field(default_factory=list)
    links: List[ParsedLink] = field(default_factory=list)
    metadata: Optional[ParsedMetadata] = None
    errors: List[MSDError] = field(default_factory=list)
    filename: str = ""

    @property
    def has_errors(self) -> bool:
        return any(e.severity == "error" for e in self.errors)
```

The `ParseResult` is the parser's single return value. It contains everything the builder needs: the parsed constructs, any metadata, the accumulated errors (from both the lexer and the parser), and the source filename for error reporting.

The `has_errors` property provides a convenient check: it returns `True` only if at least one error has severity `"error"`. Warnings (such as a size parameter on a type that does not accept one) do not prevent the build from proceeding. This distinction between errors and warnings is crucial for usability — users should be able to see warnings without losing their entire parse.

Note

The `ParseResult` also carries lexer errors, forwarded from the tokenisation stage. This means the builder receives a single, unified error list from all stages of front-end processing. The user sees all problems at once, regardless of which stage detected them.

Programmer's Perspective

The intermediate representation pattern used here – parsing into lightweight data structures before building the final model – is exactly how `go/parser` works in the Go toolchain. The `go/parser` package produces an `ast.File` containing `ast.GenDecl`, `ast.FuncDecl`, and other AST node types. These are simple structs with position information and string fields, not the resolved type-checked objects that `go/types` later produces. This separation lets you test parsing independently of semantic analysis, swap in different backends (code generation, linting, formatting), and recover gracefully from errors. In Go, your intermediate types would be plain structs with exported fields, and the `ParseResult` would be a struct with slices of those types – no interfaces, no methods, just data.

9.3 Token Stream Navigation

Every recursive descent parser needs a small set of helper methods for moving through the token stream. These helpers are the parser's fundamental operations — every parsing method is built from combinations of them. The MSD parser uses six.

`_peek()`: Look Without Consuming

```
def _peek(self) -> Token:
    return self._tokens[self._pos]
```

`_peek()` returns the current token without advancing the position. It is a read-only operation — the parser can call `_peek()` as many times as it likes without changing state. This is essential for making decisions: the parser examines the current token to determine which grammar rule to apply, then calls a more specific method to actually consume it.

`_advance()`: Consume and Return

```
def _advance(self) -> Token:
    tok = self._tokens[self._pos]
    self._pos += 1
    return tok
```

`_advance()` returns the current token and moves the position forward by one. After calling `_advance()`, the previous token is consumed — `_peek()` will now return the next token. This is the only method (along with `_expect()` and `_skip_newlines()`) that moves the position forward.

`_check(*types)`: Test Without Consuming

```
def _check(self, *types: TokenType) -> bool:
    return self._peek().type in types
```

`_check()` tests whether the current token matches one of the given types, returning a boolean without advancing. It is used for optional elements and loop conditions. For instance, `_check(TokenType.STAR)` tests whether the current attribute starts with a primary key marker, and `_check(TokenType.RBRACE)` tests whether the attribute list has ended.

`_expect(*types)`: Consume or Fail

```
def _expect(self, *types: TokenType) -> Token:
    self._skip_newlines()
    tok = self._peek()
    if tok.type in types:
```

```
        return self._advance()

    expected = " or ".join(t.name for t in types)
    self._error(f"expected {expected}, got {tok.type.name} ('{tok.value}')", tok)
    raise _ParsePanic()
```

`_expect()` is the workhorse. It asserts that the current token matches one of the expected types. If it does, the token is consumed and returned. If it does not, an error is recorded and a `_ParsePanic` exception is raised to trigger error recovery.

Note that `_expect()` calls `_skip_newlines()` before checking. This is a critical design choice: it means that newlines between tokens are transparent to the parser. Users can write entity blocks on one line or spread across many lines, and the parser handles both identically.

Also note that `_expect()` accepts **multiple** token types via `*types`. This is necessary for cardinality parsing, where the minimum value could be `0` (an `INTEGER` token) or `1` (also `INTEGER`), and the maximum value could be `1` (`INTEGER`) or `N` (an `IDENTIFIER` token). A single call `_expect(TokenType.INTEGER, TokenType.IDENTIFIER)` handles both cases.

`_skip_newlines()`: Consume Whitespace

```
def _skip_newlines(self):
    while not self._at_end() and self._peek().type == TokenType.NEWLINE:
        self._pos += 1
```

`_skip_newlines()` consumes all consecutive `NEWLINE` tokens at the current position. It is called at the start of `_expect()` and at various points where the parser transitions between constructs. By handling newlines explicitly rather than ignoring them in the lexer, the parser maintains the ability to use line boundaries for error recovery while keeping the parsing logic clean.

`_at_end()`: Check for EOF

```
def _at_end(self) -> bool:
    return self._peek().type == TokenType.EOF
```

`_at_end()` returns `True` when the parser has reached the end of the token stream. The lexer guarantees that the last token is always `EOF`, so this check is safe — `_peek()` will never read past the end of the list.

9.4 Parsing Blocks and Attributes

With the navigation helpers in place, we can build the parsing methods for MSD's block constructs. Entities and associations both follow the same pattern: a keyword, a name, an opening brace, a list of attributes, and a closing brace.

Entity Blocks

The grammar rule for entity blocks is:

```
entity_block = "entity" IDENTIFIER "{" { attribute } "}" ;
```

The parser method follows this rule directly:

```python
def _parse_entity_block(self):
    """Parse: entity Name { attributes... }"""
    kw_tok = self._expect(TokenType.ENTITY)
    name_tok = self._expect(TokenType.IDENTIFIER)
    self._skip_newlines()
    self._expect(TokenType.LBRACE)
    self._skip_newlines()

    entity = ParsedEntity(
        name=name_tok.value,
        line=name_tok.line,
        column=name_tok.column,
    )

    while not self._check(TokenType.RBRACE) and not self._at_end():
        self._skip_newlines()
        if self._check(TokenType.RBRACE):
            break
        try:
            attr = self._parse_attribute()
            entity.attributes.append(attr)
        except _ParsePanic:
            self._recover_to_brace_or_keyword()
            if self._check(TokenType.RBRACE):
                break
        self._skip_newlines()

    self._expect(TokenType.RBRACE)
    self._result.entities.append(entity)
```

The method first expects the `entity` keyword and the entity name. It then opens the brace and enters a loop that parses attributes until a closing brace is found. The `try/except _ParsePanic` block around attribute parsing is the error recovery mechanism — if a single attribute fails to parse, the parser skips to the next attribute or the closing brace and continues. This is what allows the parser to report errors in multiple attributes within the same entity.

After the loop, the closing brace is consumed with `_expect(TokenType.RBRACE)`, and the completed `ParsedEntity` is appended to the result.

Attribute Parsing

The grammar rule for attributes is:

```
attribute = [ "*" ] IDENTIFIER ":" type_expr ;
```

The parser method:

```python
def _parse_attribute(self) -> ParsedAttribute:
    """Parse: [*]name: TYPE[(size)]"""
    is_pk = False
    if self._check(TokenType.STAR):
        self._advance()
        is_pk = True

    name_tok = self._expect(TokenType.IDENTIFIER)
    self._expect(TokenType.COLON)

    data_type, size = self._parse_type_expr()

    return ParsedAttribute(
        name=name_tok.value,
        data_type=data_type,
        size=size,
        is_primary_key=is_pk,
        line=name_tok.line,
        column=name_tok.column,
    )
```

The optional `*` prefix is handled with `_check()` followed by `_advance()` — not `_expect()`, because the star is not required. The name and colon are mandatory, handled with `_expect()`. The type expression is delegated to `_parse_type_expr()`.

Type Expressions

The grammar rule for type expressions is:

```
type_expr = IDENTIFIER [ "(" INTEGER ")" ] ;
```

The parser method performs validation beyond pure syntax:

```python
def _parse_type_expr(self) -> Tuple[str, Optional[int]]:
    """Parse: TYPE or TYPE(size)"""
    type_tok = self._expect(TokenType.IDENTIFIER)
    type_name = type_tok.value.upper()

    if type_name not in DATA_TYPES:
        self._error(
            f"unknown data type: '{type_tok.value}' "
            f"(valid types: {', '.join(sorted(DATA_TYPES))})",
            type_tok,
```

```
        )
        raise _ParsePanic()

    size = None
    if self._check(TokenType.LPAREN):
        self._advance()
        size_tok = self._expect(TokenType.INTEGER)
        size = int(size_tok.value)
        self._expect(TokenType.RPAREN)

        if type_name not in SIZED_TYPES:
            self._error(
                f"data type '{type_name}' does not accept a size parameter",
                type_tok,
                severity="warning",
            )

    return type_name, size
```

The method validates the type name against the `DATA_TYPES` set — a set of thirteen uppercase strings including `INT`, `VARCHAR`, `TEXT`, `DATE`, `BOOLEAN`, and others. If the type is unrecognised, an error is raised. If the type is valid, the method checks for an optional size parameter in parentheses. If a size is provided for a type that does not accept one (e.g., `INT(10)`), a warning is emitted — not an error, because the type itself is valid and the parser can continue.

The `DATA_TYPES` and `SIZED_TYPES` sets are defined at module level:

```
DATA_TYPES = {
    "INT", "BIGINT", "SMALLINT",
    "VARCHAR", "CHAR", "TEXT",
    "BOOLEAN",
    "DATE", "TIME", "TIMESTAMP",
    "DECIMAL", "FLOAT", "DOUBLE",
}

SIZED_TYPES = {"VARCHAR", "CHAR", "DECIMAL"}
```

Association Blocks

Association blocks follow the same pattern as entity blocks. The only difference is the keyword and the result dataclass:

```
def _parse_association_block(self):
    """Parse: association Name { attributes... }"""
    kw_tok = self._expect(TokenType.ASSOCIATION)
    name_tok = self._expect(TokenType.IDENTIFIER)
    self._skip_newlines()
    self._expect(TokenType.LBRACE)
    self._skip_newlines()
```

```
    assoc = ParsedAssociation(
        name=name_tok.value,
        line=name_tok.line,
        column=name_tok.column,
    )

    while not self._check(TokenType.RBRACE) and not self._at_end():
        self._skip_newlines()
        if self._check(TokenType.RBRACE):
            break
        try:
            attr = self._parse_attribute()
            assoc.attributes.append(attr)
        except _ParsePanic:
            self._recover_to_brace_or_keyword()
            if self._check(TokenType.RBRACE):
                break
        self._skip_newlines()

    self._expect(TokenType.RBRACE)
    self._result.associations.append(assoc)
```

This structural similarity is not a coincidence — it reflects the domain. In MERISE, entities and associations both contain attributes. The grammar captures this shared structure, and the parser mirrors it.

9.5 Parsing Statements

Not every MSD construct is a block with braces. Link declarations are single-line statements.

Link Statements

The grammar rule for links is:

```
link_statement = "link" IDENTIFIER cardinality IDENTIFIER ;
cardinality    = "(" min "," max ")" ;
min            = "0" | "1" ;
max            = "1" | "N" ;
```

The parser method:

```
def _parse_link_statement(self):
    """Parse: link EntityName (min,max) AssociationName"""
    kw_tok = self._expect(TokenType.LINK)
    entity_tok = self._expect(TokenType.IDENTIFIER)
```

```
    card_min, card_max = self._parse_cardinality()

    assoc_tok = self._expect(TokenType.IDENTIFIER)

    link = ParsedLink(
        entity_name=entity_tok.value,
        cardinality_min=card_min,
        cardinality_max=card_max,
        association_name=assoc_tok.value,
        line=kw_tok.line,
        column=kw_tok.column,
    )
    self._result.links.append(link)
```

The method consumes the `link` keyword, the entity name, the cardinality (delegated to `_parse_cardinality()`), and the association name. The resulting `ParsedLink` stores names as strings — resolution to actual entity and association objects happens in the builder.

Cardinality Parsing

Cardinality parsing is the most nuanced part of the MSD parser, because the grammar involves tokens that could be either integers or identifiers:

```
def _parse_cardinality(self) -> Tuple[str, str]:
    """Parse: (min,max) where min in {0,1} and max in {1,N}"""
    self._expect(TokenType.LPAREN)
    min_tok = self._expect(TokenType.INTEGER, TokenType.IDENTIFIER)
    min_val = min_tok.value

    if min_val not in ("0", "1"):
        self._error(
            f"invalid minimum cardinality: '{min_val}' (expected 0 or 1)",
            min_tok,
        )
        raise _ParsePanic()

    self._expect(TokenType.COMMA)
    max_tok = self._expect(TokenType.INTEGER, TokenType.IDENTIFIER)
    max_val = max_tok.value.upper()

    if max_val not in ("1", "N"):
        self._error(
            f"invalid maximum cardinality: '{max_tok.value}' (expected 1 or N)",
            max_tok,
        )
        raise _ParsePanic()
```

```
    self._expect(TokenType.RPAREN)
    return min_val, max_val
```

The subtlety here is that `0` and `1` are lexed as `INTEGER` tokens, while `N` is lexed as an `IDENTIFIER` token. The lexer has no knowledge of cardinalities — it simply tokenises characters into their natural types. The parser must therefore accept both token types at each position. This is why `_expect()` accepts multiple types: `_expect(TokenType.INTEGER, TokenType.IDENTIFIER)` matches either token type.

After consuming the token, the parser validates the actual value. A minimum cardinality must be `"0"` or `"1"`. A maximum cardinality must be `"1"` or `"N"` (case-insensitive, normalised to uppercase). Any other value produces a specific, helpful error message.

Warning

It would be tempting to add dedicated `ZERO`, `ONE`, and `N` token types to the lexer to simplify cardinality parsing. Resist this temptation. Those "tokens" are context-dependent — `N` is only special inside a cardinality expression, not when used as an entity name. Overloading the lexer with parser-level semantics creates fragile, context-sensitive tokenisation that is far harder to maintain.

9.6 Error Recovery in the Parser

A parser that stops at the first error is a parser that wastes its users' time. If a file has five errors, the user must fix one, re-run, fix the next, re-run, and so on through five edit-compile cycles. A good parser reports all five errors in a single pass.

Panic-Mode Recovery

The MSD parser uses **panic-mode recovery**, the simplest strategy that produces robust results. The idea is straightforward:

1. When the parser encounters an unexpected token, it records an error.
2. It raises a lightweight `_ParsePanic` exception to unwind the call stack.
3. A handler at a higher level catches the exception.
4. The handler skips tokens until it reaches a **synchronisation point** — a position where parsing can safely resume.
5. Parsing continues from the synchronisation point.

The `_ParsePanic` exception is not a real error in the Python exception sense. It is a control flow mechanism — a way to jump from a deeply nested parsing method back to a recovery handler without threading return codes through every intermediate function:

```
class _ParsePanic(Exception):
    """Internal exception for panic-mode error recovery."""
    pass
```

Two Recovery Strategies

The MSD parser uses two recovery strategies at different granularities.

Top-level recovery handles errors between top-level declarations. When a `_ParsePanic` escapes `_parse_top_level()`, the main loop catches it and calls `_recover_to_top_level()`:

```
def _recover_to_top_level(self):
    """Panic-mode recovery: skip to next top-level keyword or closing brace."""
    while not self._at_end():
        tok = self._peek()
        if tok.type in (TokenType.ENTITY, TokenType.ASSOCIATION,
                        TokenType.LINK, TokenType.PROJECT):
            return
        if tok.type == TokenType.RBRACE:
            self._advance()
            return
        self._advance()
```

This method skips tokens until it finds either a top-level keyword (indicating the start of a new declaration) or a closing brace (indicating the end of a block). If it finds a closing brace, it consumes it — this handles the case where an error occurred inside a block and the parser needs to skip past the block's closing delimiter.

Attribute-level recovery handles errors within entity and association blocks. When `_parse_attribute()` raises `_ParsePanic`, the enclosing block parser catches it and calls `_recover_to_brace_or_keyword()`:

```
def _recover_to_brace_or_keyword(self):
    """Skip to next closing brace or top-level keyword."""
    while not self._at_end():
        tok = self._peek()
        if tok.type == TokenType.RBRACE:
            return
        if tok.type in (TokenType.ENTITY, TokenType.ASSOCIATION,
                        TokenType.LINK, TokenType.PROJECT):
            return
        if tok.type == TokenType.STAR:
            return
        # Stop at identifier followed by colon (next attribute)
        if (tok.type == TokenType.IDENTIFIER
                and self._pos + 1 < len(self._tokens)):
            next_tok = self._tokens[self._pos + 1]
            if next_tok.type == TokenType.COLON:
                return
```

```
        self._advance()
```

This method is more refined than `_recover_to_top_level()`. It recognises four synchronisation points: a closing brace (end of the block), a top-level keyword (a new declaration — the block was probably not closed), a `*` token (start of a new primary key attribute), or an identifier followed by a colon (start of a new regular attribute).

The two-level strategy means that a single bad attribute does not invalidate the entire entity. The parser reports the error, skips to the next recognisable attribute, and continues parsing. The user sees errors for every malformed attribute in a single run.

How Recovery Works in Practice

Consider this input with an error on line 3:

```
entity Student {
    *student_id: INT
    email: BLOB
    name: VARCHAR(100)
}
```

`BLOB` is not a recognised MSD data type. Here is what happens:

1. The parser enters `_parse_entity_block()` and successfully parses the entity name and opening brace.
2. It parses `*student_id: INT` successfully.
3. It enters `_parse_attribute()` for `email: BLOB`. The `_parse_type_expr()` method finds that `BLOB` is not in `DATA_TYPES`, records an error, and raises `_ParsePanic`.
4. The `except _ParsePanic` handler in `_parse_entity_block()` calls `_recover_to_brace_or_keyword()`.
5. The recovery method advances through the token stream. It finds `name` (an `IDENTIFIER`) followed by `:` (a `COLON`) — a synchronisation point. It stops.
6. The `while` loop in `_parse_entity_block()` continues. It parses `name: VARCHAR(100)` successfully.
7. The closing brace is consumed and the entity is appended to the result.

The final `ParseResult` contains one entity (`Student`) with two attributes (`student_id` and `name`) and one error (unknown data type `BLOB` on line 3). The `email` attribute is lost — it could not be parsed — but everything else was recovered.

9.7 MSD Parser: Complete Walkthrough

Let us trace the parser through a complete MSD file, following every method call and every token consumed. This walkthrough brings together all the concepts from the

preceding sections.

Input File

```
entity Student {
    *student_id: INT
    name: VARCHAR(100)
}

link Student (0,N) enrolled_in
```

Token Stream

The lexer produces (omitting NEWLINE tokens for clarity):

#	Type	Value	Line	Col
1	ENTITY	`entity`	1	1
2	IDENTIFIER	`Student`	1	8
3	LBRACE	`{`	1	16
4	STAR	`*`	2	5
5	IDENTIFIER	`student_id`	2	6
6	COLON	`:`	2	16
7	IDENTIFIER	`INT`	2	18
8	IDENTIFIER	`name`	3	5
9	COLON	`:`	3	9
10	IDENTIFIER	`VARCHAR`	3	11
11	LPAREN	`(`	3	18
12	INTEGER	`100`	3	19
13	RPAREN	`)`	3	22
14	RBRACE	`}`	4	1
15	LINK	`link`	6	1
16	IDENTIFIER	`Student`	6	6
17	LPAREN	`(`	6	14
18	INTEGER	`0`	6	15
19	COMMA	`,`	6	16
20	IDENTIFIER	`N`	6	17
21	RPAREN	`)`	6	18
22	IDENTIFIER	`enrolled_in`	6	20

#	Type	Value	Line	Col
23	EOF		6	0

Step-by-Step Trace

Step 1: Main loop calls `_parse_top_level()`. The parser skips newlines and peeks at token #1: `ENTITY`. It dispatches to `_parse_entity_block()`.

Step 2: `_parse_entity_block()` begins.

- `_expect(TokenType.ENTITY)` consumes token #1 (`entity`).
- `_expect(TokenType.IDENTIFIER)` consumes token #2 (`Student`). The entity name is `"Student"`.
- `_skip_newlines()` skips any intervening newlines.
- `_expect(TokenType.LBRACE)` consumes token #3 (`{`).
- Creates `ParsedEntity(name="Student", line=1, column=8)`.
- Enters the attribute loop.

Step 3: First attribute — `*student_id: INT`.

- `_check(TokenType.RBRACE)` → `False`. Loop continues.
- `_parse_attribute()` is called.
  - `_check(TokenType.STAR)` → `True`. Advance consumes token #4. `is_pk = True`.
  - `_expect(TokenType.IDENTIFIER)` consumes token #5 (`student_id`).
  - `_expect(TokenType.COLON)` consumes token #6 (`:`).
  - `_parse_type_expr()` is called.
    - `_expect(TokenType.IDENTIFIER)` consumes token #7 (`INT`). `type_name = "INT"`.
    - `"INT"` is in `DATA_TYPES`. No error.
    - `_check(TokenType.LPAREN)` → `False`. No size parameter.
    - Returns `("INT", None)`.
  - Returns `ParsedAttribute(name="student_id", data_type="INT", is_primary_key=True, ...)`.
- Attribute appended to entity.

Step 4: Second attribute — `name: VARCHAR(100)`.

- `_check(TokenType.RBRACE)` → `False`. Loop continues.
- `_parse_attribute()` is called.
  - `_check(TokenType.STAR)` → `False`. `is_pk = False`.
  - `_expect(TokenType.IDENTIFIER)` consumes token #8 (`name`).
  - `_expect(TokenType.COLON)` consumes token #9 (`:`).
  - `_parse_type_expr()` is called.
    - `_expect(TokenType.IDENTIFIER)` consumes token #10 (`VARCHAR`). `type_name = "VARCHAR"`.

        * "VARCHAR" is in DATA_TYPES. No error.
        * _check(TokenType.LPAREN) → True. Advance consumes token #11 (().
        * _expect(TokenType.INTEGER) consumes token #12 (100). size = 100.
        * _expect(TokenType.RPAREN) consumes token #13 ()).
        * "VARCHAR" is in SIZED_TYPES. No warning.
        * Returns ("VARCHAR", 100).
    - Returns ParsedAttribute(name="name", data_type="VARCHAR", size=100, ...).
- Attribute appended to entity.

Step 5: End of entity block.

- _skip_newlines() skips any newlines.
- _check(TokenType.RBRACE) → True. Loop exits.
- _expect(TokenType.RBRACE) consumes token #14 (}).
- ParsedEntity with two attributes appended to self._result.entities.

Step 6: Main loop calls _parse_top_level() again. Skips newlines. Peeks at token #15: LINK. Dispatches to _parse_link_statement().

Step 7: _parse_link_statement() begins.

- _expect(TokenType.LINK) consumes token #15 (link).
- _expect(TokenType.IDENTIFIER) consumes token #16 (Student).
- _parse_cardinality() is called.
    - _expect(TokenType.LPAREN) consumes token #17 (().
    - _expect(TokenType.INTEGER, TokenType.IDENTIFIER) consumes token #18 (0). min_val = "0".
    - "0" is in ("0", "1"). Valid.
    - _expect(TokenType.COMMA) consumes token #19 (,).
    - _expect(TokenType.INTEGER, TokenType.IDENTIFIER) consumes token #20 (N). max_val = "N".
    - "N" is in ("1", "N"). Valid.
    - _expect(TokenType.RPAREN) consumes token #21 ()).
    - Returns ("0", "N").
- _expect(TokenType.IDENTIFIER) consumes token #22 (enrolled_in).
- ParsedLink appended to self._result.links.

Step 8: Main loop checks for more declarations. Peeks at token #23: EOF. _at_end() returns True. Loop exits.

Final ParseResult

```
ParseResult(
    entities=[
        ParsedEntity(
            name="Student",
            attributes=[
                ParsedAttribute(name="student_id", data_type="INT",
                                size=None, is_primary_key=True,
                                line=2, column=6),
```

```
                ParsedAttribute(name="name", data_type="VARCHAR",
                                size=100, is_primary_key=False,
                                line=3, column=5),
            ],
            line=1, column=8,
        ),
    ],
    associations=[],
    links=[
        ParsedLink(entity_name="Student",
                   cardinality_min="0", cardinality_max="N",
                   association_name="enrolled_in",
                   line=6, column=1),
    ],
    metadata=None,
    errors=[],
    filename="",
)
```

No errors, no warnings. The parse result is a complete, clean intermediate representation of the source file, ready for the builder to transform into a Merisio project.

9.8 Design Decisions

Several design decisions in the MSD parser deserve explicit justification. These decisions are not unique to MSD — they arise in almost every hand-written parser.

Why Recursive Descent Over Parser Generators?

Parser generators like ANTLR, PLY, or Lark could generate a parser from the MSD grammar automatically. Why write one by hand?

Simplicity of the grammar. MSD has four top-level constructs and roughly fifteen grammar rules. A recursive descent parser for this grammar is under 200 lines of code. The overhead of learning a parser generator, writing a grammar file, integrating the generated code, and debugging generated error messages would exceed the cost of writing the parser directly.

Natural error messages. A hand-written parser can produce error messages that read like natural English: “expected data type after ‘:’, got ‘{’ on line 7”. A parser generator’s error messages are typically generic — “unexpected token” or “parse error” — and improving them requires fighting the generator’s infrastructure.

No dependencies. The MSD parser is pure Python. It requires no external tool, no build step, and no generated files. Anyone can read the code, set a breakpoint, and step through the parsing logic. This matters enormously for a teaching project.

Control over recovery. The two-level panic-mode recovery strategy (top-level and attribute-level) would be difficult to express in most parser generator frameworks. Hand-written code gives complete control over when and how the parser recovers.

Why Intermediate Representation?

The separation between `ParsedEntity` (parser output) and `Entity` (model object) exists for clean architectural boundaries. The parser knows about tokens and grammar rules. The builder knows about UUIDs, name resolution, and model invariants. Neither needs to understand the other's concerns. If the model layer changes — adding a colour field to entities, for instance — the parser remains unchanged.

Why Multiple Error Reporting?

A parser that reports only the first error forces users into a slow, frustrating cycle: fix one error, re-run, discover the next error, fix it, re-run. The MSD parser reports every error it can find in a single pass. The `ParseResult.errors` list may contain errors from the lexer (unexpected characters), from the parser (unexpected tokens, invalid types, invalid cardinalities), and later from the builder (duplicate names, unresolved references). The user sees all of them at once.

This is only possible because of panic-mode recovery. Without recovery, the parser would abort at the first error, and all subsequent errors would go undetected.

Why `_ParsePanic` as an Exception?

The alternative to `_ParsePanic` is threading error states through return values. Every parsing method would return an `Optional` or a result type, and every call site would check for failure. This is feasible but verbose — it obscures the happy path with error-handling boilerplate.

`_ParsePanic` is a lightweight control flow mechanism. It unwinds the call stack to the nearest `try/except` handler, which performs recovery. The happy path — the normal parsing code — reads cleanly without error checks. The unhappy path — the recovery logic — is concentrated in a few well-defined handlers. This separation makes both paths easier to read, write, and test.

Info

The `_ParsePanic` class is deliberately named with a leading underscore and defined at module level rather than as a public class. It is an internal implementation detail of the parser, not part of the public API. No code outside the parser module should ever catch or raise this exception.

9.9 Exercises

Exercise 9.1 — Project Block Parsing. The MSD parser includes a `_parse_project_block()` method for parsing `project { key: value ... }` blocks. Write this method yourself, following the patterns established in Section 9.4. Your method should: (a) consume the `project` keyword and opening brace, (b) loop over key-value pairs where the key is an `IDENTIFIER`, followed by a `COLON`, followed by a `STRING_VALUE`, (c) populate a `ParsedMetadata` dataclass with recognised keys (`name`, `author`, `description`), (d) emit a warning for unrecognised keys, and (e) consume the closing brace. Compare your implementation with the production code.

Exercise 9.2 — Comment Block Construct. Add support for a `comment` block construct to MSD: `comment { free text here }`. Write the `_parse_comment_block()` method and add it to `_parse_top_level()`. The comment block should be parsed but not stored in the `ParseResult` — it is purely for human documentation. What changes to the lexer are needed? Consider edge cases: what happens if the free text contains `}` characters?

Exercise 9.3 — Error Trace. Trace the parser through this input with an error:

```
entity A {
    x: BLOB
    name: TEXT
}
```

Show which methods are called, which tokens are consumed, when `_ParsePanic` is raised, which recovery method runs, and what the final `ParseResult` contains. Include both the successfully parsed attributes and the error list.

Exercise 9.4 — Arithmetic Expression Parser. Write a recursive descent parser for arithmetic expressions with `+`, `-`, `*`, `/`, parentheses, and integer literals. Handle operator precedence correctly (`*` and `/` bind tighter than `+` and `-`). Your parser should produce an abstract syntax tree using dataclasses. Hint: you will need separate rules for `expression`, `term`, and `factor`.

Exercise 9.5 — Multi-Type `_expect()`. The MSD parser's `_expect()` method accepts multiple token types via `*types`. Why is this necessary for cardinality parsing? Could the grammar be redesigned to avoid this — for instance, by making `N` a keyword? What are the trade-offs of each approach?

Exercise 9.6 — Recovery Strategy Comparison. The MSD parser uses two recovery strategies: `_recover_to_top_level()` and `_recover_to_brace_or_keyword()`. Design a third recovery strategy for a hypothetical MSD extension that allows nested entity blocks (entities within entities). What synchronisation points would your recovery method use? How would it interact with the existing two strategies?

Exercise 9.7 — Parser Testing. Write a test function that parses the following MSD input and asserts that the `ParseResult` contains the correct entities, attributes, links, and no errors:

```
entity Book {
    *isbn: VARCHAR(13)
```

```
    title: VARCHAR(255)
}

entity Author {
    *author_id: INT
    name: VARCHAR(100)
}

association written_by {
}

link Book (1,N) written_by
link Author (0,N) written_by
```

Then write a second test function that introduces a deliberate error (e.g., an invalid data type) and asserts that the `ParseResult` contains the expected error message while still parsing all recoverable constructs. What does your test tell you about the parser's robustness?

Summary

This chapter built a complete recursive descent parser for MSD — the longest and most detailed implementation chapter in the book. We covered seven key topics:

1. **The recursive descent technique** maps grammar productions to methods, uses top-down parsing with single-token lookahead, and is the most popular approach for hand-written parsers due to its simplicity, control, and debuggability.
2. **Intermediate representations** decouple the parser from the model layer. The `Parsed*` dataclasses carry no UUIDs, no references, and no invariants — they are pure data containers that the builder transforms into the final model.
3. **Token stream navigation** relies on six essential helpers: `_peek()`, `_advance()`, `_check()`, `_expect()`, `_skip_newlines()`, and `_at_end()`. These form the parser's fundamental vocabulary.
4. **Block and attribute parsing** follows the grammar rule by rule: entity blocks contain attribute lists, attributes have optional primary key markers and mandatory type expressions, and type expressions validate against a known set of data types.
5. **Statement parsing** handles link declarations and cardinality expressions, demonstrating how the parser accepts multiple token types at a single position.
6. **Error recovery** uses panic-mode recovery at two granularities — top-level and attribute-level — allowing the parser to report multiple errors per pass and to continue parsing after failures.

7. **Design decisions** justified the choice of recursive descent over parser generators, intermediate representations over direct model construction, multi-error reporting over single-error abort, and exception-based recovery over return-code threading.

In the next chapter, we move from syntax to semantics. The builder takes the `ParseResult` produced here and transforms it into a complete Merisio project — resolving names to objects, detecting duplicates, suggesting corrections for typos, and generating UUIDs for every element.

Chapter 10: Semantic Analysis

The parser, as we built it in Chapter 9, produces an intermediate representation — a collection of `ParsedEntity`, `ParsedAssociation`, and `ParsedLink` objects. This representation is *syntactically* valid: every brace is matched, every cardinality is well-formed, every data type is recognised. But syntactic validity is not enough. A file can be perfectly structured and still meaningless: an entity named `Student` might be declared twice, a link might reference an association called `enrolment` that does not exist, or an association might share its name with an entity. These are not syntax errors — they are *semantic* errors, and catching them is the job of the semantic analyser.

This chapter covers the design and implementation of MSD's semantic analyser, which we call the **builder**. The builder takes the parser's intermediate representation and transforms it into the final `Project` model — resolving names to identifiers, detecting duplicates and conflicts, and producing helpful error messages when something goes wrong.

10.1 From Syntax Tree to Meaning

The gap between syntax and semantics

A parser answers the question: *is this text structurally valid?* A semantic analyser answers a different question: *does this text mean something coherent?* The distinction is fundamental. Consider the following MSD fragment:

```
entity Student {
    *student_id: INT
    name: VARCHAR(100)
}

entity Student {
    *id: INT
    email: VARCHAR(255)
}
```

The parser will happily produce two `ParsedEntity` objects, both named `Student`. The syntax is perfectly valid — each entity block has a name, braces, and well-formed attributes. But the *meaning* is incoherent: which `Student` should a link refer to? The parser does not know and does not care. Detecting this conflict is the semantic analyser's responsibility.

The builder's role

In MSD's architecture, the semantic analyser is called the **builder** because its primary job is to *build* the final `Project` model from the parser's intermediate representation. The builder is not merely a validator — it is a transformer. It takes `ParsedEntity` objects (which carry names, line numbers, and raw attribute data) and produces `Entity` objects (which carry UUIDs, attribute objects, and layout coordinates). It takes `ParsedLink` objects (which reference entities and associations by name) and produces `Link` objects (which reference them by UUID).

The builder's signature captures this transformation:

```
class MSDProjectBuilder:
    def build(self, parse_result: ParseResult) -> Tuple[Optional[Project], List[MSDErro
        ...
```

It accepts a `ParseResult` — the parser's output — and returns a tuple of the constructed `Project` and a list of errors. Crucially, the builder always returns a project, even when errors are present. Partial results are useful: a GUI can display the entities and associations that *were* valid, even if some links failed to resolve.

Three phases of semantic analysis

The builder's work falls into three phases, each addressing a distinct class of semantic problem:

1. **Name resolution.** The parser records that a link connects entity `"Student"` to association `"enrolment"`. The builder must find the actual `Entity` and `Association` objects that those names refer to, then replace the names with UUIDs.
2. **Duplicate and conflict detection.** Two entities with the same name, two associations with the same name, or an association that shares its name with an entity — all must be detected and reported.
3. **Constraint checking.** Domain-specific rules that span multiple constructs: does every entity have a primary key? Are the references in links valid?

These three phases are not implemented as separate passes over the data. Instead, they are interleaved within a single traversal — but understanding them as conceptually distinct phases clarifies the design.

10.2 Name-to-ID Resolution and Symbol Tables

The problem

In an MSD file, links reference entities and associations by **name**:

```
link Student (0,N) enrolment
link Course (1,N) enrolment
```

But in the final `Project` model, links reference them by **UUID**:

```
@dataclass
class Link:
    entity_id: str       # UUID, e.g. "a3f8c1d2-..."
    association_id: str   # UUID, e.g. "7b9e4f01-..."
    cardinality_min: str
    cardinality_max: str
    id: str = field(default_factory=lambda: str(uuid.uuid4()))
```

The builder must bridge this gap. It must take the name `"Student"` and find the `Entity` object whose `.name` is `"Student"`, then use that object's `.id` (a UUID generated automatically by the dataclass default factory) when constructing the `Link`.

Symbol tables

The classical solution is a **symbol table** — a dictionary that maps names to the objects they denote. The builder maintains two symbol tables:

```
entity_names: Dict[str, Entity] = {}
assoc_names: Dict[str, Association] = {}
```

As the builder processes each entity, it registers the entity's name in `entity_names`. As it processes each association, it registers the name in `assoc_names`. When it later processes links, it looks up names in these tables to find the corresponding objects and their UUIDs.

Build order enables forward references

The builder processes constructs in a fixed order: entities first, then associations, then links. This ordering is deliberate. Because links are processed last, *all* entity and association names are already registered by the time any link is resolved. This means the MSD file can place constructs in any order:

```
# Links appear before entities — still works
link Student (0,N) enrolment
link Course (1,N) enrolment

entity Student {
    *student_id: INT
    name: VARCHAR(100)
}

entity Course {
    *course_id: INT
    title: VARCHAR(200)
}

association enrolment {
    grade: DECIMAL(4)
```

```
}
```

The parser collects all constructs into separate lists (`parse_result.entities`, `parse_result.associations`, `parse_result.links`). The builder then processes these lists in the correct order, regardless of the order they appeared in the source file. Forward references are thus supported without any additional complexity.

The resolution code

When resolving a link's entity reference, the builder performs a simple dictionary lookup:

```
entity = entity_names.get(pl.entity_name)
if entity is None:
    msg = f"unknown entity: '{pl.entity_name}'"
    suggestion = _suggest(pl.entity_name, all_entity_names)
    if suggestion:
        msg += f" (did you mean '{suggestion}'?)"
    errors.append(MSDError(
        message=msg,
        line=pl.line,
        column=pl.column,
        filename=filename,
        severity="error",
    ))
    continue
```

If the lookup succeeds, the builder has the `Entity` object and can use `entity.id` to construct the link. If it fails, the builder reports an error — and, as we shall see in Section 10.4, attempts to suggest the correct name.

Note

The `continue` statement after reporting an unresolved name is important. The builder does not attempt to create a `Link` with a missing entity or association — doing so would produce a malformed `Link` object with a `None` ID, which would cause errors downstream. Skipping the link entirely is the safer choice.

Programmer's Perspective

The visitor pattern and Go's `ast.Walk` function offer two contrasting approaches to walking a parsed tree for validation. The visitor pattern (used in Java and Python AST libraries) defines an interface with a `Visit` method for each node type, and the tree drives the traversal. Go's `ast.Walk` takes the opposite approach: a single `func(ast.Node) bool` callback visits every node, and the walker handles the traversal. For small DSLs like MSD, a simple loop over slices (entities, then associations, then links) is more straightforward than either pattern. For larger DSLs with deeply nested ASTs, implementing Go's `ast.Inspect` pattern – where a visitor function returns `true` to descend into children or `false` to skip them – gives you clean, composable validation passes without the boilerplate of a full visitor interface.

10.3 Duplicate Detection and Conflict Checking

Duplicate entity names

When the builder encounters an entity whose name already exists in the `entity_names` symbol table, it reports an error and skips the duplicate:

```
for pe in parse_result.entities:
    if pe.name in entity_names:
        errors.append(MSDError(
            message=f"duplicate entity name: '{pe.name}'",
            line=pe.line,
            column=pe.column,
            filename=filename,
            severity="error",
        ))
        continue

    entity = Entity(name=pe.name)
    # ... build attributes ...
    entity_names[pe.name] = entity
    project.add_entity(entity)
```

The first entity with a given name wins. The duplicate is discarded entirely — it is not added to the project, and its name is not registered in the symbol table. This "first wins" policy is simple and predictable. An alternative would be to discard *both* and require the user to resolve the ambiguity, but in practice, the first-wins approach produces clearer error messages and fewer cascading errors.

Duplicate association names

The same logic applies to associations. If two associations share a name, the second is reported as a duplicate and skipped:

```
for pa in parse_result.associations:
    if pa.name in assoc_names:
        errors.append(MSDError(
            message=f"duplicate association name: '{pa.name}'",
            line=pa.line,
            column=pa.column,
            filename=filename,
            severity="error",
        ))
        continue
```

Cross-namespace conflicts

MSD also checks for a subtler problem: an association whose name matches an existing entity name:

```
if pa.name in entity_names:
    errors.append(MSDError(
        message=f"association name '{pa.name}' conflicts with an entity of the same nam
        line=pa.line,
        column=pa.column,
        filename=filename,
        severity="error",
    ))
    continue
```

Why does this matter? Consider a link statement:

```
link Booking (1,1) Booking
```

If `Booking` is both an entity and an association, the link's entity reference and association reference are both `"Booking"`. In our current builder, these resolve against separate symbol tables, so there would be no ambiguity at the implementation level. However, from the user's perspective, having an entity and an association with the same name is deeply confusing and almost certainly a mistake. The cross-namespace conflict check catches this.

Warning

Cross-namespace conflicts are an error, not a warning. Although the builder *could* resolve the names unambiguously (entity names and association names live in separate dictionaries), allowing the same name in both namespaces would make the MSD file confusing for human readers and error-prone for future maintenance.

Because the builder processes entities before associations, all entity names are already in `entity_names` when associations are checked. This ordering makes the conflict check trivial — a single dictionary lookup.

10.4 "Did You Mean?" Suggestions via Levenshtein Distance

The problem of near-misses

When a name cannot be resolved, the raw error message — `unknown entity: 'Tourits'` — is accurate but unhelpful. The user stares at the error, checks their file, and eventually realises they meant `Tourist`. A good semantic analyser should make this connection for them:

```
Error: unknown entity: 'Tourits' (did you mean 'Tourist'?)
```

This "did you mean?" feature turns a frustrating debugging session into an instant fix. It is one of the highest-value-to-effort-ratio improvements you can make to a DSL's error reporting.

The Levenshtein edit distance

The **Levenshtein distance** between two strings is the minimum number of single-character operations — insertions, deletions, or substitutions — needed to transform one string into the other.

For example, transforming `"Tourits"` into `"Tourist"`:

```
Tourits
     ^   — the 'i' and 't' are swapped compared to 'Tourist'

Step 1: Tourits → Tourists  (insert 's'? No, let's think differently)

Actually:
  T o u r i t s     (source, length 7)
  T o u r i s t     (target, length 7)

Position:  1 2 3 4 5 6 7
Source:    T o u r i t s
Target:    T o u r i s t

Positions 1–5 match. Position 6: 't' vs 's' — substitution.
Position 7: 's' vs 't' — substitution.

Levenshtein distance = 2
```

Two substitutions. The distance is 2, which is close enough to suggest `"Tourist"` as a correction.

The dynamic programming algorithm

The classic algorithm computes the Levenshtein distance using a dynamic programming matrix. For strings `a` of length `m` and `b` of length `n`, we build an `(m+1) x (n+1)` matrix where cell `(i, j)` holds the edit distance between the first `i` characters of `a` and the first `j` characters of `b`.

MSD uses a space-optimised variant that keeps only two rows of the matrix at a time, reducing space complexity from O(m*n) to O(min(m, n)):

```python
def _levenshtein(a: str, b: str) -> int:
    """Compute Levenshtein edit distance between two strings."""
    if len(a) < len(b):
        return _levenshtein(b, a)
    if not b:
        return len(a)

    prev = list(range(len(b) + 1))
    for i, ca in enumerate(a):
        curr = [i + 1]
        for j, cb in enumerate(b):
            cost = 0 if ca == cb else 1
            curr.append(min(curr[j] + 1, prev[j + 1] + 1, prev[j] + cost))
        prev = curr
    return prev[-1]
```

The algorithm works as follows:

- `prev` holds the previous row of the DP matrix. Initially, `prev = [0, 1, 2, ..., len(b)]`, representing the cost of transforming an empty prefix of `a` into each prefix of `b` (all insertions).
- For each character `ca` in `a`, we build a new row `curr`. The first element is `i + 1` (deleting `i + 1` characters from `a` to get an empty string).
- For each character `cb` in `b`, we compute the minimum of three operations:
  - `curr[j] + 1`: insert `cb` (cost from the cell to the left, plus one)
  - `prev[j + 1] + 1`: delete `ca` (cost from the cell above, plus one)
  - `prev[j] + cost`: substitute (cost from the diagonal, plus 0 if characters match or 1 if they differ)
- After processing all characters of `a`, `prev[-1]` holds the final distance.

The swap `if len(a) < len(b): return _levenshtein(b, a)` ensures that `a` is always the longer string, so `prev` and `curr` are as short as possible.

The suggestion function

The builder compares the unresolved name against all candidates in the relevant symbol table, returning the closest match within a maximum distance:

```python
def _suggest(name: str, candidates: list, max_distance: int = 3) -> Optional[str]:
    """Find the closest match by Levenshtein distance."""
    best = None
    best_dist = max_distance + 1
    for c in candidates:
        d = _levenshtein(name.lower(), c.lower())
        if d < best_dist:
            best_dist = d
            best = c
    return best if best_dist <= max_distance else None
```

Several design decisions are worth noting:

- **Case-insensitive comparison.** The function compares `name.lower()` against `c.lower()`. This catches cases where the user wrote `student` instead of `Student` — a distance of 0 in case-insensitive mode, but distance 1 in case-sensitive mode.
- **Maximum distance of 3.** Beyond 3 edits, the suggestion is unlikely to be helpful. A distance of 4 between two short names (say, `"Foo"` and `"Barr"`) is meaningless. The threshold of 3 is a practical compromise: it catches common typos (transpositions, missing letters, extra letters) without suggesting wildly different names.
- **Returns the original casing.** Although comparison is case-insensitive, the function returns `c` (the candidate as it was registered), not the lowercased version. The suggestion shows the correct name exactly as it was defined.

Worked example

Suppose the file contains:

```
entity Tourist {
    *tourist_id: INT
    name: VARCHAR(100)
}

association guided_tour {
}

link Tourits (1,N) guided_tour
```

The builder processes the link, looks up `"Tourits"` in `entity_names`, and finds no match. It then calls `_suggest("Tourits", ["Tourist"])`:

- `_levenshtein("tourits", "tourist")` = 2 (two substitutions at positions 6 and 7)
- 2 <= 3 (the maximum distance), so `"Tourist"` is returned

The error message becomes:

```
schema.msd:9: error: unknown entity: 'Tourits' (did you mean 'Tourist'?)
```

Tip

The Levenshtein distance algorithm is useful far beyond DSL error messages. It appears in spell checkers, DNA sequence alignment, plagiarism detection, and fuzzy search. If you implement it once as a utility function, you will find uses for it throughout your codebase.

Programmer's Perspective

Go's `go/types.Config.Check()` is the model semantic analyser to study when building your own. It takes a parsed `*ast.Package`, resolves all identifiers to their declarations, type-checks every expression, and returns a `*types.Info` struct populated with the resolved type of every node. The errors it produces are collected into a slice rather than halting at the first failure – exactly the continue-and-collect pattern MSD uses. For "did you mean?" suggestions, Go's `gopls` language server computes edit distances between unresolved identifiers and in-scope names, surfacing quick-fix suggestions in the IDE. If you implement your DSL's semantic analyser in Go, storing errors in a `[]error` slice (or a custom diagnostic type with position and severity) and collecting all of them before returning gives callers maximum flexibility in how they present the results.

10.5 Warning vs Error Severity Decisions

The severity spectrum

Not all problems are equal. Some make the model definitively broken — a link referencing a non-existent entity cannot be resolved, period. Others indicate that something is suspicious but not necessarily wrong — an entity without a primary key might be intentional (perhaps the user plans to add one later).

MSD distinguishes between two severity levels:

- **Error**: the model is definitively wrong. The problem must be fixed before the model can be considered valid. Examples: duplicate names, unknown references, cross-namespace conflicts.
- **Warning**: the model is suspicious but not broken. The user should review the issue but may choose to ignore it. Example: an entity without a primary key.

Where to draw the line

The decision of whether something is an error or a warning is a design judgement, not a technical one. Here is MSD's classification:

Problem	Severity	Rationale
Duplicate entity name	Error	Link resolution would be ambiguous
Duplicate association name	Error	Link resolution would be ambiguous
Association name conflicts with entity	Error	Confusing and likely a mistake
Unknown entity in link	Error	Cannot create a valid link
Unknown association in link	Error	Cannot create a valid link
Entity without primary key	Warning	Legal in some modelling scenarios

The error types correspond to situations where the builder cannot produce a correct result. The warning type corresponds to a situation where the builder *can* produce a result (an entity without a primary key is still a valid `Entity` object) but the result is likely incomplete.

The design principle

When you are unsure whether something should be an error or a warning, **make it a warning**. Users can always configure their tooling to treat warnings as errors (a common `-Werror` pattern), but they cannot downgrade an error to a warning. Starting strict and relaxing is far harder than starting lenient and tightening — users grow to depend on the lenient behaviour, and promoting a warning to an error becomes a breaking change.

That said, do not make everything a warning out of timidity. If the builder genuinely cannot produce a correct result — if the link would reference a non-existent entity, if two entities would collide — that must be an error. Reserve warnings for cases where the output is technically valid but likely incomplete or surprising.

10.6 MSD Builder: Complete Walkthrough

Let us trace the builder's execution on a complete MSD file. Consider the following input:

```
project {
    name: University Enrolment
    author: Dr. H
}
```

```
entity Student {
    *student_id: INT
    name: VARCHAR(100)
    email: VARCHAR(255)
}

entity Course {
    *course_id: INT
    title: VARCHAR(200)
    credits: SMALLINT
}

association enrolment {
    enrolled_on: DATE
    grade: DECIMAL(4)
}

link Student (0,N) enrolment
link Course (0,N) enrolment
```

The parser produces a `ParseResult` containing two `ParsedEntity` objects, one `ParsedAssociation`, two `ParsedLink` objects, and a `ParsedMetadata`. Assuming no parse errors, `parse_result.errors` is empty.

Step 1: Collect existing errors

```
errors: List[MSDError] = list(parse_result.errors)
```

The builder starts by copying the parser's errors into its own list. This is important: the parser may have emitted warnings (for example, about an unknown project property), and those should propagate through to the final error list. By starting with the parser's errors, the builder ensures that the returned error list is comprehensive — it contains issues from *every* stage of processing.

Step 2: Apply metadata

```
if parse_result.metadata:
    m = parse_result.metadata
    if m.name:
        project.name = m.name
    if m.author:
        project.author = m.author
    if m.description:
        project.description = m.description
```

The builder applies the project metadata. Each field is only overwritten if the parsed value is non-empty, preserving the `Project` class's defaults (`"Untitled Project"`, etc.) for missing fields.

Step 3: Build entities

The builder iterates over `parse_result.entities`:

1. **`Student`**: not in `entity_names`, so proceed. Create an `Entity(name="Student")` — this automatically generates a UUID via the dataclass default factory. Build each `ParsedAttribute` into an `Attribute` object and add it to the entity. Check for a primary key: `student_id` is marked as primary, so `has_pk` is `True`. Register `"Student"` in `entity_names`. Add the entity to the project.
2. **`Course`**: not in `entity_names`, so proceed. Same process. UUID generated, attributes built, primary key found. Registered and added.

After this step: - `entity_names = {"Student": Entity(...), "Course": Entity(...)}` - No errors or warnings produced.

Step 4: Build associations

The builder iterates over `parse_result.associations`:

1. **`enrolment`**: not in `assoc_names` (no duplicate). Not in `entity_names` (no cross-namespace conflict). Create an `Association(name="enrolment")` with a fresh UUID. Build the carrying attributes (`enrolled_on`, `grade`). Register in `assoc_names`. Add to the project.

After this step: - `assoc_names = {"enrolment": Association(...)}` - No errors produced.

Step 5: Build links

The builder prepares candidate lists for suggestions:

```
all_entity_names = list(entity_names.keys())   # ["Student", "Course"]
all_assoc_names = list(assoc_names.keys())      # ["enrolment"]
```

Then it iterates over `parse_result.links`:

1. **`link Student (0,N) enrolment`**: look up `"Student"` in `entity_names` — found. Look up `"enrolment"` in `assoc_names` — found. Create a `Link` with `entity_id=student_entity.id`, `association_id=enrolment_assoc.id`, `cardinality_min="0"`, `cardinality_max="N"`. Add to the project.
2. **`link Course (0,N) enrolment`**: look up `"Course"` — found. Look up `"enrolment"` — found. Create and add the link.

Both links resolve successfully. No errors.

Step 6: Auto-layout

Finally, the builder runs the automatic layout algorithm on all entities and associations, positioning them on the canvas:

```
if all_entities or all_associations:
    auto_layout(all_entities, all_associations, layout_links)
```

The layout algorithm (covered in Chapter 12) assigns `x` and `y` coordinates to each entity and association, producing a visually sensible initial arrangement. The builder then marks the project as unmodified (`project.modified = False`) and returns it along with the (empty) error list.

The complete return

```
return project, errors
```

The caller receives a fully populated `Project` containing two entities, one association, and two links — all connected by UUIDs, all with layout coordinates, and all validated. The error list is empty, indicating a clean build.

Had there been errors — say, a duplicate entity or an unresolved link — the project would still be returned with whatever could be built successfully. The error list would contain the problems, and the caller (whether CLI or GUI) would decide how to present them.

Info

The builder's "always return a project" policy is a deliberate design choice. In a GUI context, showing a partially valid model with errors highlighted is far more useful than showing nothing. In a CLI context, the caller can check `len(errors) > 0` and exit with a non-zero status code if strict validation is required.

10.7 Exercises

Exercise 10.1 — Levenshtein distance by hand. Compute the Levenshtein distance between `"Tourist"` and `"Tourits"` by hand. Draw the full (8 x 8) dynamic programming matrix, filling in each cell with the minimum cost. Verify that the final cell contains the value 2. Identify which two operations (insertion, deletion, or substitution) are performed along the optimal path.

Exercise 10.2 — "Did you mean?" for data types. Implement a "did you mean?" system for data type names in the parser. If a user writes `entity Foo { name: STRIG }`, the error message should suggest `VARCHAR` or `TEXT` or whichever valid type is closest. What maximum Levenshtein distance would you use for data type names, given that most type names are short (3–9 characters)? Justify your choice.

Exercise 10.3 — Single-pass semantic analysis. MSD allows forward references because the builder processes entities and associations before links, regardless of source order. Design a semantic analyser that does *not* allow forward references — one that processes constructs in the order they appear in the file, in a single pass. What new error messages would you need? How would the user documentation change? Give an example of an MSD file that is valid under the current builder but would be rejected by your single-pass analyser.

Exercise 10.4 — Association connectivity rule. Add a new semantic rule to the builder: every association must be connected by at least two links. After all links have been built, iterate over the associations and check that each one has at least two links referencing it. Should this be an error or a warning? Implement the validation and write three test cases: one that passes, one that fails with zero links, and one that fails with exactly one link.

Exercise 10.5 — Duplicate attribute detection. MSD currently does not check for duplicate attribute names within an entity or association. Add this check to the builder. When a duplicate is found, report an error including the name of the owning entity or association, the duplicate attribute name, and the line number. Should the builder skip the duplicate attribute or keep both? Argue your design choice.

Exercise 10.6 — Orphan entity detection. An orphan entity is one that is not connected to any association via a link. Add a post-build check that reports orphan entities. Should this be an error or a warning? Consider the user experience: a student building a model incrementally will often have orphan entities during development. How does this affect your severity choice?

Exercise 10.7 — Bidirectional conflict detection. MSD currently checks whether an association name conflicts with an entity name, but not the reverse — if an entity is defined *after* an association with the same name, the entity would be registered first (because entities are processed before associations) and the association would be caught by the cross-namespace check. Convince yourself that this is correct by tracing the builder's execution with a file where both `entity Booking { ... }` and `association Booking { }` are defined. Then consider: what would happen if the builder processed associations before entities? Would the conflict still be detected? Why or why not?

Chapter 11: Output Generation

A parser extracts structure from text. A semantic analyser validates that structure and resolves references. But neither produces anything a user can *use*. The final stage of a DSL pipeline is **output generation** — transforming the validated semantic model into a target format that downstream tools can consume. This chapter examines the principles and techniques of output generation, using MSD's production of `.merisio` JSON files as our running case study.

11.1 Transforming the Intermediate Representation

After parsing and semantic analysis, the pipeline holds a validated intermediate representation — a structured, in-memory model of everything the user declared. The output generator's job is to convert this model into the target format. Different DSLs have different targets:

- **SQL DSLs** generate DDL statements (`CREATE TABLE`, `ALTER TABLE`, `ADD CONSTRAINT`).
- **SASS/LESS** generate CSS stylesheets.
- **Terraform HCL** generates execution plans and API calls to cloud providers.
- **MSD** generates `.merisio` JSON files that the Merisio application can load.

Despite these differences, the underlying principle is the same: **the output generator should be a thin layer over the semantic model**. It should not perform validation, resolve references, or make design decisions — all of that belongs in earlier pipeline stages. The output generator's sole responsibility is serialisation: walking the model and writing it in the target format.

This separation of concerns is not merely aesthetic. It has practical consequences:

- **Testability**: the model can be inspected and tested independently of its serialised form.
- **Multiple outputs**: the same model can be serialised to JSON, SQL, SVG, or any other format by swapping the output generator.
- **Stability**: changes to the output format do not require changes to the parser or semantic analyser.

In MSD, the builder (Chapter 10) validates the parse result and produces a standard `Project` object. The output generator then calls `FileIO.save_project()` to write JSON. The builder never knows or cares what format the output will take. The serialiser never knows or cares how the model was constructed.

11.2 UUID Generation for Persistent Identity

Every element in a Merisio project — every entity, every association, every link — needs a unique identifier. These identifiers serve as stable references: a link connects an entity to an association by referencing their IDs, not their names. Names can be renamed; IDs persist.

Why UUIDs?

MSD uses UUIDs (Universally Unique Identifiers) rather than sequential integers. The choice is deliberate:

- **No central counter needed.** Sequential IDs require a shared counter — who assigns ID 1? ID 2? With UUIDs, every object generates its own identifier independently.
- **Merge-friendly.** If two users create models independently and merge them, sequential IDs would collide. UUIDs are statistically unique across all models, on all machines, for all time.
- **Stable across re-imports.** When a `.merisio` file is saved and reloaded, the IDs are preserved exactly. The `from_dict()` class methods on `Entity`, `Association`, and `Link` all restore the original UUID from the JSON data.

How UUIDs Appear

In MSD's implementation, UUIDs are generated by Python's `uuid` module via dataclass default factories. Here is the pattern, drawn directly from the `Entity` class:

```
from dataclasses import dataclass, field
import uuid

@dataclass
class Entity:
    name: str
    attributes: list = field(default_factory=list)
    x: float = 0.0
    y: float = 0.0
    id: str = field(default_factory=lambda: str(uuid.uuid4()))
```

The `field(default_factory=lambda: str(uuid.uuid4()))` declaration means that every time an `Entity` is created *without* an explicit `id` argument, a fresh UUID is generated automatically. The builder never calls `uuid.uuid4()` directly — it simply writes `Entity(name=pe.name)` and the dataclass machinery handles the rest. The same pattern applies to `Association` and `Link`.

The Determinism Concern

UUIDs are random. Re-parsing the same MSD source file produces different UUIDs each time, because new `Entity` and `Association` objects are created with fresh random IDs. This means two `.merisio` files generated from the same `.msd` source will not be byte-identical.

This is acceptable for two reasons:

1. **The `.merisio` file preserves IDs once saved.** After the first save, the JSON contains explicit IDs. Reloading and re-saving preserves them.
2. **The semantic content is identical.** The entity names, attributes, links, and cardinalities are the same — only the internal identifiers differ. For version control purposes, the `.msd` source file is the canonical artefact, not the generated `.merisio` output.

Note

If deterministic output were required — for example, if the generated file needed to be committed to version control — the UUIDs could be derived from a hash of the entity name plus a project-level seed. MSD deliberately avoids this complexity because the `.msd` source, not the `.merisio` output, is the version-controlled artefact.

Programmer's Perspective

Code generation from an AST is one of the most powerful applications of DSL design, and Go's `text/template` package is the tool of choice for textual output. When your DSL needs to generate SQL, YAML, or configuration files, you define a Go template with placeholders like `{{.EntityName}}` and execute it against your semantic model. For structured output like JSON, Go's `encoding/json` package with `json.Marshal` handles serialisation directly from Go structs, including nested types and custom `MarshalJSON` methods. For visual output like SVG diagrams, you can either use `text/template` with SVG markup or Go's `image` and `image/draw` packages for raster rendering. The visitor pattern enables multi-target output: define an `OutputVisitor` interface with methods like `VisitEntity` and `VisitAssociation`, then implement one visitor per target format (SQL, JSON, SVG).

11.3 Metadata Application and Defaults

The `project {}` block in an MSD file provides optional metadata: a project name, an author, and a description. The builder applies this metadata to the `Project` object during construction:

```
# Apply metadata
if parse_result.metadata:
    m = parse_result.metadata
    if m.name:
        project.name = m.name
    if m.author:
        project.author = m.author
    if m.description:
        project.description = m.description
```

Several details are worth noting:

- **Metadata is optional.** If the MSD file contains no `project {}` block, `parse_result.metadata` is `None`, and the `Project` retains its defaults (`"Untitled Project"` for the name, empty strings for author and description).
- **Empty values are not applied.** If the user writes `name:` with no value, the parsed metadata will have `name = ""`. The `if m.name:` guard prevents this empty string from overwriting the default. This preserves the principle of least surprise: omitting a field and leaving it blank have the same effect.
- **The builder owns the application logic.** The output generator does not need to understand metadata semantics — it simply serialises whatever the `Project` object contains.

This pattern — *if present and non-empty, apply; otherwise, preserve defaults* — is common across DSL output pipelines. It keeps the metadata system additive: users can provide as much or as little metadata as they wish, and the output is always valid.

11.4 JSON Serialisation for Tool Interoperability

MSD's target output is the `.merisio` file format — a JSON document that the Merisio application can load directly. The choice of JSON as the serialisation format was not arbitrary.

Why JSON?

- **Universal.** Every programming language has a JSON parser. The `.merisio` file can be read by Python, Go, JavaScript, or any other language.
- **Human-readable.** A developer can open a `.merisio` file in a text editor and inspect its contents. This is invaluable for debugging.
- **Version-controllable.** JSON diffs are meaningful in Git. Adding an entity or changing a cardinality produces a clean, reviewable diff.
- **Schema-flexible.** New fields can be added without breaking older readers (they simply ignore unknown keys). This supports forward compatibility as Merisio evolves.

The .merisio Structure

The `.merisio` format stores the complete project state. The `Project.to_dict()` method produces the following structure:

```
def to_dict(self) -> dict:
    self.modified_at = datetime.now().isoformat()
    return {
        "version": self.VERSION,
        "metadata": {
            "name": self.name,
            "description": self.description,
            "author": self.author,
            "created_at": self.created_at,
            "modified_at": self.modified_at
        },
        "mcd": {
            "entities": [e.to_dict() for e in self._entities.values()],
            "associations": [a.to_dict() for a in self._associations.values()],
            "links": [l.to_dict() for l in self._links.values()]
        },
        "mld": {
            "column_overrides": self._mld_column_overrides
        },
        "colors": self.colors
    }
```

And `FileIO.save_project()` writes it as formatted JSON:

```
@staticmethod
def save_project(project: Project, file_path: str) -> bool:
    try:
        data = project.to_dict()
        with open(file_path, 'w', encoding='utf-8') as f:
            json.dump(data, f, indent=2, ensure_ascii=False)
        project.file_path = file_path
        project.modified = False
        return True
    except Exception as e:
        print(f"Error saving project: {e}")
        return False
```

Reusing Existing Serialisation

A key benefit of building a standard `Project` object is that the MSD pipeline reuses Merisio's existing serialisation code unchanged. The builder produces a `Project`; `FileIO.save_project()` writes it to disk. No special MSD-specific serialiser is needed. This is a significant advantage: every downstream tool that works with `.merisio` files — the GUI, the CLI, the SQL generator, the diagram exporter — works automatically

with MSD-generated projects.

Tip

When designing an output generator for your DSL, aim to produce the same model object that the rest of your toolchain already uses. If your GUI application has a `Project` class with a `save()` method, make your DSL builder produce a `Project`. You get serialisation, loading, validation, and all downstream transformations for free.

Programmer's Perspective

Go's `encoding/json` package with struct tags provides an elegant solution for DSL output serialisation. Define your model structs with `json:"field_name"` tags, and `json.MarshalIndent` produces human-readable JSON with no additional code. For more complex output formats, Go's `text/template` package supports template inheritance, custom functions, and pipeline composition – enabling you to generate SQL DDL, HTML documentation, or configuration files from the same semantic model by swapping templates. The key architectural insight is the same one Go's standard library follows throughout: separate the data model (structs) from the serialisation format (encoding packages), so that adding a new output target never requires changing the core DSL pipeline.

11.5 Round-Trip Fidelity

A critical quality metric for any DSL output pipeline is **round-trip fidelity**: the ability to parse a source file, generate the output, reload that output, and verify that no information was lost.

The Round-Trip Test

For MSD, the round-trip is:

1. **Parse** an `.msd` file into a `ParseResult`.
2. **Build** a `Project` from the `ParseResult`.
3. **Save** the `Project` as a `.merisio` JSON file.
4. **Load** the `.merisio` file back into a new `Project`.
5. **Verify** that the loaded project matches the original.

What Is Preserved

The round-trip preserves all *semantic* content:

- **Entity names** and their attributes (names, data types, sizes, primary key status).

- **Association names** and their carrying attributes.
- **Link connections** (which entity connects to which association) and their cardinalities.
- **Metadata** (project name, author, description).
- **Positions** (x, y coordinates from auto-layout).

What Is NOT Preserved

Certain source-level concerns are deliberately lost:

- **Comments.** MSD supports `#` and `//` comments, but these are discarded by the lexer and never reach the model. The `.merisio` format has no comment storage.
- **Whitespace and formatting.** The original indentation, blank lines, and spacing are not part of the semantic model.
- **Original ordering.** Entities may appear in a different order in the JSON than they did in the MSD source, because the `Project` stores them in a dictionary keyed by UUID.
- **Source location information.** Line numbers and column offsets exist only during parsing and building — they are not serialised.

These losses are acceptable because the `.msd` source file remains available. The `.merisio` output is a *derived artefact*, not the source of truth. If the user needs comments or specific ordering, they maintain the `.msd` file and regenerate the `.merisio` as needed.

Testing Round-Trip Fidelity

A round-trip test follows a straightforward pattern:

```python
def test_round_trip(msd_source: str, tmp_path):
    # Parse and build
    parser = MSDParser()
    result = parser.parse(msd_source)
    builder = MSDProjectBuilder()
    project, errors = builder.build(result)
    assert not any(e.severity == "error" for e in errors)

    # Save
    out_path = str(tmp_path / "test.merisio")
    FileIO.save_project(project, out_path)

    # Reload
    loaded = FileIO.load_project(out_path)
    assert loaded is not None
```

```python
    # Verify entities
    orig_entities = project.get_all_entities()
    loaded_entities = loaded.get_all_entities()
    assert len(orig_entities) == len(loaded_entities)

    for orig in orig_entities:
        match = loaded.get_entity_by_name(orig.name)
        assert match is not None
        assert len(match.attributes) == len(orig.attributes)
        for oa, la in zip(orig.attributes, match.attributes):
            assert oa.name == la.name
            assert oa.data_type == la.data_type
            assert oa.size == la.size
            assert oa.is_primary_key == la.is_primary_key
```

If any assertion fails, the pipeline has a bug — either something was lost during serialisation or corrupted during deserialisation. Round-trip tests are among the most valuable tests in a DSL implementation because they exercise the entire pipeline end to end.

11.6 Output Formats for Other DSLs

While MSD generates JSON, other DSLs target radically different output formats. The general pattern remains the same — *DSL source to parsed model to validated model to target format* — but the serialisation concerns vary.

SQL DSLs

A DSL that describes database schemas (MSD itself is one) might generate PostgreSQL DDL:

```sql
CREATE TABLE student (
    student_id INT,
    name VARCHAR(100),
    email VARCHAR(255),
    PRIMARY KEY (student_id)
);
```

The output generator walks each entity, maps attribute types to SQL types, identifies primary keys, and emits `CREATE TABLE` statements. Foreign keys are derived from link cardinalities. Merisio's `SQLGenerator` class demonstrates this pattern — it takes a `Project` and produces a complete DDL script.

Configuration DSLs

A DSL for application configuration might output YAML, TOML, or environment variables:

```yaml
database:
  host: localhost
  port: 5432
  name: university
```

Here, the output generator must handle nesting, quoting rules, and format-specific escaping. The semantic model is flat (key-value pairs with optional grouping), but the output format has its own syntax.

Diagram DSLs

A DSL for diagrams might output SVG, PNG, or PDF. Merisio's `HeadlessRenderer` demonstrates this: it creates a `QGraphicsScene`, adds `EntityItem` and `AssociationItem` widgets using the same paint logic as the GUI, and exports the scene to the target format. The semantic model is the same `Project` object — only the output generator differs.

Query DSLs

A DSL for queries might output API calls, SQL subqueries, or execution plans. The output is not a file but a runtime action. The "serialisation" is the translation from the declarative query model to the imperative operations the runtime must perform.

The Common Pattern

Regardless of the target format, the pattern holds:

```
DSL source → Lex → Parse → Validate → Build Model → Serialise to Target
```

Each stage has a single responsibility. The serialiser knows the target format's syntax and conventions. The model knows the domain semantics. Neither intrudes on the other's territory.

11.7 MSD: Building a Merisio Project from ParseResult

Let us walk through the complete MSD output pipeline, from parsed tokens to saved JSON.

Step 1: Create the Project Object

The builder begins with a fresh `Project`:

```
project = Project()
```

This creates an empty project with default metadata (`"Untitled Project"`), empty entity/association/link dictionaries, default diagram colours, and timestamps set to the current time.

Step 2: Apply Metadata

If the parse result contains a `project {}` block, its values are applied:

```
if parse_result.metadata:
    m = parse_result.metadata
    if m.name:
        project.name = m.name
    if m.author:
        project.author = m.author
    if m.description:
        project.description = m.description
```

Step 3: Create Entities with Attributes

For each parsed entity, the builder creates an `Entity` object (which automatically receives a UUID) and populates its attributes:

```
for pe in parse_result.entities:
    entity = Entity(name=pe.name)
    for pa in pe.attributes:
        attr = Attribute(
            name=pa.name,
            data_type=pa.data_type,
            size=pa.size,
            is_primary_key=pa.is_primary_key,
        )
        entity.add_attribute(attr)
    project.add_entity(entity)
```

Step 4: Create Associations

Associations follow the same pattern. They too receive automatic UUIDs and may carry their own attributes:

```
for pa in parse_result.associations:
    assoc = Association(name=pa.name)
    for attr_parsed in pa.attributes:
        attr = Attribute(
            name=attr_parsed.name,
            data_type=attr_parsed.data_type,
            size=attr_parsed.size,
            is_primary_key=attr_parsed.is_primary_key,
        )
        assoc.add_attribute(attr)
    project.add_association(assoc)
```

Step 5: Create Links with Resolved References

Links are where name resolution meets output generation. The builder has already constructed dictionaries mapping entity names to `Entity` objects and association names to `Association` objects. Each link resolves its text-based references to UUID-based references:

```
entity = entity_names.get(pl.entity_name)
assoc = assoc_names.get(pl.association_name)

link = Link(
    entity_id=entity.id,
    association_id=assoc.id,
    cardinality_min=pl.cardinality_min,
    cardinality_max=pl.cardinality_max,
)
project.add_link(link)
```

The link stores `entity.id` and `assoc.id` — the UUIDs — not the names. This is the critical transformation: the output format uses identity-based references, while the source format uses name-based references. The builder bridges the two.

Step 6: Auto-Layout

The builder invokes the Fruchterman-Reingold layout algorithm (Chapter 12) to assign x, y positions to all entities and associations. This ensures that the diagram will render without overlapping nodes when opened in the Merisio GUI.

Step 7: Mark as Clean and Save

Finally, the builder marks the project as unmodified and returns it:

```
project.modified = False
return project, errors
```

The caller — whether the CLI tool or the GUI's import function — then saves the project:

```
FileIO.save_project(project, "university.merisio")
```

The Resulting JSON

For an MSD file defining a university enrolment model, the saved `.merisio` file looks like this:

```
{
  "version": "2.1",
  "metadata": {
    "name": "University Enrolment",
    "description": "Student course enrolment system",
    "author": "Dr. H",
```

```
    "created_at": "2025-04-15T10:30:00.000000",
    "modified_at": "2025-04-15T10:30:00.000000"
  },
  "mcd": {
    "entities": [
      {
        "id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
        "name": "Student",
        "attributes": [
          {"name": "student_id", "type": "INT", "size": null, "pk": true},
          {"name": "name", "type": "VARCHAR", "size": 100, "pk": false},
          {"name": "email", "type": "VARCHAR", "size": 255, "pk": false}
        ],
        "x": -120,
        "y": 45
      },
      {
        "id": "b2c3d4e5-f6a7-8901-bcde-f12345678901",
        "name": "Course",
        "attributes": [
          {"name": "course_id", "type": "INT", "size": null, "pk": true},
          {"name": "title", "type": "VARCHAR", "size": 200, "pk": false},
          {"name": "credits", "type": "SMALLINT", "size": null, "pk": false}
        ],
        "x": 130,
        "y": 45
      }
    ],
    "associations": [
      {
        "id": "c3d4e5f6-a7b8-9012-cdef-123456789012",
        "name": "enrolment",
        "attributes": [
          {"name": "enrolled_on", "type": "DATE", "size": null, "pk": false},
          {"name": "grade", "type": "DECIMAL", "size": 4, "pk": false}
        ],
        "x": 5,
        "y": -80
      }
    ],
    "links": [
      {
        "id": "d4e5f6a7-b8c9-0123-defa-234567890123",
        "entity_id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
        "association_id": "c3d4e5f6-a7b8-9012-cdef-123456789012",
        "card_min": "0",
        "card_max": "N"
      },
```

```
      {
        "id": "e5f6a7b8-c9d0-1234-efab-345678901234",
        "entity_id": "b2c3d4e5-f6a7-8901-bcde-f12345678901",
        "association_id": "c3d4e5f6-a7b8-9012-cdef-123456789012",
        "card_min": "0",
        "card_max": "N"
      }
    ]
  },
  "mld": {
    "column_overrides": {}
  },
  "colors": {
    "entity_fill": "#E3F2FD",
    "entity_border": "#1976D2",
    "association_fill": "#FFF3E0",
    "association_border": "#F57C00",
    "link_color": "#000000"
  }
}
```


Every piece of information from the MSD source is present: entity names, attribute definitions, cardinalities, and metadata. The auto-layout has assigned positions. The colour defaults are included. The MLD column overrides section is empty because MSD does not define overrides — but the field exists so that the GUI can add them later.

This JSON file can be opened in the Merisio GUI, processed by `merisio-cli`, or consumed by any tool that reads JSON. The MSD pipeline has produced a standard project file, fully compatible with the existing toolchain.

Warning

When designing your output format, always include a version field (here, `"version": "2.1"`). Without it, future changes to the format become breaking changes with no way to detect or migrate older files. The version field is cheap to add and invaluable when you need to evolve the format.

11.8 Exercises

Exercise 11.1 — PostgreSQL DDL from MSD. Design an output generator that produces PostgreSQL DDL from an MSD model. For each entity, generate a `CREATE TABLE` statement. Map MSD data types to PostgreSQL types (e.g., `VARCHAR(100)` stays as-is, `INT` becomes `INTEGER`). Handle primary key constraints. For each association with two `(0,N)` links, generate a junction table with composite foreign keys. Write the generator as a function that takes a `Project` and returns a string.

Exercise 11.2 — UUIDs vs Deterministic Hashes. MSD uses random UUIDs rather than deterministic hashes of entity names. What are the trade-offs? Consider: what happens if the user renames an entity — does the UUID change? What if two entities in different projects share the same name — should they have the same UUID? What if the DSL needs to support incremental updates to an existing `.merisio` file? Write a paragraph arguing for each approach, then state which you would choose and why.

Exercise 11.3 — Round-Trip Test. Implement a round-trip test for MSD. Write an MSD source string containing at least two entities, one association with carrying attributes, and two links. Parse it, build a project, save it to a temporary file, reload it, and verify that: (a) entity names and counts match, (b) attribute names, types, sizes, and primary key flags match, (c) link cardinalities match, and (d) metadata matches. If any assertion fails, explain what the failure reveals about the pipeline.

Exercise 11.4 — Output Format Design. Design an output format for a hypothetical recipe DSL. The DSL describes recipes with ingredients (name, quantity, unit), steps (ordered instructions), and metadata (servings, preparation time, difficulty). Would you use JSON, YAML, or a custom format? Justify your choice. Sketch the output structure and explain how each DSL concept maps to a field in the output.

Exercise 11.5 — Multiple Output Targets. MSD currently outputs `.merisio` JSON. Suppose you wanted to add a second output target: a Markdown document listing all entities, their attributes, and their relationships in a human-readable table format. Design the output format (show an example of the generated Markdown). Would you modify the builder, add a new output generator, or both? Explain the separation of concerns.

Exercise 11.6 — Metadata Evolution. The `project {}` block currently supports three fields: `name`, `author`, and `description`. Propose three additional metadata fields that would be useful for a database modelling DSL (e.g., `database_engine`, `schema_name`, `version`). For each field, explain: (a) what default value it should have if omitted, (b) whether it should be validated (and what valid values look like), and (c) how it would affect the output — does it change the JSON structure, the SQL generation, or both?

Chapter 12: Automatic Graph Layout

The previous chapters built a complete pipeline: a lexer that tokenises MSD source files, a parser that assembles tokens into structured representations, a semantic analyser that validates meaning, and an output generator that serialises the result as JSON. But the output is not yet complete. An MSD file describes entities, associations, and links — the logical structure of a conceptual data model. It says nothing about where those elements should appear on screen. When a user opens the generated `.merisio` file in Merisio's visual editor, the entities and associations need positions — x and y coordinates that produce a readable, well-organised diagram. This chapter addresses the problem of computing those positions automatically.

12.1 The Graph Drawing Problem

A text-based DSL is a compact, declarative way to describe structure. But structure alone is not enough when the output is visual. Consider this MSD fragment:

```
entity Student {
    *student_id: INT
    name: VARCHAR(100)
}

entity Course {
    *course_id: INT
    title: VARCHAR(200)
}

association Enrol {
    grade: DECIMAL(4,2)
}

link Student (0,N) Enrol
link Course (1,N) Enrol
```

The parser produces two entities, one association, and two links. But where should `Student` appear relative to `Course`? Should `Enrol` sit between them, above them, or off to the side? The MSD source contains no position information whatsoever — and it should not. Positions are a visual concern, not a logical one. Forcing the user to specify coordinates in a text file would defeat the purpose of having a concise, readable language.

This means the tool must compute positions automatically. The requirements for a

good automatic layout are:

1. **Proximity**: connected elements should be near each other. `Student` and `Course` are both linked to `Enrol`, so all three should form a visual cluster.
2. **No overlaps**: nodes must not sit on top of one another. Overlapping entities make labels unreadable and the diagram useless.
3. **Visual balance**: the diagram should be roughly centred and evenly distributed, not bunched into one corner.
4. **Determinism**: the same input must always produce the same output. A user who runs the MSD compiler twice on the same file should get identical coordinates both times.

This problem is not unique to MSD. Any DSL that generates visual output faces it:

- **GraphViz** (DOT language) — arranges arbitrary directed and undirected graphs
- **Network topology DSLs** — position routers, switches, and links
- **State machine DSLs** — arrange states and transitions
- **Entity-relationship DSLs** — position entities and relationships (our case)

The field of automatic graph drawing has been studied extensively since the 1960s. Dozens of algorithms exist, each suited to different graph types and aesthetic criteria. For the kind of undirected, moderately sized graphs that MSD produces, **force-directed algorithms** are the natural choice.

12.2 Force-Directed Algorithms: The Physical Analogy

Force-directed algorithms model a graph as a physical system. The analogy is intuitive: imagine each node as a small charged particle, and each edge as a spring connecting two particles.

- **Charged particles repel each other.** This is the force that prevents overlap and clutter. Every pair of nodes pushes apart, regardless of whether they are connected. Without repulsion, unrelated nodes would drift together and overlap.
- **Springs attract connected nodes.** This is the force that keeps related elements close. An edge between two nodes acts like a spring — it pulls them together if they drift too far apart, and the further apart they are, the stronger the pull.

The algorithm simulates this physical system over many iterations. At each step, it computes the net force on every node (the sum of all repulsive and attractive forces acting on it), then moves each node a small distance in the direction of its net force. Over time, the system reaches **equilibrium** — a state where the forces approximately balance and nodes stop moving. The resulting positions form the layout.

Info

The physical analogy is not just a metaphor — it is the actual mathematical model. The repulsive force follows an inverse-distance law (like electrostatic repulsion), and the attractive force follows a quadratic law (like Hooke's law for springs). The algorithm literally simulates physics.

Programmer's Perspective

Layout algorithms in DSL tools solve the same problem as CSS flexbox and grid layout in web browsers: positioning elements according to constraints without manual coordinate specification. Go's `image` package provides the coordinate system and drawing primitives for implementing layout engines, whilst SVG output can be generated via `text/template` with computed x/y coordinates injected into the template. For production-quality graph layout, the `gonum` library (`gonum.org/v1/gonum/graph`) provides graph data structures and algorithms that you can pair with a force-directed layout implementation. The constraint-based approach – where you specify relationships (proximity, non-overlap, alignment) and let the algorithm compute positions – is fundamentally declarative, mirroring the DSL's own philosophy.

This family of algorithms includes several well-known variants:

Algorithm	Year	Key Idea
Eades	1984	Logarithmic springs, inverse-square repulsion
Fruchterman-Reingold	1991	Simulated annealing with linear cooling
Kamada-Kawai	1989	Minimise energy based on graph-theoretic distances
ForceAtlas2	2014	Adaptive speed, repulsion proportional to degree

For MSD, we use **Fruchterman-Reingold** — it is simple to implement, produces good results for small to medium graphs, and its simulated annealing approach guarantees convergence.

12.3 The Fruchterman-Reingold Algorithm

Published by Thomas Fruchterman and Edward Reingold in 1991, this algorithm refines earlier force-directed methods by introducing **simulated annealing** — a tech-

nique borrowed from metallurgy where a material is heated and slowly cooled to reach a low-energy state.

The algorithm defines two forces:

Repulsive force (applied to all pairs of nodes):

$$F_{rep}(d) = \frac{k^2}{d}$$

where *d* is the distance between two nodes, and *k* is the ideal edge length. Every pair of nodes experiences this repulsion. The closer they are, the stronger the repulsive force.

Attractive force (applied only to edges):

$$F_{attr}(d) = \frac{d^2}{k}$$

where *d* is the distance between two connected nodes. The further apart connected nodes are, the stronger the attractive force pulls them back together. The quadratic growth ensures that long edges create strong attraction.

The ideal edge length *k* scales with the size of the graph:

$$k = \sqrt{\frac{\text{area}}{n}}$$

where *area* is the drawing area and *n* is the number of nodes. This ensures that the layout adapts to the graph's size — a graph with 5 nodes uses a different scale than one with 50.

Simulated annealing controls how far nodes move at each step. The algorithm maintains a **temperature** that starts high and decreases linearly over iterations. At high temperatures, nodes make large movements — this allows them to escape local minima and explore the layout space. As the temperature decreases, movements shrink, allowing the layout to converge to a stable configuration.

- Initial temperature: `t = 2k`
- Cooling per iteration: `cooling = t / iterations`
- At each step: `t = t - cooling`

After enough iterations (typically 50–100), the temperature approaches zero and nodes settle into their final positions.

12.4 Implementation Walkthrough

MSD's layout engine lives in `layout.py` — a single file of under 200 lines. It implements the Fruchterman-Reingold algorithm with overlap removal and centring as post-processing steps.

Initial Placement

Before the simulation begins, nodes need starting positions. A common choice is random placement, but this introduces non-determinism. MSD uses **circular placement with a fixed random seed**:

```
random.seed(42)
radius = k * math.sqrt(n) / 2
for i, node in enumerate(nodes):
    angle = 2 * math.pi * i / n
    node.x = radius * math.cos(angle)
    node.y = radius * math.sin(angle)
```

Arranging nodes in a circle ensures a reasonable starting configuration — no two nodes begin at the same point, and the initial spacing is even. The fixed seed (42) ensures that if any random perturbation is needed later (to break symmetry between coincident nodes), it produces the same perturbation every time.

Main Loop

The simulation runs for 100 iterations. Each iteration has five steps:

```
temp = k * 2
cooling = temp / iterations

for _ in range(iterations):
    # 1. Reset displacement arrays
    for i in range(n):
        dx[i] = 0.0
        dy[i] = 0.0

    # 2. Compute repulsive forces (all pairs)
    # 3. Compute attractive forces (edges only)
    # 4. Apply displacements clamped by temperature
    # 5. Cool: reduce temperature
    temp -= cooling
    if temp < 0.1:
        temp = 0.1
```

Let us examine each force computation in detail.

Repulsive Forces

For every pair of nodes (i, j), the algorithm computes the distance between them and applies an inverse repulsive force:

```
for i in range(n):
    for j in range(i + 1, n):
        diffx = nodes[i].x - nodes[j].x
        diffy = nodes[i].y - nodes[j].y
        dist = math.sqrt(diffx * diffx + diffy * diffy)
        if dist < 0.01:
            dist = 0.01
            diffx = random.uniform(-0.1, 0.1)
            diffy = random.uniform(-0.1, 0.1)
        force = (k * k) / dist
        fx = (diffx / dist) * force
        fy = (diffy / dist) * force
        dx[i] += fx
        dy[i] += fy
        dx[j] -= fx
        dy[j] -= fy
```

Three details deserve attention:

1. **Minimum distance of 0.01** prevents division by zero. If two nodes happen to occupy the same position, their distance would be zero and the force would be infinite. Clamping to a small positive value avoids this.
2. **Random perturbation** when nodes coincide. If the distance is below 0.01, a small random offset is applied to the direction vector. Without this, two nodes at the exact same position would have a zero direction vector — the force would have magnitude but no direction, and the nodes would never separate. The random perturbation breaks this symmetry.
3. **Newton's third law.** The force on node *i* from node *j* is equal and opposite to the force on node *j* from node *i*. By computing the force once and applying it in opposite directions (`dx[i] += fx`, `dx[j] -= fx`), we halve the number of force computations.

Attractive Forces

For each edge, the algorithm computes the distance between the two connected nodes and applies a quadratic attractive force:

```
for ei, ai in edges:
    diffx = nodes[ei].x - nodes[ai].x
    diffy = nodes[ei].y - nodes[ai].y
    dist = math.sqrt(diffx * diffx + diffy * diffy)
    if dist < 0.01:
        dist = 0.01
    force = (dist * dist) / k
```

```
    fx = (diffx / dist) * force
    fy = (diffy / dist) * force
    dx[ei] -= fx
    dy[ei] -= fy
    dx[ai] += fx
    dy[ai] += fy
```

Note the sign difference from repulsive forces: attractive forces are subtracted from the source node and added to the target node, pulling them together rather than pushing them apart.

Displacement Clamping

After computing all forces, each node's accumulated displacement is clamped by the current temperature:

```
for i in range(n):
    disp = math.sqrt(dx[i] * dx[i] + dy[i] * dy[i])
    if disp > 0:
        scale = min(disp, temp) / disp
        nodes[i].x += dx[i] * scale
        nodes[i].y += dy[i] * scale
```

The clamping formula `scale = min(disp, temp) / disp` ensures that a node never moves further than the current temperature allows in a single step. Early in the simulation, the temperature is high and nodes can make large jumps. Late in the simulation, the temperature is low and nodes make only fine adjustments. This is the essence of simulated annealing — exploration gives way to refinement.

Warning

Without displacement clamping, the algorithm may oscillate indefinitely. A node pushed far by repulsive forces would overshoot, land near another node, get pushed back, and repeat. The temperature acts as a speed limit that gradually brings the system to rest.

12.5 Post-Processing

The force-directed algorithm treats nodes as dimensionless points. But real nodes have width and height — an entity box might be 140 pixels wide and 100 pixels tall, depending on its number of attributes. Three post-processing steps address this.

Overlap Removal

After the simulation converges, some nodes may still overlap because the algorithm did not account for their physical size. An iterative overlap removal pass fixes this:

```
for _ in range(20):
    moved = False
    for i in range(n):
        wi, hi = node_sizes[i]
        for j in range(i + 1, n):
            wj, hj = node_sizes[j]
            ox = (wi + wj) / 2 + 30 - abs(nodes[i].x - nodes[j].x)
            oy = (hi + hj) / 2 + 30 - abs(nodes[i].y - nodes[j].y)
            if ox > 0 and oy > 0:
                if ox < oy:
                    shift = ox / 2 + 1
                    if nodes[i].x < nodes[j].x:
                        nodes[i].x -= shift
                        nodes[j].x += shift
                    else:
                        nodes[i].x += shift
                        nodes[j].x -= shift
                else:
                    shift = oy / 2 + 1
                    if nodes[i].y < nodes[j].y:
                        nodes[i].y -= shift
                        nodes[j].y += shift
                    else:
                        nodes[i].y += shift
                        nodes[j].y -= shift
                moved = True
    if not moved:
        break
```

The overlap test computes the overlap amount on each axis separately, using the node sizes plus a 30-pixel margin. If overlap exists on both axes simultaneously (the bounding boxes intersect), nodes are pushed apart along the axis with the least overlap — this minimises disruption to the layout.

Node sizes are estimated from the model:

- **Entity width**: 140 pixels (fixed)
- **Entity height**: 60 + 18 per attribute
- **Association width**: 120 pixels (fixed)
- **Association height**: 50 + 18 per attribute

The algorithm iterates up to 20 times but stops early if no overlaps remain. For typical MCD models, one or two passes suffice.

Centring

After overlap removal, the diagram's centroid is shifted to the origin:

```
cx = sum(nd.x for nd in nodes) / n
cy = sum(nd.y for nd in nodes) / n
```

```
for nd in nodes:
    nd.x -= cx
    nd.y -= cy
```

This ensures the diagram appears in the centre of the canvas when opened in the visual editor, rather than being offset to one side.

Rounding

Finally, all positions are rounded to integers:

```
for nd in nodes:
    nd.x = round(nd.x)
    nd.y = round(nd.y)
```

Integer positions produce cleaner JSON output (no floating-point noise like `142.00000000003`) and ensure pixel-aligned rendering in the GUI.

12.6 Determinism Through Fixed Seeds

Determinism is a critical property for any tool in a software development workflow. If the same MSD file produces a different `.merisio` file each time it is compiled, version control systems will show spurious diffs, automated tests cannot assert exact positions, and CI/CD pipelines become unreliable.

MSD's layout achieves determinism through four mechanisms:

1. **Fixed random seed (42).** The `random.seed(42)` call before initial placement ensures that any random values used during the simulation are identical across runs.
2. **Fixed iteration count (100).** The simulation always runs for exactly 100 iterations — it does not terminate based on a convergence threshold, which could vary with floating-point rounding differences across platforms.
3. **Linear cooling.** The temperature decreases by a fixed amount each iteration (`cooling = temp / iterations`), not by a multiplicative factor that could accumulate rounding errors differently.
4. **Integer rounding.** The final rounding step eliminates any residual floating-point differences between runs.

Tip

When building a DSL that generates visual output, always verify determinism in your test suite. A simple test is: run the layout twice on the same input and assert that all coordinates are identical. This catches accidental sources of non-determinism such as dictionary iteration order (in older Python versions), thread-local state, or system clock-seeded random number generators.

Programmer's Perspective

When implementing layout algorithms in Go, the `math` package provides all the trigonometric and distance functions you need, and Go's deterministic map iteration order (since Go 1.12, maps iterate in a randomised but consistent-per-runtime order) means you must be careful to sort node lists before processing if you want reproducible layouts across runs. Go's `image` package defines the `image.Point` and `image.Rectangle` types that are natural fits for node positions and bounding boxes. For SVG output, compute coordinates as `float64` values, then format them into SVG `<rect>`, `<ellipse>`, and `<line>` elements using `fmt.Sprintf` or `text/template`. The entire Fruchterman-Reingold algorithm translates cleanly into Go: the outer iteration loop, the pairwise force computations, and the temperature cooling are all straightforward arithmetic with no dependencies beyond the standard library.

The only non-deterministic element in MSD's pipeline is UUID generation (used by the builder to assign unique identifiers to entities, associations, and links). But UUIDs are generated in the builder, not the layout engine, and they do not affect position computation.

12.7 Complexity Analysis

Understanding the computational complexity of the layout algorithm helps predict its performance on models of different sizes.

Operation	Complexity
Initial placement	O(n)
Repulsive forces per iteration	$O(n^2)$
Attractive forces per iteration	O(e)
Total (100 iterations)	$O(100 \times (n^2 + e))$
Overlap removal	$O(20 \times n^2)$

The dominant cost is the repulsive force computation, which considers all pairs of nodes — $O(n^2)$ per iteration. For typical MCD models with 5 to 20 entities and associations, this means at most a few hundred pair-wise calculations per iteration, completing in well under a millisecond.

For larger models (100+ nodes), the total computation remains well under a second. The quadratic cost only becomes a genuine bottleneck for very large graphs with thousands of nodes. In that regime, the **Barnes-Hut approximation** reduces the repulsive force computation to O(n log n) by grouping distant nodes and treating

clusters as single points. However, DSL-scale diagrams rarely exceed a few dozen nodes, making this optimisation unnecessary.

For comparison, GraphViz's `neato` engine uses a similar Fruchterman-Reingold approach for moderate graphs, switching to stress majorisation (Kamada-Kawai variant) for larger ones. ForceAtlas2, used in network visualisation tools like Gephi, handles tens of thousands of nodes through adaptive speed and approximation — far beyond what a database modelling DSL requires.

12.8 MSD: Auto-Layout for Entities and Associations

With the algorithm understood, let us see how it integrates into MSD's pipeline. The builder calls `auto_layout` after constructing all model objects:

```
# Auto-layout all elements
all_entities = project.get_all_entities()
all_associations = project.get_all_associations()
all_links = project.get_all_links()

if all_entities or all_associations:
    auto_layout(all_entities, all_associations, layout_links)
```

Duck Typing via Protocol

The layout engine does not depend on MSD's specific model classes. It works with any objects that have `x`, `y`, and `name` attributes, defined via a structural `Protocol`:

```
class Positionable(Protocol):
    x: float
    y: float
    name: str
```

This decoupling means the layout engine could be reused with entirely different model classes — any DSL that produces objects with positions and names can use the same layout algorithm.

Edge Resolution

A subtle challenge arises from the difference between the parser's representation and the builder's representation. The parser produces links with **name-based references** (`entity_name="Student"`, `association_name="Enrol"`). The builder converts these into **UUID-based references** (`entity_id="a7f3..."`, `association_id="b2c8..."`). The layout engine needs to resolve edges regardless of which representation it receives.

The solution is two-fold. The layout engine checks for both attribute names:

```
ename = getattr(lnk, "entity_name", None) or getattr(lnk, "entity_id", "")
aname = getattr(lnk, "association_name", None) or getattr(lnk, "association_id", "")
```

And the builder creates lightweight proxy objects that translate UUIDs back to names:

```
class _LayoutLink:
    def __init__(self, entity_name, association_name):
        self.entity_name = entity_name
        self.association_name = association_name

layout_links = []
for lnk in all_links:
    en = id_to_name.get(lnk.entity_id, "")
    an = id_to_name.get(lnk.association_id, "")
    layout_links.append(_LayoutLink(en, an))
```

This proxy pattern keeps the layout engine independent of the model's internal ID scheme while allowing the builder to use it without restructuring its data.

Note

The layout engine indexes nodes by both `name` and `id` (if present) in its `name_to_idx` dictionary. This means it handles name-based links from the parser, UUID-based links from the model, and the proxy objects from the builder — all through the same code path. Duck typing eliminates the need for adapter classes or interface hierarchies.

12.9 Exercises

Exercise 12.1 — Implement a simple grid layout as an alternative to force-directed placement. Place entities in rows and columns, with associations positioned between the entities they connect. Compare the results with Fruchterman-Reingold for a model with 6 entities and 5 associations. Under what conditions does the grid layout produce better results? Under what conditions does it produce worse results?

Exercise 12.2 — The MSD layout uses 100 iterations. Experiment with 10, 50, 100, and 500 iterations on a model with 15 nodes. At what point do additional iterations stop improving the layout? How would you define "improvement" quantitatively — what metric would you use?

Exercise 12.3 — Modify the overlap removal algorithm to also enforce a minimum edge length of 100 pixels. If two connected nodes are closer than 100 pixels after overlap removal, push them apart to at least that distance. What additional data does the overlap removal step need? How does this interact with the force-directed result?

Exercise 12.4 — Why is the cooling schedule linear rather than exponential? Implement an exponential cooling schedule where the temperature halves each iteration

instead of decreasing by a fixed amount. Run both schedules on the same graph and compare the results. What happens to convergence behaviour? What happens to layout quality?

Exercise 12.5 — Compute by hand: for 4 nodes arranged in a square with side length 200 and k = 200, what are the repulsive forces between adjacent nodes (distance 200)? Between diagonal nodes (distance 200 x sqrt(2))? If nodes A and B are connected by an edge, what is the attractive force between them? What is the net force on node A from node B?

Exercise 12.6 — The current algorithm has $O(n^2)$ complexity for repulsive forces. Research the Barnes-Hut approximation and describe, in pseudocode, how it would reduce this to O(n log n). What data structure does it use? At what graph size would this optimisation become worthwhile for MSD?

Exercise 12.7 — The layout engine uses duck typing via Python's `Protocol` class. Rewrite the `Positionable` protocol to include optional `width` and `height` attributes. Modify the layout algorithm to use actual node dimensions (when available) during the force-directed simulation itself, not just during post-processing overlap removal. What effect does this have on layout quality?

Chapter 13: Error Handling and Recovery

A DSL that only works on perfect input is of limited value. Real users make mistakes — typos, forgotten braces, invalid types, misspelt names. The quality of a DSL implementation is measured not just by what it does with correct input, but by how gracefully it handles errors. This chapter covers the complete error handling strategy: how errors are represented, how they flow through the pipeline, and how the parser recovers from syntax errors to report multiple problems in a single pass.

13.1 The Error Model

Every error in a DSL pipeline should be a first-class data object, not an exception thrown to the caller. This allows multiple errors to be collected, sorted, filtered, and presented together.

Representing Errors with Metadata

MSD uses a single dataclass for all errors and warnings:

```
@dataclass
class MSDError:
    message: str
    line: int = 0
    column: int = 0
    filename: str = ""
    severity: str = "error"  # "error" or "warning"

    def __str__(self) -> str:
        loc = self.filename or "<string>"
        if self.line:
            loc += f":{self.line}"
        return f"{loc}: {self.severity}: {self.message}"
```

Each error carries five pieces of information:

Field	Purpose
`message`	Human-readable description of the problem
`line`	Source line number (1-based), 0 if unknown
`column`	Source column number (1-based), 0 if unknown

Field	Purpose
`filename`	Source filename, empty for string input
`severity`	`"error"` (fatal) or `"warning"` (informational)

The Compiler-Style Format

The string representation follows the convention used by GCC, Clang, and rustc:

```
schema.msd:5: error: unknown data type: 'BLOB'
<string>:3: warning: entity 'Foo' has no primary key
```

When no filename is provided (e.g. in tests), `<string>` is used as a placeholder. This format integrates naturally with text editors that can parse compiler output and jump to error locations.

Tip

Use a structured error object rather than raising exceptions for user-facing errors. A list of error objects supports multiple errors, mixed severities, and unified reporting — none of which work well with exceptions.

Programmer's Perspective

Good error messages are a DSL's user experience, and Go's error handling philosophy provides an excellent foundation. In Go, `fmt.Errorf("line %d: unknown type %q", line, typeName)` produces contextual error messages with format verbs that include the exact values causing the problem. Go 1.13's `errors.Is` and `errors.As` functions enable typed error checking, so callers can distinguish between a parse error and a semantic error programmatically. For DSL implementations in Go, define a `Diagnostic` struct with `Pos`, `Severity`, `Message`, and `Suggestions` fields, collect diagnostics into a `[]Diagnostic` slice, and return them alongside the result. This mirrors how `gopls` (Go's language server) reports diagnostics to IDEs, enabling features like inline error squiggles and quick-fix suggestions.

13.2 Error vs Warning: When to Stop vs When to Continue

The distinction between errors and warnings has real consequences for tooling:

Errors (severity = `"error"`): - The input is definitively wrong — the tool cannot produce correct output - CLI: exit code 1, no output file generated - GUI: critical dialog, import blocked - Examples: unknown type, duplicate name, unresolved reference

Warnings (severity = `"warning"`): - The input is suspicious but technically valid — the tool can still produce output

- CLI: printed to stderr, but exit code 0 and output file produced
- GUI: warning dialog, import proceeds
- Examples: entity without primary key, size parameter on an unsized type, unknown project property

The `ParseResult.has_errors` property checks for fatal errors only:

```
@property
def has_errors(self) -> bool:
    return any(e.severity == "error" for e in self.errors)
```

Note

When in doubt, make it a warning rather than an error. Users can always choose to treat warnings as errors (e.g. with a `--strict` flag), but they cannot downgrade errors to warnings. Starting permissive gives the tool designer flexibility.

Design Guideline

A useful heuristic: if the tool can still produce *some* meaningful output despite the issue, it should be a warning. If the output would be wrong or incomplete, it should be an error.

13.3 Error Flow Through a Multi-Stage Pipeline

MSD's pipeline has three error-producing stages. Errors accumulate as data flows through:

```
Lexer
  │ errors: [lexer errors]
  ▼
Parser
  │ errors: [lexer errors] + [parser errors]
  ▼
Builder
  │ errors: [lexer errors] + [parser errors] + [builder errors]
  ▼
CLI/GUI
  │ reports all errors to user
```

Each stage copies the error list from the previous stage and appends its own:

- **Lexer** returns `(tokens, errors)`
- **Parser** starts with `self._result.errors.extend(lex_errors)`, then appends parser errors
- **Builder** starts with `errors = list(parse_result.errors)`, then appends builder errors

The final consumer — CLI or GUI — receives a single, comprehensive error list from all stages. One call to `builder.build()` returns every diagnostic from every stage.

This design has a subtle benefit: the builder copies the error list (`list(parse_result.errors)`) rather than mutating the original. The `ParseResult` remains immutable after parsing, which is important for testability.

13.4 Panic-Mode Recovery

Without error recovery, a parser stops at the first error:

```
entity A {
    x: BLOB       # Error — parser stops
}
entity B {
    *id: INT      # Never parsed
}
```

With panic-mode recovery, the parser reports the error and continues:

```
schema.msd:2: error: unknown data type: 'BLOB'
```

Entity A is created (with just its valid attributes), and entity B is parsed normally.

How Panic Mode Works

1. The parser encounters an unexpected token
2. It records an `MSDError` with the error details
3. It raises `_ParsePanic` — a lightweight internal exception
4. The nearest `try/except _ParsePanic` handler catches it
5. A recovery method skips tokens until a **synchronisation point**
6. Parsing resumes from the synchronisation point

Synchronisation Points

The parser defines two sets of synchronisation points for different contexts:

Top-level recovery — used when an entire declaration is malformed: - Top-level keywords: `entity`, `association`, `link`, `project` - Closing braces: `}`

```python
def _recover_to_top_level(self):
    while not self._at_end():
        tok = self._peek()
        if tok.type in (TokenType.ENTITY, TokenType.ASSOCIATION,
                        TokenType.LINK, TokenType.PROJECT):
            return
        if tok.type == TokenType.RBRACE:
            self._advance()
            return
        self._advance()
```

Attribute-level recovery — used when a single attribute fails within a block: - Closing braces: `}` - Top-level keywords - Star: `*` (start of a primary key attribute) - Identifier followed by colon (start of a non-PK attribute)

```
def _recover_to_brace_or_keyword(self):
    while not self._at_end():
        tok = self._peek()
        if tok.type == TokenType.RBRACE:
            return
        if tok.type in (TokenType.ENTITY, TokenType.ASSOCIATION,
                        TokenType.LINK, TokenType.PROJECT):
            return
        if tok.type == TokenType.STAR:
            return
        if tok.type == TokenType.IDENTIFIER and self._pos + 1 < len(self._tokens):
            next_tok = self._tokens[self._pos + 1]
            if next_tok.type == TokenType.COLON:
                return
        self._advance()
```

The attribute-level recovery is more fine-grained: if one attribute has an invalid type, the parser skips to the next attribute and continues. The entity is still created with its valid attributes.

Recovery Scopes

The parser has `try/except` blocks at two levels:

```
# Top level — in the main parsing loop
while not self._at_end():
    try:
        self._parse_top_level()
    except _ParsePanic:
        self._recover_to_top_level()

# Attribute level — inside entity/association blocks
while not self._check(TokenType.RBRACE) and not self._at_end():
    try:
        attr = self._parse_attribute()
        entity.attributes.append(attr)
    except _ParsePanic:
        self._recover_to_brace_or_keyword()
        if self._check(TokenType.RBRACE):
            break
```

13.5 Multi-Error Reporting

The combination of error accumulation and panic-mode recovery enables multi-error reporting — the user sees all problems at once:

```
schema.msd:2: error: unknown data type: 'BLOB'
schema.msd:5: error: unknown data type: 'INVALID_TYPE'
schema.msd:8: error: unknown entity: 'Unknown'
```

This is far more productive than one-at-a-time reporting. Consider a file with five errors: without multi-error reporting, the user must run the tool five times, fixing one error per run. With multi-error reporting, they fix all five in one editing session.

What Gets Preserved After Errors

When an error occurs, the parser does not discard everything:

- Entity A has an invalid attribute `x: BLOB` → Entity A is created with its remaining valid attributes
- Entity B follows Entity A → Entity B is parsed normally
- A link references an unknown entity → The link is skipped, but all other links are processed

Warning

Panic-mode recovery can occasionally produce **cascade errors** — false errors triggered by the recovery process rather than by genuine mistakes. For example, if the parser skips past a closing brace during recovery, it might misinterpret the next construct. In practice, MSD's grammar is simple enough that cascading is rare, but always review the first error before addressing subsequent ones.

Programmer's Perspective

Error recovery strategies in DSLs directly parallel techniques used in production compilers and IDE tooling. Go's parser recovers from syntax errors by scanning forward to the next statement boundary (a semicolon or a keyword like `func`, `type`, or `var`), which is exactly the synchronisation-point approach MSD uses with top-level keywords. For IDE integration via the Language Server Protocol (LSP), error recovery is essential – the language server must produce a partial AST even when the file contains errors, because the user is actively typing and the file is almost always in an invalid state. If you plan to build LSP support for your DSL, design your parser's recovery strategy from the start, ensuring it always produces a usable partial result rather than an empty failure.

13.6 Error Message Quality

Error messages are part of the user experience. A clear error message is the difference between a frustrating tool and a helpful one.

Principles

Be specific — name what went wrong:

`unknown data type: 'BLOB'`

Not: `invalid input` or `syntax error`

Include alternatives — tell the user what is valid:

`unknown data type: 'BLOB' (valid types: BIGINT, BOOLEAN, CHAR, DATE, DECIMAL, DOUBLE, FLOAT, INT, SMALLINT, TEXT, TIME, TIMESTAMP, VARCHAR)`

Suggest corrections — catch typos:

`unknown entity: 'Tourits' (did you mean 'Tourist'?)`

Show context — include both expected and actual:

`expected IDENTIFIER, got RBRACE ('}')`

Include location — filename and line number:

`schema.msd:5: error: unknown data type: 'BLOB'`

Anti-Patterns

- `"Syntax error"` — tells the user nothing
- `"Error on line 5"` — no description of what is wrong
- `"E0042"` — error codes without explanation
- `"Parse failed"` — no indication of where or why

13.7 The `_ParsePanic` Exception

```
class _ParsePanic(Exception):
    """Internal exception for panic-mode error recovery."""
    pass
```

Key properties of `_ParsePanic`:

- **Private** — the underscore prefix signals it is not part of the public API
- **Never escapes** — always caught within the parser; the caller never sees it
- **Lightweight** — no message, no payload; the error is already recorded before raising
- **Not an error in itself** — it is a control flow mechanism, not a diagnostic

The error is recorded via `self._error()` *before* the exception is raised. The exception's sole purpose is to unwind the call stack to the nearest recovery handler.

13.8 MSD: Complete Error Handling Walkthrough

Consider this MSD file with multiple errors:

```
entity A {
    x: BLOB
    name: TEXT
}
entity A {
    *id: INT
}
association voyager { }
link Tourits (0,N) voyager
```

Lexer stage: No lexical errors — all characters are valid.

Parser stage: - Line 2: `BLOB` is not a valid data type → error recorded, `_ParsePanic` raised

- Attribute-level recovery: skip to next attribute start (`name:`)
- Line 3: `name: TEXT` parsed successfully
- Entity A created with one attribute: `name: TEXT`
- Lines 4-6: second entity `A` parsed successfully

Builder stage: - First entity A: registered in `entity_names`

- Second entity A: duplicate detected → error "duplicate entity name: 'A'"
- Association `voyager`: built successfully
- Link: entity name `Tourits` not found → Levenshtein search → distance 2 from `Tourist`... but `Tourist` is not defined either. No suggestion within distance 3 → error "unknown entity: 'Tourits'"

Final error list (3 errors, 0 warnings):

```
<string>:2: error: unknown data type: 'BLOB' (valid types: ...)
<string>:5: error: duplicate entity name: 'A'
<string>:9: error: unknown entity: 'Tourits'
```

13.9 Testing Error Paths

Error handling is part of the specification and must be tested as rigorously as the happy path.

Test Categories

Category	What to verify
Unknown data types	Error message content, parsing continues
Invalid cardinalities	Both min and max validation
Missing braces	Detection and recovery
Multi-error reporting	Multiple errors found in one pass
Recovery after error	Valid constructs after errors are still parsed
Unknown references	Entity/association lookup errors
Duplicate names	Duplicate detection
Suggestions	"Did you mean?" appears for close matches
Line numbers	Errors reference the correct source line

Example Test

```
def test_recovery_after_error(self, parser):
    source = """entity A {
    x: BLOB
}
entity B {
    *id: INT
}"""
    result = parser.parse(source)
    # Entity B should be parsed despite error in A
    valid_entities = [e for e in result.entities if e.name == "B"]
    assert len(valid_entities) == 1
    assert result.has_errors  # BLOB error recorded
```

This test verifies three things: the error was detected, recovery worked, and the subsequent entity was parsed correctly.

13.10 Exercises

Exercise 13.1. Design an error recovery strategy for a DSL where statements are separated by semicolons instead of newlines. What would the synchronisation points be? How would recovery differ from MSD's approach?

Exercise 13.2. MSD uses two severity levels: `"error"` and `"warning"`. Would adding `"info"` and `"hint"` levels be valuable? Define the semantics of each level. What changes would be needed in the CLI and GUI?

Exercise 13.3. Write a test that verifies the parser reports exactly three errors for this input:

```
entity A { x: BLOB }
```

```
entity A { *id: INT }
link Unknown (0,N) missing
```

Verify each error message contains the expected substring.

Exercise 13.4. The `_ParsePanic` exception carries no information — the error is recorded before raising. What would change if the exception carried the error object as a payload? Would that be better or worse? Argue both sides.

Exercise 13.5. MSD's cascade error risk is low because the grammar is simple. Design a more complex grammar (e.g. with nested blocks or expressions) where panic-mode recovery would be more likely to produce cascade errors. Propose mitigation strategies.

Exercise 13.6. Add a `--strict` mode to the MSD CLI that treats all warnings as errors. What changes to `cmd_parse()` are needed? Should `--strict` change the severity in the error objects, or should it change how the error list is evaluated?

Exercise 13.7. Compile a catalogue of error messages from a tool you use regularly (a compiler, linter, or build tool). Rate each message on specificity, actionability, and context. Identify the three best and three worst messages, and explain what makes them effective or unhelpful.

Chapter 14: Testing DSL Implementations

A DSL implementation without tests is a liability. The input space of a language — even a small one — is vast, and the interactions between lexer, parser, and semantic analyser create subtle edge cases. This chapter describes the testing strategy for DSL implementations, using MSD's 64-test suite as the model.

14.1 Testing Philosophy

MSD's test suite follows three principles:

Test Each Layer Independently

The lexer tests do not depend on the parser. The parser tests do not depend on the builder. Each layer is tested with its own inputs and expected outputs:

```
# Lexer test — directly calls tokenize()
def test_all_keywords(self, lexer):
    tokens, errors = lexer.tokenize("project entity association link")
    types = _types(tokens)
    assert types == [TokenType.PROJECT, TokenType.ENTITY,
                     TokenType.ASSOCIATION, TokenType.LINK]

# Parser test — directly calls parse()
def test_single_entity(self, parser):
    result = parser.parse("entity Foo {\n    *id: INT\n}")
    assert not result.has_errors
    assert len(result.entities) == 1

# Builder test — parses then builds
def test_builds_project(self, parser, builder):
    result = parser.parse(SIMPLE_MSD)
    project, errors = builder.build(result)
    assert project is not None
```

Independent testing isolates bugs: if a builder test fails, you know the bug is in the builder, not the parser.

Test Both Success and Failure Paths

For every feature, there are tests for correct input *and* tests for invalid input:

```python
# Success: valid cardinalities
def test_all_cardinalities(self, parser):
    for card in ["(0,1)", "(0,N)", "(1,1)", "(1,N)"]:
        source = f"entity E {{ *id: INT }}\nassociation A {{ }}\nlink E {card} A"
        result = parser.parse(source)
        assert not result.has_errors

# Failure: invalid cardinality
def test_invalid_cardinality_min(self, parser):
    result = parser.parse("entity E { *id: INT }\nassociation A { }\nlink E (5,N) A")
    assert result.has_errors
    assert any("invalid minimum cardinality" in e.message for e in result.errors)
```

Tip

Aim for at least one positive test and one negative test per language feature. The negative tests often catch more bugs than the positive ones, because error paths are where implementations tend to be weakest.

Programmer's Perspective

Go's testing infrastructure is exceptionally well suited for DSL testing. The `testdata/` directory convention provides a natural home for golden files – expected output snapshots that your tests compare against actual output. Table-driven tests, Go's signature testing pattern, let you cover all valid token types, all cardinality combinations, and all error cases in a single test function with a `[]struct` slice. Go 1.18's fuzzing support via `testing.F` is particularly valuable for DSL parsers: `f.Fuzz(func(t *testing.T, input string) { ... })` generates random byte strings and feeds them to your parser, finding crash-inducing inputs that manual tests would never cover. Run `go test -fuzz=FuzzParse -fuzztime=30s` regularly to discover edge cases in your lexer and parser.

Test the Contract, Not the Implementation

Tests verify observable behaviour, not internal state:

```python
# Good: tests the output
def test_entities_get_positions(self, parser, builder):
    result = parser.parse(SIMPLE_MSD)
    project, _ = builder.build(result)
    positions = [(e.x, e.y) for e in project.get_all_entities()]
    assert not all(x == 0 and y == 0 for x, y in positions)

# Bad: tests internal state
# assert layout._temperature == 0.1  # implementation detail
```

If you test the contract, you can refactor the implementation freely. If you test internals, every refactoring breaks tests.

14.2 Lexer Tests

Token Extraction Helpers

Lexer tests use helper functions to filter out structural tokens:

```
def _types(tokens):
    """Extract non-NEWLINE, non-EOF token types."""
    return [t.type for t in tokens
            if t.type not in (TokenType.NEWLINE, TokenType.EOF)]

def _values(tokens):
    """Extract non-NEWLINE, non-EOF token values."""
    return [t.value for t in tokens
            if t.type not in (TokenType.NEWLINE, TokenType.EOF)]
```

These helpers let tests focus on meaningful tokens without noise from structural tokens.

What to Test: Lexer

Category	Examples
Empty input	Empty string, whitespace only
Comments	Hash, double-slash, inline after code
Keywords	All four keywords, case insensitivity
Symbols	All seven symbol tokens
Identifiers	Regular identifiers, underscores, digits
Integers	Single-digit, multi-digit
Project values	STRING_VALUE capture, comment stripping
Positions	Correct line numbers, correct columns
Invalid characters	Single error, multiple errors, recovery
Full constructs	Entity block, link statement

Notable Test: Context-Sensitive Project Values

```
def test_project_string_value_with_comment(self, lexer):
    source = "project {\n    name: My Project # a comment\n}"
    tokens, errors = lexer.tokenize(source)
    string_vals = [t.value for t in tokens if t.type == TokenType.STRING_VALUE]
    assert string_vals == ["My Project"]
```

This verifies that comments are stripped from project values — a subtle behaviour of the context-sensitive lexer that would be easy to break accidentally.

14.3 Parser Tests

What to Test: Parser

Category	Examples
Minimal constructs	Single entity, empty entity, single link
Composite PKs	Multiple * attributes
Sized types	VARCHAR(n), DECIMAL(n), CHAR(n)
Size warnings	Size on unsized type (INT(10))
Associations	Empty body, with carrying attributes
Links	All four cardinalities
Metadata	Project block present, absent, unknown property
Full file	Complete MSD with all constructs
Errors	Missing brace, invalid type, invalid cardinality
Multi-error	Multiple errors found, recovery after error

Notable Test: Error Recovery

```
def test_recovery_after_error(self, parser):
    source = """entity A {
    x: BLOB
}
entity B {
    *id: INT
}"""
    result = parser.parse(source)
    # Entity B should be parsed despite error in A
    valid_entities = [e for e in result.entities if e.name == "B"]
    assert len(valid_entities) == 1
```

This test exercises attribute-level panic-mode recovery. Entity A's invalid attribute triggers recovery, but Entity B is still parsed successfully.

Note

Error recovery tests are among the most valuable in a DSL test suite. They verify that a single mistake does not cascade into a flood of false errors, which is the most common complaint users have about compilers and linters.

Programmer's Perspective

Golden file testing and `go test -cover` form the backbone of DSL test suites in Go. For golden file tests, store expected token streams and parse results as `.golden` files in `testdata/`, then use `os.ReadFile` to load them and compare against actual output. The `go test -update` flag pattern (checking `os.Getenv("UPDATE_GOLDEN")`) lets you regenerate golden files when the output format intentionally changes. For coverage, `go test -cover -coverprofile=coverage.out` followed by `go tool cover -html=coverage.out` produces a visual coverage report showing exactly which parser branches and error paths are exercised. Aim for 100% coverage of your lexer and parser – these are pure functions with well-defined inputs and outputs, making exhaustive coverage practical rather than aspirational.

Notable Test: All Cardinalities

```
def test_all_cardinalities(self, parser):
    for min_val, max_val in [("0", "1"), ("0", "N"), ("1", "1"), ("1", "N")]:
        source = f"""entity E {{ *id: INT }}
association A {{ }}
link E ({min_val},{max_val}) A"""
        result = parser.parse(source)
        assert not result.has_errors
        assert len(result.links) == 1
        link = result.links[0]
        assert link.cardinality_min == min_val
        assert link.cardinality_max == max_val
```

Parameterised tests cover all valid combinations concisely.

14.4 Semantic Tests (Builder Tests)

What to Test: Semantics

Category	Examples
Simple build	Project creation, attribute mapping
UUIDs	Uniqueness, link reference validity
Metadata	Applied correctly, defaults when absent
Duplicate errors	Duplicate entity, duplicate association
Name conflicts	Association name matching entity name
Unknown references	Unknown entity in link, unknown association
Suggestions	"Did you mean?" for entity, for association

Category	Examples
Warnings	Entity without primary key
Auto-layout	Positions assigned, no all-zero positions
Round-trip	Save to .merisio, reload, verify

Notable Test: "Did You Mean?"

```
def test_suggestion_for_entity(self, parser, builder):
    source = """entity Tourist { *id: INT }
association R { }
link Tourits (0,N) R"""
    result = parser.parse(source)
    project, errors = builder.build(result)
    fatal = [e for e in errors if e.severity == "error"]
    assert any("did you mean 'Tourist'" in e.message for e in fatal)
```

This verifies the complete suggestion pipeline: Levenshtein distance is computed between "Tourits" and "Tourist" (distance 2, within threshold 3), and the suggestion appears in the error message.

Notable Test: Round-Trip

```
def test_save_and_load(self, parser, builder, tmp_path):
    from src.utils.file_io import FileIO

    result = parser.parse(FULL_MSD)
    project, errors = builder.build(result)
    assert not any(e.severity == "error" for e in errors)

    file_path = str(tmp_path / "test.merisio")
    assert FileIO.save_project(project, file_path) is True

    loaded = FileIO.load_project(file_path)
    assert loaded is not None
    assert len(loaded.get_all_entities()) == len(project.get_all_entities())
    assert loaded.name == "Tourism System"
```

This test verifies the complete pipeline: MSD text → parse → build → save as JSON → reload → verify. It uses pytest's `tmp_path` fixture for a clean temporary directory.

14.5 Integration Tests

Integration tests exercise the complete pipeline end-to-end, verifying that all stages work together correctly.

The Full Pipeline Test

```
def test_full_pipeline(self, tmp_path):
    parser = MSDParser()
    builder = MSDProjectBuilder()

    result = parser.parse(COMPLETE_MSD, filename="test.msd")
    assert not result.has_errors

    project, errors = builder.build(result)
    fatal = [e for e in errors if e.severity == "error"]
    assert len(fatal) == 0

    # Verify project structure
    assert project.name == "Tourism Management System"
    entities = project.get_all_entities()
    entity_names = {e.name for e in entities}
    assert entity_names == {"Tourist", "Role", "Guide", "Experience"}

    # Save and reload
    file_path = str(tmp_path / "output.merisio")
    assert FileIO.save_project(project, file_path) is True

    loaded = FileIO.load_project(file_path)
    assert loaded is not None

    # Verify JSON structure
    with open(file_path, "r", encoding="utf-8") as f:
        data = json.load(f)
    assert len(data["mcd"]["entities"]) == 4
```

Other Integration Tests

Test	What it verifies
Line numbers	Error messages include correct line numbers
Comments	Comments do not affect parsing
Case-insensitive keywords	`ENTITY`, `Entity`, `entity` all work
Case-sensitive identifiers	`Foo` and `foo` are different entities

14.6 Test Fixtures and Shared Test Data

Constant MSD Strings

Builder and integration tests share MSD strings defined at module level:

```
SIMPLE_MSD = """entity Tourist {
    *id: INT
    name: VARCHAR(255)
}
association voyager { }
link Tourist (1,N) voyager
"""

FULL_MSD = """project {
    name: Tourism System
    author: Test Author
    description: A test project
}
entity Tourist {
    *id: INT
    name: VARCHAR(255)
    email: VARCHAR(255)
}
entity Guide {
    *id: INT
    name: VARCHAR(200)
}
association accompagner {
    date_debut: DATE
}
link Tourist (0,N) accompagner
link Guide (1,N) accompagner
"""
```

Pytest Fixtures

```
@pytest.fixture
def lexer():
    return MSDLexer()

@pytest.fixture
def parser():
    return MSDParser()

@pytest.fixture
def builder():
    return MSDProjectBuilder()
```

Each test gets a fresh instance, ensuring no state leaks between tests.

Info

MSD's 64 tests run in approximately 0.06 seconds. Fast tests encourage running them frequently — after every change, not just before committing. If your test suite takes minutes, developers will skip it.

14.7 MSD: Walkthrough of the 64-Test Suite

Organisation

File	Tests	Focus
`test_msd_lexer.py`	18	Tokenisation, comments, keywords, context sensitivity
`test_msd_parser.py`	22	All constructs, error cases, multi-error recovery
`test_msd_builder.py`	19	Project building, validation, suggestions, round-trip
`test_msd_integration.py`	5	End-to-end pipeline, JSON verification

Coverage Strategy

Every token type has at least one test. Every parse rule has at least one test. Every error type has at least one test. Every validation rule has at least one test. This systematic coverage ensures that changes to any part of the pipeline will be caught by at least one test.

Running the Tests

```python
# Run all MSD tests
python -m pytest tests/test_msd_*.py -v

# Run a specific file
python -m pytest tests/test_msd_lexer.py -v

# Run a specific test class
python -m pytest tests/test_msd_parser.py::TestErrorCases -v

# Run with coverage
python -m pytest tests/test_msd_*.py --cov=src/msd
```

14.8 Exercises

Exercise 14.1. Write three lexer tests for edge cases: empty input, input with only comments, and input with only whitespace. What should the token stream contain in each case?

Exercise 14.2. Write a parser test that verifies error recovery works across three consecutive entities, where the middle one has an invalid type. Verify that the first and third entities are parsed correctly.

Exercise 14.3. Design a test for a new semantic rule: "every association must be connected by at least two links." Write both the validation code in the builder and the test.

Exercise 14.4. MSD has 64 tests for approximately 600 lines of implementation code — roughly one test per 10 lines. Is this ratio appropriate? Compare with other projects you have worked on. What is the relationship between test count and code confidence?

Exercise 14.5. The round-trip test verifies parse $\rightarrow$ build $\rightarrow$ save $\rightarrow$ load. Design a **reverse round-trip** test: load a .merisio file, generate MSD text from it, parse the text, and verify the result matches the original. What additional code would you need to implement?

Exercise 14.6. Write a property-based test (using Hypothesis or a similar library) that generates random valid MSD files and verifies that they parse without errors. What generators would you need?

Exercise 14.7. MSD's tests use constant MSD strings at module level. An alternative is to store test fixtures as separate .msd files. Compare the two approaches. When would file-based fixtures be preferable?

Chapter 15: Tool Integration

A DSL without tool integration is an academic exercise. The language pipeline — lexer, parser, builder — is a library. To be useful, it must be accessible through the interfaces that users actually work with: command-line tools for automation and scripting, graphical interfaces for visual exploration, and documentation for discoverability. This chapter covers the patterns for integrating a DSL into real-world tooling.

15.1 CLI Integration

Subcommands and Argument Parsing

A DSL CLI typically offers multiple subcommands — one per major operation:

```
merisio-cli schema.msd parse [-o output.merisio]
merisio-cli schema.merisio validate
merisio-cli schema.merisio sql -o schema.sql
merisio-cli schema.merisio mld
merisio-cli schema.merisio export --format png -o diagram.png
```

The `parse` subcommand accepts `.msd` files; the others accept `.merisio` files. The file is a positional argument; subcommand-specific options use flags.

Exit Codes

Exit codes communicate success or failure to shell scripts and CI systems:

Exit Code	Meaning	Example
0	Success	File parsed and saved
1	Validation/parse errors	Unknown type, duplicate name
2	Runtime error	File not found, write failure

Tip

Always use distinct exit codes for different failure modes. A CI pipeline that treats all non-zero codes identically cannot distinguish between “the model has errors” (fixable by the author) and “the tool crashed” (needs infrastructure attention).

Programmer's Perspective

The Language Server Protocol (LSP) is the modern standard for integrating a DSL into IDEs, and Go's `gopls` is the reference implementation to study. An LSP server for your DSL would expose your lexer as a token provider for syntax highlighting (via TextMate grammars or semantic tokens), your parser for real-time error diagnostics (red squiggles as the user types), and your semantic analyser for features like go-to-definition and hover information. In Go, building an LSP server is straightforward using libraries like `go.lsp.dev/protocol`. The key insight is that your DSL pipeline already does the heavy lifting – the LSP server is merely a thin wrapper that calls `Parse()` and `Build()` on every file change and translates the resulting error list into LSP diagnostic objects.

Error Output

Errors go to stderr; success messages go to stdout. This lets users pipe output cleanly:

```
# Errors appear on terminal; output goes to file
merisio-cli schema.msd parse > /dev/null
```

Implementation: The `cmd_parse` Function

```
def cmd_parse(args):
    from src.msd import MSDParser, MSDProjectBuilder
    from src.utils.file_io import FileIO

    file_path = args.file
    if not os.path.isfile(file_path):
        print(f"Error: file not found: {file_path}", file=sys.stderr)
        sys.exit(2)

    try:
        with open(file_path, "r", encoding="utf-8") as f:
            source = f.read()
    except OSError as e:
        print(f"Error reading file: {e}", file=sys.stderr)
        sys.exit(2)

    parser = MSDParser()
    parse_result = parser.parse(source, filename=file_path)

    builder = MSDProjectBuilder()
    project, errors = builder.build(parse_result)

    has_fatal = False
    for err in errors:
        print(str(err), file=sys.stderr)
```

```
        if err.severity == "error":
            has_fatal = True

    if has_fatal:
        print("Parse failed with errors.", file=sys.stderr)
        sys.exit(1)

    output = args.output
    if not output:
        base, _ = os.path.splitext(file_path)
        output = base + ".merisio"

    if FileIO.save_project(project, output):
        print(f"Saved project to {output}")
    else:
        print(f"Error: failed to save project to {output}", file=sys.stderr)
        sys.exit(2)
```

Key design choices:

- **Lazy imports**: `MSDParser` and `MSDProjectBuilder` are imported inside the function, not at module level. This avoids loading MSD modules when the user runs unrelated commands like `info` or `sql`.
- **Default output path**: if `-o` is not specified, the output file is the input filename with a `.merisio` extension (`school.msd` → `school.merisio`).
- **All errors printed**: even when there are fatal errors, all warnings are also shown so the user sees everything.

15.2 GUI Integration

The Import Workflow

GUI integration adds a menu item that triggers the DSL pipeline:

1. **File > Import MSD...** — placed after "Open..." in the File menu
2. A file dialog opens, filtered to `.msd` files
3. The file is read and processed through the MSD pipeline
4. Errors are presented in modal dialogs
5. On success, the model appears on the canvas with auto-layout

Error Presentation

The GUI adapts error presentation to the graphical context:

Situation	Behaviour
Fatal errors	`QMessageBox.critical` — import blocked, all errors listed
Warnings only	`QMessageBox.warning` — import proceeds, warnings listed
No errors	Silent import, status bar message

```
if fatal_errors:
    msg = "Import failed with errors:\n\n"
    msg += "\n".join(f"- {e}" for e in fatal_errors)
    if warnings:
        msg += "\n\nWarnings:\n"
        msg += "\n".join(f"- {e}" for e in warnings)
    QMessageBox.critical(self, "Import Error", msg)
    return

if warnings:
    msg = "Import succeeded with warnings:\n\n"
    msg += "\n".join(f"- {e}" for e in warnings)
    QMessageBox.warning(self, "Import Warnings", msg)
```

Post-Import Actions

After a successful import, the GUI must update all views:

```
self._project = project
self._dictionary_view.set_project(self._project)
self._mcd_canvas.set_project(self._project)
self._mcd_canvas.apply_colors(self._project.colors)
self._mld_view.set_project(self._project)
self._sql_view.set_project(self._project)
self._update_title()
self._update_status(f"Imported MSD: {file_path}")
self._mcd_canvas.zoom_fit()
```

The `zoom_fit()` call is essential: auto-layout positions may be far from the default viewport, so without it the user would see an empty canvas.

Note

The GUI and CLI follow exactly the same pipeline. The only differences are input source (file dialog vs command-line argument), error presentation (message boxes vs stderr), and success action (canvas update vs file save). This consistency is a direct result of building the DSL core as a library.

15.3 Man Pages and Documentation

Unix Convention

CLI tools on Unix should provide man pages. MSD's `merisio-cli(1)` man page documents all subcommands:

```
.SS parse
Parse an MSD (Merisio Schema Definition) file and convert it to a
\&.merisio project.
.PP
.RS
.B merisio-cli
.I file.msd
.B parse
.RB [ \-o
.IR output ]
.RE
```

Essential Documentation

At minimum, a DSL tool should provide:

- `--help` flag on every command and subcommand
- `--version` flag for the main binary
- Man page (Unix) or help file (Windows) for detailed reference
- Error message catalogue (implicitly, through good error messages)

Documentation as a First-Class Deliverable

Documentation should be treated as part of the implementation, not an afterthought. When adding a new subcommand:

1. Implement the command
2. Add tests
3. Update the man page
4. Update `--help` text

If documentation is always updated alongside code, it stays accurate.

15.4 Batch Processing and CI Pipelines

One of the primary motivations for creating a text-based DSL is enabling automation. Text files can be version-controlled, diffed, reviewed, and processed in bulk.

Batch Conversion

```
# Convert all .msd files in a directory
for f in models/*.msd; do
```

```
    merisio-cli "$f" parse -o "output/$(basename "${f%.msd}.merisio")"
done
```

CI Pipeline

```
# Parse, validate, and generate SQL in a CI pipeline
merisio-cli schema.msd parse -o /tmp/schema.merisio
merisio-cli /tmp/schema.merisio validate
merisio-cli /tmp/schema.merisio sql -o schema.sql
```

Version Control Workflows

MSD files are plain text, which enables:

- **Clean diffs**: adding an attribute is a one-line diff, not a binary blob change
- **Code review**: reviewers can read and comment on model changes in pull requests
- **CI validation**: automated parsing catches errors before merge
- **Blame**: `git blame` shows who added each entity and when

Info

This is perhaps the strongest argument for text-based DSLs. A graphical-only tool cannot participate in the code review workflow that modern software teams depend on. A text DSL turns domain models into reviewable, versionable artefacts.

Programmer's Perspective

Go's `go generate` directive is the canonical pattern for embedding DSL tools into a Go build pipeline. By adding `//go:generate merisio-cli schema.msd parse -o schema.merisio` to a Go source file, you ensure that the DSL is processed automatically whenever `go generate ./...` runs, and the generated output is checked into version control alongside the source. For CI/CD, you can lint DSL files in `go test` by writing a `TestValidateSchemas` function that walks a directory of `.msd` files and asserts that each one parses without errors. This treats your DSL files as first-class artefacts in the Go build system, with the same compile-test-deploy workflow that Go source code enjoys. Syntax highlighting for your DSL in VS Code requires only a TextMate grammar (a JSON file mapping regex patterns to token scopes) – no server-side code needed.

15.5 The Integration Pattern: Library → CLI Wrapper → GUI Wrapper

The most robust integration architecture has three layers:

```
┌──────────────────────────────┐
│          DSL Library         │
│   (lexer, parser, builder)   │
└──────────────────────────────┘
             │
             │
      ┌──────┴──────┐
      │             │
┌─────┴─────┐  ┌────┴──────┐
│   CLI     │  │   GUI     │
│ Wrapper   │  │ Wrapper   │
└───────────┘  └───────────┘
```

The library is the core: importable modules with no I/O dependencies. It reads strings, returns objects and error lists.

The CLI wrapper is a thin adapter: reads files from disk, calls the library, formats output for the terminal, sets exit codes.

The GUI wrapper is another thin adapter: opens file dialogs, calls the library, shows message boxes, updates the canvas.

Both wrappers call the same pipeline:

`Read source → MSDParser().parse() → MSDProjectBuilder().build() → Handle errors → Use proje`

The library does not know about files, terminals, or windows. This separation provides:

- **Testability**: test the library directly with string input, no I/O needed
- **Reusability**: new wrappers (web API, language server, IDE plugin) are trivial to build
- **Consistency**: every wrapper uses the same pipeline, so behaviour is identical

15.6 MSD: The `parse` Command and File > Import MSD

A side-by-side comparison shows the pattern clearly:

Aspect	CLI	GUI
Input source	`args.file` (command line)	`QFileDialog.getOpenFileName()`
Read file	`open(file_path, "r")`	`open(file_path, "r")`
Parse	`MSDParser().parse(sou filename)`	`MSDParser().parse(source, filename)`
Build	`MSDProjectBuilder().b`	`MSDProjectBuilder().build(result`

Aspect	CLI	GUI
Error output	`print(err, file=sys.stderr)`	`QMessageBox.critical()`
Warning output	`print(err, file=sys.stderr)`	`QMessageBox.warning()`
Success action	`FileIO.save_project()`	`self._mcd_canvas.set_project()`
Post-action	`sys.exit(0)`	`self._mcd_canvas.zoom_fit()`

The parse and build lines are identical. Only the input and output handling differs.

15.7 Exercises

Exercise 15.1. Design a CLI for a hypothetical recipe DSL. What subcommands would you offer? (`parse`, `validate`, `nutrition`, `convert-to-metric`?) What exit codes would you define? What arguments would each subcommand accept?

Exercise 15.2. Add a `--strict` flag to the MSD CLI that treats warnings as errors. Write the implementation. Should `--strict` change the severity in error objects, or should it change how the error list is evaluated?

Exercise 15.3. Design a VS Code extension for MSD files. What features would you provide? (Syntax highlighting, error squiggles, hover information, auto-complete, go-to-definition?) Which features require just the lexer, which require the parser, and which require the builder?

Exercise 15.4. MSD uses lazy imports in `cmd_parse()`. Measure the import time of the MSD modules. Is lazy importing worthwhile? At what point does a module become expensive enough to justify lazy loading?

Exercise 15.5. Extend the integration pattern to include a **language server** (LSP). What would the language server wrapper look like? How would it differ from the CLI and GUI wrappers? What library features would it need that the current wrappers do not?

Exercise 15.6. Design a web-based import tool for MSD: a web page where users paste MSD text and see a rendered diagram. What components from the existing pipeline can you reuse? What new components are needed?

Appendix A: MSD Formal Grammar

This appendix defines the MSD grammar in Extended Backus-Naur Form (EBNF).

A.1 Notation

Symbol	Meaning
`=`	Definition
`|`	Alternative
`[ ... ]`	Optional (0 or 1)
`{ ... }`	Repetition (0 or more)
`( ... )`	Grouping
`"..."`	Terminal (literal string)
`UPPERCASE`	Token type
`lowercase`	Non-terminal

A.2 Grammar

```
(* Top level *)
file            = { top_level_decl } ;
top_level_decl  = project_block
                | entity_block
                | association_block
                | link_statement ;

(* Project metadata *)
project_block   = "project" "{" { project_prop } "}" ;
project_prop    = IDENTIFIER ":" STRING_VALUE ;

(* Entities *)
entity_block    = "entity" IDENTIFIER "{" { attribute } "}" ;

(* Associations *)
assoc_block     = "association" IDENTIFIER "{" { attribute } "}" ;
```

```
(* Attributes *)
attribute       = [ "*" ] IDENTIFIER ":" type_expr ;
type_expr       = IDENTIFIER [ "(" INTEGER ")" ] ;

(* Links *)
link_statement  = "link" IDENTIFIER cardinality IDENTIFIER ;
cardinality     = "(" card_min "," card_max ")" ;
card_min        = "0" | "1" ;
card_max        = "1" | "N" ;
```

A.3 Lexical Rules

```
(* Whitespace and comments *)
whitespace      = " " | "\t" ;
newline         = "\n" ;
comment         = "#" { any_char }
                | "//" { any_char } ;

(* Identifiers *)
IDENTIFIER      = ( letter | "_" ) { letter | digit | "_" } ;
letter          = "a" .. "z" | "A" .. "Z" ;
digit           = "0" .. "9" ;

(* Integers *)
INTEGER         = digit { digit } ;

(* String values — only inside project blocks *)
STRING_VALUE    = { any_char_except_comment_or_newline } ;

(* Keywords — matched case-insensitively *)
keyword         = "project" | "entity" | "association" | "link" ;
```

A.4 Semantic Constraints

The following constraints are not expressed in the grammar but are enforced by the parser and builder:

1. The `project` block may appear at most once
2. `type_expr` IDENTIFIER must be one of: `INT`, `BIGINT`, `SMALLINT`, `VARCHAR`, `CHAR`, `TEXT`, `BOOLEAN`, `DATE`, `TIME`, `TIMESTAMP`, `DECIMAL`, `FLOAT`, `DOUBLE`
3. Size parameter (`INTEGER`) is only meaningful for `VARCHAR`, `CHAR`, `DECIMAL`
4. `card_min` must be `"0"` or `"1"`
5. `card_max` must be `"1"` or `"N"`

6. Entity names must be unique
7. Association names must be unique and must not conflict with entity names
8. Entity and association names in link statements must reference declared entities or associations

A.5 Grammar Properties

The MSD grammar is:

- **LL(1)** — each production can be determined by looking at one token of lookahead
- **Context-free** — with the exception of the `STRING_VALUE` token in `project {}` blocks, which is context-sensitive at the lexical level
- **Unambiguous** — every valid input has exactly one parse tree

The LL(1) property makes the grammar ideal for recursive descent parsing. No backtracking is ever needed.

A.6 Token Summary

Token	Pattern	Example
`PROJECT`	`project` (case-insensitive)	`project`, `PROJECT`
`ENTITY`	`entity` (case-insensitive)	`entity`, `Entity`
`ASSOCIATION`	`association` (case-insensitive)	`association`
`LINK`	`link` (case-insensitive)	`link`, `LINK`
`LBRACE`	`{`	`{`
`RBRACE`	`}`	`}`
`LPAREN`	`(`	`(`
`RPAREN`	`)`	`)`
`COLON`	`:`	`:`
`COMMA`	`,`	`,`
`STAR`	`*`	`*`
`IDENTIFIER`	`[a-zA-Z_][a-zA-Z0-9_]*`	`Tourist`, `id`, `VARCHAR`
`INTEGER`	`[0-9]+`	`255`, `0`, `1`
`STRING_VALUE`	Rest of line after `:` in project block	`Tourism System`

Token	Pattern	Example
`NEWLINE`	End of line	(implicit)
`EOF`	End of input	(implicit)

Appendix B: MSD Data Type Reference

This appendix documents the 13 data types supported by MSD. Each type maps directly to a standard SQL data type.

B.1 Type Summary

Type	Category	Size Parameter	Description
`INT`	Numeric	No	Standard integer (32-bit)
`BIGINT`	Numeric	No	Large integer (64-bit)
`SMALLINT`	Numeric	No	Small integer (16-bit)
`DECIMAL`	Numeric	Optional	Fixed-point decimal
`FLOAT`	Numeric	No	Single-precision floating point
`DOUBLE`	Numeric	No	Double-precision floating point
`VARCHAR`	String	Required	Variable-length character string
`CHAR`	String	Optional	Fixed-length character string
`TEXT`	String	No	Unlimited-length text
`BOOLEAN`	Logical	No	True/false value
`DATE`	Temporal	No	Calendar date (year, month, day)
`TIME`	Temporal	No	Time of day (hour, minute, second)
`TIMESTAMP`	Temporal	No	Date and time combined

B.2 Numeric Types

INT

Standard 32-bit signed integer. Range: -2,147,483,648 to 2,147,483,647.

```
*id: INT
age: INT
quantity: INT
```

Use `INT` as the default integer type. It is the most common choice for primary keys, counters, and general-purpose integer fields.

BIGINT

64-bit signed integer. Range: -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807.

```
*id: BIGINT
population: BIGINT
```

Use `BIGINT` when values may exceed the `INT` range — for example, auto-incrementing primary keys in high-volume tables, or fields that store large counts.

SMALLINT

16-bit signed integer. Range: -32,768 to 32,767.

```
rating: SMALLINT
priority: SMALLINT
```

Use `SMALLINT` for fields with a known small range. It saves storage compared to `INT` but offers limited range.

DECIMAL

Fixed-point decimal number. Accepts an optional size parameter specifying precision.

```
price: DECIMAL
salary: DECIMAL(10)
```

Use `DECIMAL` for monetary values and any field where exact decimal representation matters. Floating-point types (`FLOAT`, `DOUBLE`) introduce rounding errors that are unacceptable for financial data.

FLOAT

Single-precision (32-bit) IEEE 754 floating-point number. Approximately 7 significant decimal digits.

```
latitude: FLOAT
temperature: FLOAT
```

Use `FLOAT` for scientific measurements and approximate values where storage efficiency matters more than precision.

DOUBLE

Double-precision (64-bit) IEEE 754 floating-point number. Approximately 15 significant decimal digits.

```
longitude: DOUBLE
distance: DOUBLE
```

Use `DOUBLE` when `FLOAT` does not provide sufficient precision. Prefer `DECIMAL` over `DOUBLE` for financial calculations.

B.3 String Types

VARCHAR

Variable-length character string. Requires a size parameter specifying the maximum length in characters.

```
name: VARCHAR(255)
email: VARCHAR(320)
code: VARCHAR(10)
```

`VARCHAR` is the most commonly used string type. Always specify a size — it documents the expected maximum length and enables database-level validation.

Tip

Common sizes: `VARCHAR(255)` for general text fields, `VARCHAR(320)` for email addresses (per RFC 5321), `VARCHAR(50)` for short codes and abbreviations.

CHAR

Fixed-length character string. Accepts an optional size parameter (defaults to 1 if omitted).

```
gender: CHAR(1)
country_code: CHAR(2)
currency: CHAR(3)
```

Use `CHAR` for fields with a known, fixed length — such as ISO country codes (`CHAR(2)`), currency codes (`CHAR(3)`), or single-character flags (`CHAR(1)`).

Note

`CHAR` values are padded with spaces to the specified length. For variable-length data, prefer `VARCHAR`.

TEXT

Unlimited-length text. No size parameter.

```
biography: TEXT
description: TEXT
notes: TEXT
```

Use `TEXT` for long-form content where no maximum length can reasonably be specified — such as descriptions, comments, or document bodies.

B.4 Logical Types

BOOLEAN

True/false value.

```
is_active: BOOLEAN
verified: BOOLEAN
published: BOOLEAN
```

Use `BOOLEAN` for binary flags and yes/no fields.

B.5 Temporal Types

DATE

Calendar date (year, month, day). No time component.

```
birth_date: DATE
hire_date: DATE
```

Use `DATE` for fields that represent calendar dates without a time-of-day component.

TIME

Time of day (hour, minute, second). No date component.

```
opening_time: TIME
departure_time: TIME
```

Use `TIME` for fields that represent a time of day independently of any specific date.

TIMESTAMP

Combined date and time, typically with timezone awareness.

```
created_at: TIMESTAMP
updated_at: TIMESTAMP
last_login: TIMESTAMP
```

Use `TIMESTAMP` for audit fields, event times, and any field that records when something happened.

Tip

Prefer `TIMESTAMP` over separate `DATE` and `TIME` fields when both components are needed. It avoids the complexity of joining two fields and ensures atomic date-time values.

B.6 Size Parameter Rules

Only three types accept a size parameter:

Type	Size Parameter	Meaning	Example
`VARCHAR`	Required	Maximum character length	`VARCHAR(255)`
`CHAR`	Optional	Fixed character length (default 1)	`CHAR(3)`
`DECIMAL`	Optional	Precision (total digits)	`DECIMAL(10)`

If a size parameter is specified on any other type, the parser emits a warning:

```
schema.msd:3: warning: size parameter ignored for type 'INT'
```

The warning is non-fatal — the model is still built, but the size value is discarded.

B.7 Type Selection Guide

Use Case	Recommended Type
Primary keys	`INT` or `BIGINT`
Short text (names, codes)	`VARCHAR(n)`
Long text (descriptions)	`TEXT`
Fixed-length codes	`CHAR(n)`
Money and prices	`DECIMAL`
Scientific measurements	`FLOAT` or `DOUBLE`
Yes/no flags	`BOOLEAN`
Calendar dates	`DATE`
Event timestamps	`TIMESTAMP`
Time-of-day values	`TIME`
Counters and quantities	`INT`
Small-range integers	`SMALLINT`
Large-range integers	`BIGINT`

Appendix C: DSL Design Checklist

This appendix provides a step-by-step checklist for designing and implementing a new domain-specific language from scratch. Each phase builds on the previous one. The chapter references point to where each topic is covered in detail.

Phase 1: Domain Analysis (Chapter 3)

- ☐ Identify the target domain and its boundaries
- ☐ List the core domain concepts (nouns: entities, objects, resources)
- ☐ List the core domain operations (verbs: create, link, transform, validate)
- ☐ List the relationships between concepts (one-to-one, one-to-many, many-to-many)
- ☐ List the constraints that the domain imposes (uniqueness, cardinality, type restrictions)
- ☐ Identify the target users — who will write in this language?
- ☐ Identify the target tools — what will consume the language output?
- ☐ Study existing solutions — what tools do users currently use?
- ☐ Document what works and what is painful about current solutions
- ☐ Define the scope — what is explicitly *not* part of the language?

Tip

Scope is the most important decision. A DSL that tries to do everything becomes a poor general-purpose language. Define what is out of scope before designing what is in scope.

Phase 2: Lexical Design (Chapter 4)

- ☐ Choose keywords — one per major concept, short and familiar
- ☐ Decide on case sensitivity (keywords, identifiers, or both)
- ☐ Choose identifier rules — what characters are allowed in names?
- ☐ Choose literal types — strings, integers, floats, booleans?
- ☐ Choose symbol tokens — braces, parentheses, colons, commas, special markers
- ☐ Choose comment syntax — line comments, block comments, or both?

- ☐ Decide on whitespace significance — is indentation meaningful?
- ☐ Decide on line significance — are newlines meaningful?
- ☐ Identify any context-sensitive lexing needs
- ☐ Verify that no keyword conflicts with likely user identifiers
- ☐ Write down the complete token type list

Warning

Avoid choosing keywords that are common domain terms. If users frequently need to name things "type" or "name", do not make those keywords.

Phase 3: Syntactic Design (Chapter 5)

- ☐ Write the grammar in EBNF notation
- ☐ Verify the grammar is unambiguous — every valid input has exactly one parse tree
- ☐ Verify the grammar is LL(1) — each rule can be determined by one token of lookahead
- ☐ Choose the top-level structure — file as a sequence of declarations?
- ☐ Choose the block structure — braces, indentation, or keywords (begin/end)?
- ☐ Choose delimiter style — commas, semicolons, newlines, or nothing?
- ☐ Decide on optional elements — what can be omitted?
- ☐ Decide on ordering constraints — must declarations appear in a specific order?
- ☐ Write at least three example files covering simple, typical, and complex cases
- ☐ Read the example files aloud — do they sound natural?
- ☐ Show the example files to a potential user — can they guess the meaning?

Phase 4: Semantic Design (Chapter 6)

- ☐ Define the type system — what types exist? Are they parameterised?
- ☐ Define name binding rules — where are names declared? Where are they referenced?
- ☐ Define scoping rules — are names global or local?
- ☐ Define uniqueness constraints — what names must be unique?
- ☐ Define reference resolution — how are forward references handled?
- ☐ Define validation rules — what constraints beyond syntax must be checked?

- ☐ Classify each validation rule as error (fatal) or warning (informational)
- ☐ Design error messages for each validation rule — specific, actionable, with context
- ☐ Consider "did you mean?" suggestions for name resolution failures
- ☐ Document the complete semantic specification

Phase 5: Implementation (Chapters 8–12)

Lexer (Chapter 8)

- ☐ Define the `Token` data structure (type, value, line, column)
- ☐ Implement the tokeniser — line-by-line or character-by-character
- ☐ Handle keyword recognition (identifier-to-keyword mapping)
- ☐ Handle comments (strip or skip)
- ☐ Handle whitespace and newlines
- ☐ Handle context-sensitive tokens (if any)
- ☐ Report lexical errors with line and column numbers
- ☐ Test: empty input, whitespace only, comments only
- ☐ Test: every keyword, every symbol, every literal type
- ☐ Test: invalid characters (recovery, not crash)

Parser (Chapter 9)

- ☐ Choose the parsing technique — recursive descent for LL(1) grammars
- ☐ Define the intermediate representation (parse result, not final output)
- ☐ Implement one function per grammar rule
- ☐ Implement token navigation helpers (peek, advance, expect, skip)
- ☐ Implement panic-mode error recovery with synchronisation points
- ☐ Accumulate errors from the lexer and add parser errors
- ☐ Test: every grammar rule with valid input
- ☐ Test: every error case (missing tokens, invalid values)
- ☐ Test: error recovery — valid constructs after errors are still parsed

Semantic Analyser / Builder (Chapter 10)

- ☐ Implement name registration and duplicate detection
- ☐ Implement reference resolution with error reporting
- ☐ Implement "did you mean?" suggestions (Levenshtein distance)
- ☐ Implement type validation
- ☐ Implement all semantic constraints
- ☐ Separate errors from warnings
- ☐ Accumulate errors from the parser and add builder errors
- ☐ Test: every validation rule (both success and failure)
- ☐ Test: duplicate names, unknown references, type errors
- ☐ Test: suggestion accuracy for common typos

Output Generation (Chapter 11)

- ☐ Define the output format (JSON, XML, SQL, code, etc.)
- ☐ Implement the transformation from intermediate representation to output
- ☐ Generate stable identifiers (UUIDs) for persistent references
- ☐ Apply metadata and defaults
- ☐ Test: round-trip fidelity (parse → build → save → load → verify)

Layout (Chapter 12, if applicable)

- ☐ Choose a layout algorithm appropriate to the domain
- ☐ Implement deterministic layout (fixed random seed)
- ☐ Implement post-processing (overlap removal, centering, rounding)
- ☐ Test: positions are assigned, no overlaps, deterministic results

Phase 6: Integration (Chapters 13–15)

Error Handling (Chapter 13)

- ☐ Use structured error objects, not exceptions, for user-facing errors
- ☐ Include location (file, line, column) in every error
- ☐ Use compiler-style error format (`file:line: severity: message`)
- ☐ Implement multi-error reporting — never stop at the first error
- ☐ Review every error message for specificity, actionability, and context

Testing (Chapter 14)

- ☐ Test each layer independently (lexer, parser, builder)
- ☐ Test both success and failure paths for every feature
- ☐ Test the contract (observable behaviour), not the implementation
- ☐ Write integration tests that exercise the complete pipeline
- ☐ Aim for at least one positive and one negative test per language feature
- ☐ Ensure tests run fast (seconds, not minutes)

Tool Integration (Chapter 15)

- ☐ Build the core as a library with no I/O dependencies
- ☐ Build a CLI wrapper — subcommands, exit codes, stderr for errors
- ☐ Build a GUI wrapper (if needed) — file dialogs, message boxes
- ☐ Provide `--help` and `--version` flags
- ☐ Write man pages or help documentation
- ☐ Test batch processing and CI pipeline usage

Summary

Phase	Key Deliverable
1. Domain Analysis	Scope document, concept list, constraint list
2. Lexical Design	Token type list, keyword list
3. Syntactic Design	EBNF grammar, example files
4. Semantic Design	Validation rules, error message catalogue
5. Implementation	Working lexer, parser, builder, output generator
6. Integration	CLI tool, test suite, documentation

Appendix D: Glossary

General DSL Terms

Abstract Syntax Tree (AST) A tree representation of the syntactic structure of source code. Each node represents a construct in the language. In simpler DSLs, a flat intermediate representation may be used instead of a full tree.

Ambiguity A property of a grammar where a single input can be parsed in more than one way, producing multiple valid parse trees. Ambiguous grammars should be avoided in DSL design.

Backtracking A parsing technique where the parser tries one alternative, and if it fails, reverses its progress and tries another. LL(1) grammars do not require backtracking.

Block Structure A syntactic pattern where related declarations are grouped inside delimiters, typically braces (`{ }`), keywords (`begin`/`end`), or indentation.

Builder The semantic analysis component that transforms a parser's intermediate representation into the final output. It performs validation, name resolution, and object construction.

Cascade Error A false error caused by the parser's recovery from a previous genuine error. Cascade errors are a common problem in multi-error reporting.

Case Sensitivity Whether uppercase and lowercase letters are treated as distinct. A language may be case-sensitive for identifiers but case-insensitive for keywords.

Comment Text in source code that is ignored by the lexer. Common styles include line comments (`#`, `//`) and block comments (`/* */`).

Context-Free Grammar A grammar where every production rule can be applied regardless of context. Most programming languages and DSLs are defined by context-free grammars.

Context-Sensitive Lexing A lexing technique where the tokenisation of a character sequence depends on the surrounding context. For example, a colon inside a project block may trigger a different token type than a colon elsewhere.

Domain-Specific Language (DSL) A programming language designed for a specific application domain, as opposed to a general-purpose language. Examples: SQL (databases), CSS (styling), Make (build automation).

EBNF (Extended Backus-Naur Form) A notation for defining context-free grammars. Extends BNF with repetition (`{ }`), optionality (`[ ]`), and grouping (`( )`).

Error Recovery The ability of a parser to continue parsing after encountering a syntax error, enabling it to report multiple errors in a single pass.

Exit Code A numeric value returned by a command-line program to indicate success (0) or failure (non-zero). Different exit codes can distinguish between different failure modes.

External DSL A DSL with its own syntax, lexer, and parser, independent of any host language. MSD is an external DSL. Contrast with *internal DSL*.

Forward Reference A reference to a name that is declared later in the source. Some DSLs support forward references; others require declaration before use.

General-Purpose Language (GPL) A programming language designed for a wide variety of tasks, such as Python, Java, Go, or C. Contrast with *domain-specific language*.

Identifier A name given by the user to a concept in the language — such as an entity name, attribute name, or association name. Typically matches the pattern `[a-zA-Z_][a-zA-Z0-9_]*`.

Intermediate Representation (IR) A data structure produced by the parser that captures the syntactic content of the input. The builder transforms the IR into the final output. Using an IR decouples parsing from semantic analysis.

Internal DSL A DSL implemented within a host language, using the host's syntax. Ruby's Rake and Kotlin's Gradle scripts are internal DSLs. Contrast with *external DSL*.

Keyword A reserved word in a language with a predefined meaning. In MSD, the keywords are `project`, `entity`, `association`, and `link`.

Levenshtein Distance A measure of the difference between two strings, defined as the minimum number of single-character insertions, deletions, or substitutions needed to transform one string into the other. Used for "did you mean?" suggestions.

Lexer (Tokeniser) The component that reads source text and produces a sequence of tokens. The lexer handles whitespace, comments, keywords, identifiers, literals, and symbols.

LL(1) A class of grammars that can be parsed by reading input left-to-right, producing a leftmost derivation, with one token of lookahead. LL(1) grammars are ideal for recursive descent parsers.

Lookahead The number of tokens a parser examines ahead of the current position to decide which grammar rule to apply. LL(1) parsers use a single token of lookahead.

Multi-Error Reporting The ability to report all errors in a single pass, rather than stopping at the first error. Requires error recovery in the parser.

Newline Significance A design choice where newline characters carry syntactic meaning — for example, separating statements or attributes. Contrast with languages where newlines are mere whitespace.

Panic Mode An error recovery strategy where the parser, upon encountering an unexpected token, skips tokens until it reaches a synchronisation point, then resumes normal parsing.

Parse Result The intermediate representation produced by the parser. Contains the parsed declarations (entities, associations, links) and any errors or warnings accumulated during parsing.

Parser The component that reads a token stream and produces a structured representation (AST or IR) according to the grammar rules.

Production Rule A rule in a grammar that defines how a non-terminal can be expanded into a sequence of terminals and non-terminals. For example: `entity_block = "entity" IDENTIFIER "{" { attribute } "}"`.

Recursive Descent Parser A top-down parsing technique where each non-terminal in the grammar is implemented as a function. The function calls other functions corresponding to the non-terminals in its production rule.

Semantic Analysis The phase after parsing that checks meaning-level constraints: name resolution, type checking, uniqueness validation, and reference integrity.

Severity The classification of a diagnostic message. Common levels: *error* (fatal — the tool cannot produce correct output) and *warning* (informational — the tool can proceed).

Synchronisation Point A token or set of tokens used by panic-mode recovery to determine where normal parsing can resume. Typical synchronisation points include keywords, closing braces, and statement delimiters.

Token The smallest meaningful unit produced by the lexer. Each token has a type (e.g., `IDENTIFIER`, `LBRACE`), a value (e.g., `"Tourist"`, `"{"`), and a position (line and column).

Tokeniser See *Lexer*.

MSD-Specific Terms

Association A named relationship between two or more entities in a MERISE conceptual data model. Declared with the `association` keyword. May carry its own attributes.

Attribute A named property of an entity or association, with a data type. Declared as `name: TYPE` or `*name: TYPE` (for primary keys).

Cardinality The constraint on how many instances of one entity can participate in an association. Expressed as a pair `(min, max)` where min is `0` or `1` and max is `1` or `N`.

Entity A named concept in a MERISE conceptual data model, representing a distinct class of objects. Declared with the `entity` keyword. Contains attributes, at least one of which should be a primary key.

Link A statement connecting an entity to an association with a cardinality constraint. Declared as `link EntityName (min,max) AssociationName`.

MCD (Modele Conceptuel de Donnees) The Conceptual Data Model in the MERISE methodology. Consists of entities, associations, and links with cardinalities.

MSD describes MCDs in text form.

MERISE A French information systems design methodology developed in the late 1970s. Widely used in French-speaking countries for database design. Distinguishes conceptual (MCD), logical (MLD), and physical models.

Merisio A visual MERISE modelling tool. MSD is the text-based input format for Merisio. The tool provides both a GUI (visual editor) and a CLI (`merisio-cli`) for headless processing.

MLD (Modele Logique de Donnees) The Logical Data Model in the MERISE methodology. Derived from the MCD by applying transformation rules (e.g., resolving many-to-many relationships into junction tables).

MSD (Merisio Schema Definition) A domain-specific language for describing MERISE conceptual data models in plain text. Features 4 keywords, 13 data types, and an LL(1) grammar.

`.merisio` File The JSON project file format used by Merisio. Contains the complete model (entities, associations, links, layout positions, metadata). Produced by the MSD pipeline or by saving from the GUI.

`.msd` File A plain text file containing MSD source code. Processed by the MSD pipeline (lexer → parser → builder) to produce a `.merisio` project file.

Primary Key An attribute (or set of attributes) that uniquely identifies an instance of an entity. Marked with the * prefix in MSD. An entity without a primary key triggers a warning.

Project Block An optional metadata section in an MSD file, declared with the `project` keyword. Contains key-value pairs such as `name`, `author`, and `description`.

`STRING_VALUE` A context-sensitive token type used inside project blocks. Captures the rest of the line after a colon, allowing unquoted text values (e.g., `name: Tourism System`).